

Artificial Intelligence – Deep Learning fundamentals

Master in Artificial Intelligence & Computational Engineering
Luis Montesano



Universidad
Zaragoza

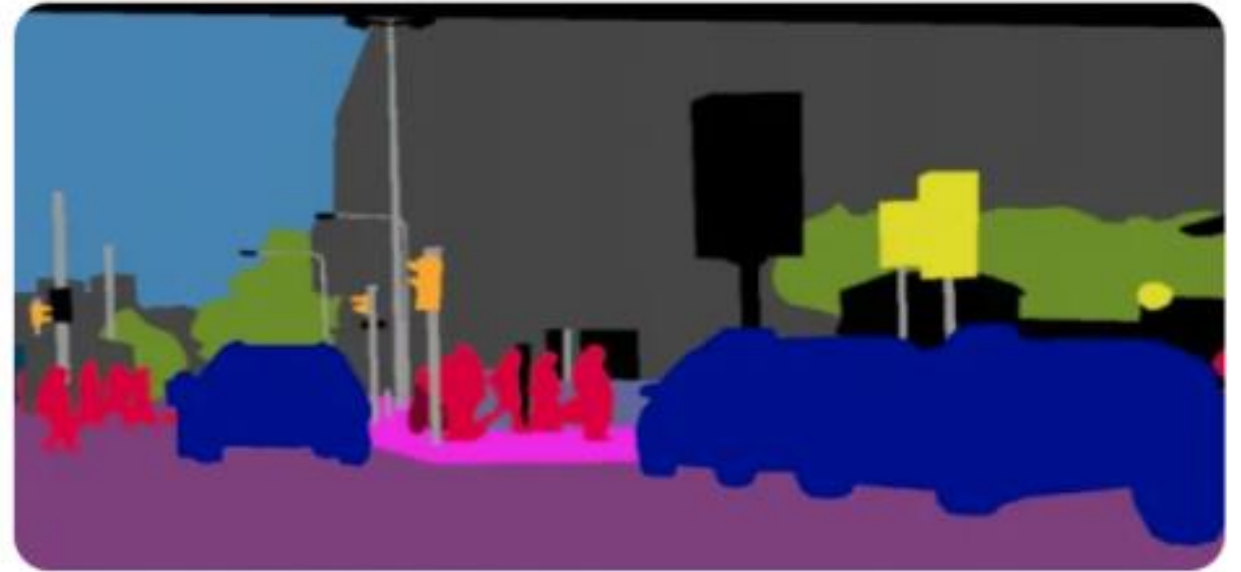
Today

- What's a CNN?
- Well-known CNN architectures
- Transfer learning
- Training diagnostics
- Other tasks (detection, segmentation, regression, video)

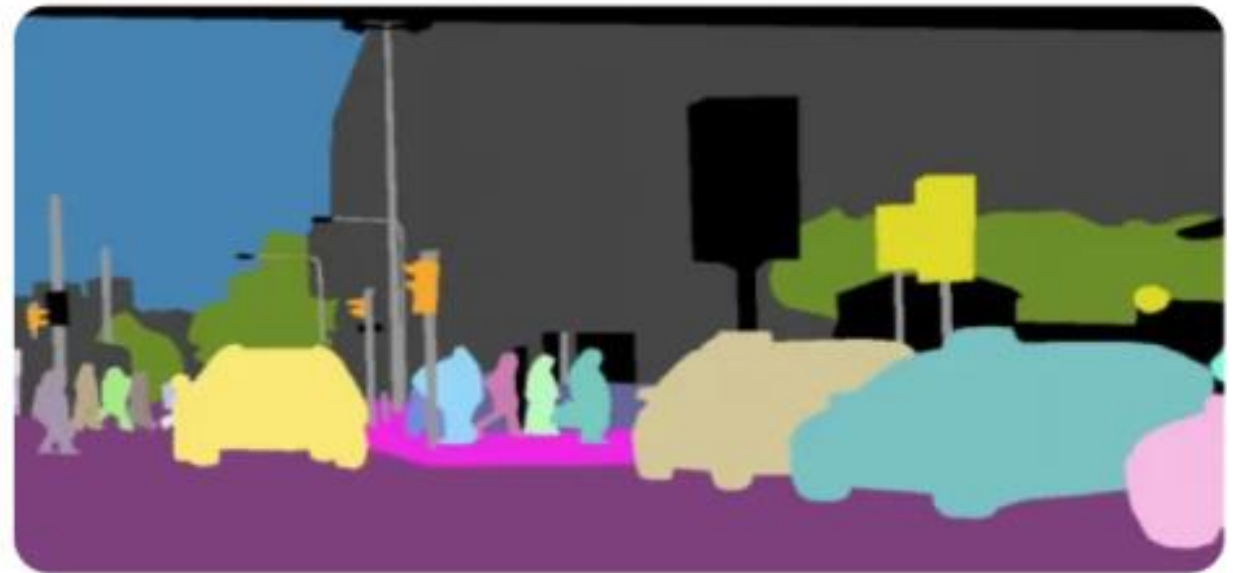
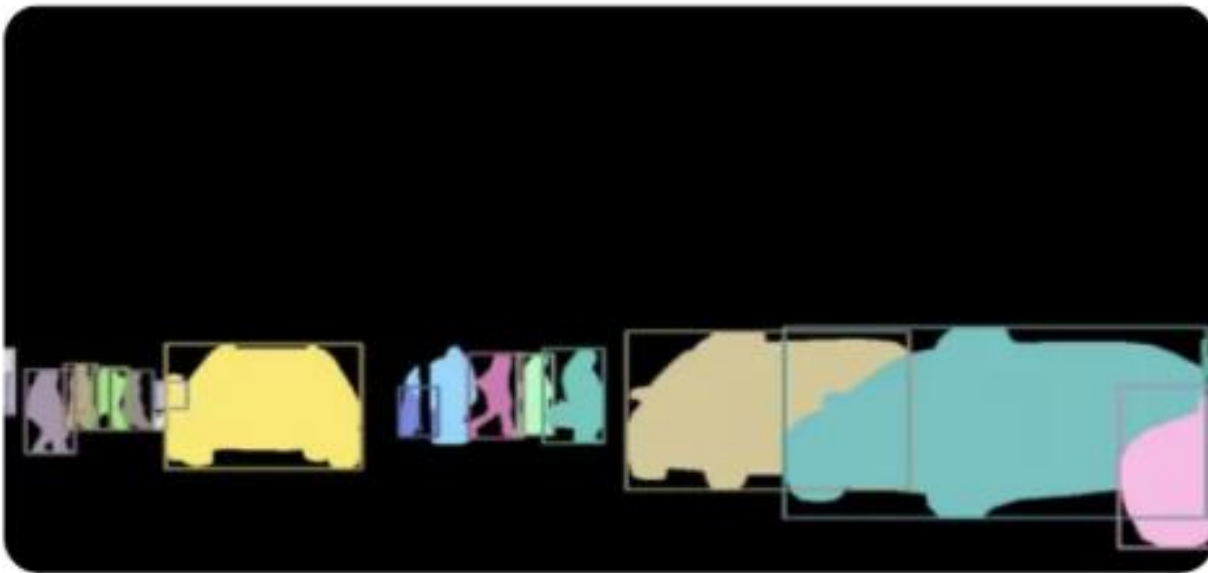
Computer vision



(a) image



(b) semantic segmentation



Networks for images

- Can you think of limitations or issues with fully-connected networks?

Networks for images

- Can you think of limitations or issues with fully-connected networks?
 1. Size
 - 224x224 RGB image = 150,528 dimensions
 - Hidden layers generally larger than inputs
 - One hidden layer = 150,528x150,528 weights aprx 22.7 billion
 2. Nearby pixels statistically related
 - But could permute pixels and relearn and get same results with FC
 3. Should be stable under transformations
 - Don't want to re-learn appearance at different parts of image

Networks for images

- Can you think of desirable properties for image processing?

Networks for images

- Can you think of desirable properties for image processing?

Living room



Living room



Networks for images

- Can you think of desirable properties for image processing?

Living room



Living room

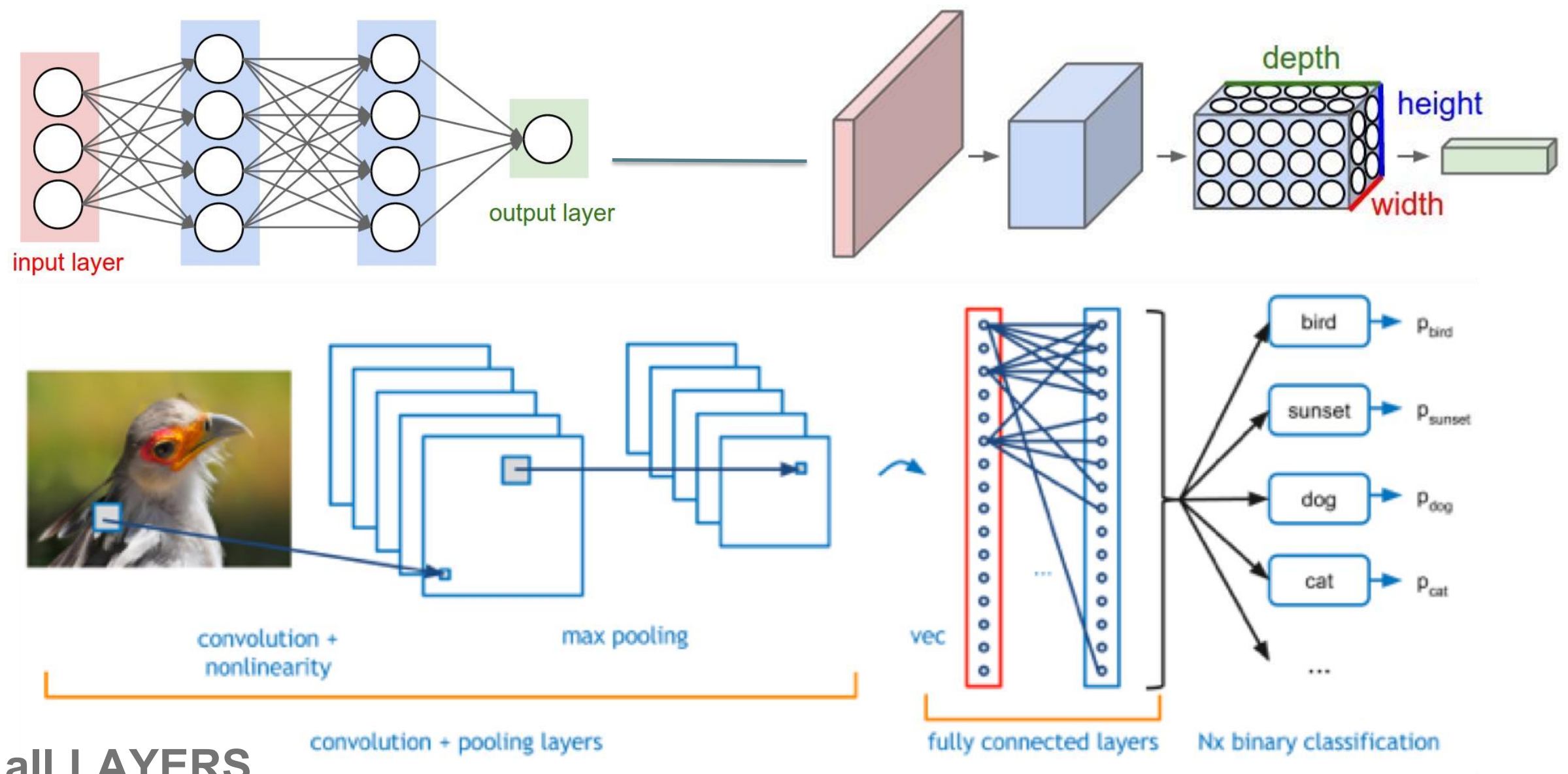


Invariance

Equivariance

CNNs

- Convolutional (deep) Neural Networks (CNN)



**NOT all LAYERS
are the same**

CNNs - Basic Layers

- **Input:** raw pixel values. RGB image 32x32, volume [32x32x3]
- **Convolutional:** compute output of neurons connected to local input regions (each computes dot product between their weights and the region). Larger volume [32x32x12].
- **RELU:** element-wise activation function (e.g. thresholding at zero). Volume unchanged ([32x32x12]).
- **Pooling:** downsampling operation. e.g. reduce volume to [16x16x12].
- **Fully-connected:** compute class scores. Each neuron in this layer will be connected to all the numbers in the previous volume. Volume of size [1x1x10].

Convolutional layer

- Look only at local image patches (at each level)
- Share parameters across the image

1D Convolution

- Output is just a weighted sum of neighbour elements in a vector

$$\mathbf{x} = [x_1, x_2, \dots, x_I]$$

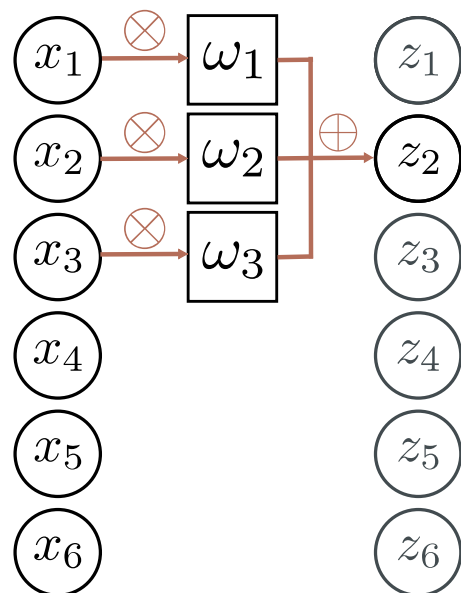
$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

$$\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^T$$

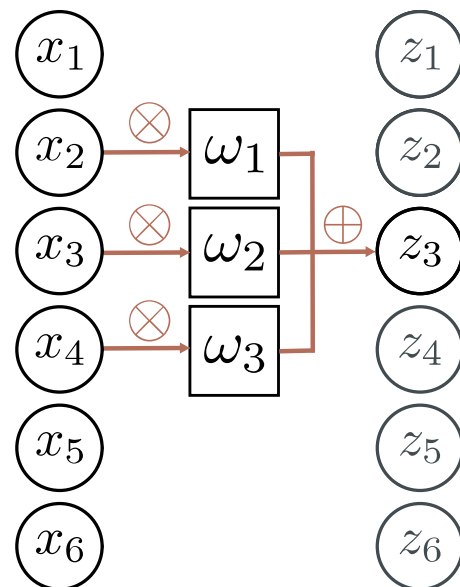
- The parameter $\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^T$ is called a kernel, in this case with size 3
- We can then add a bias and an activation to make it a layer

Visual interpretation of kernel size 3 convolution

a)

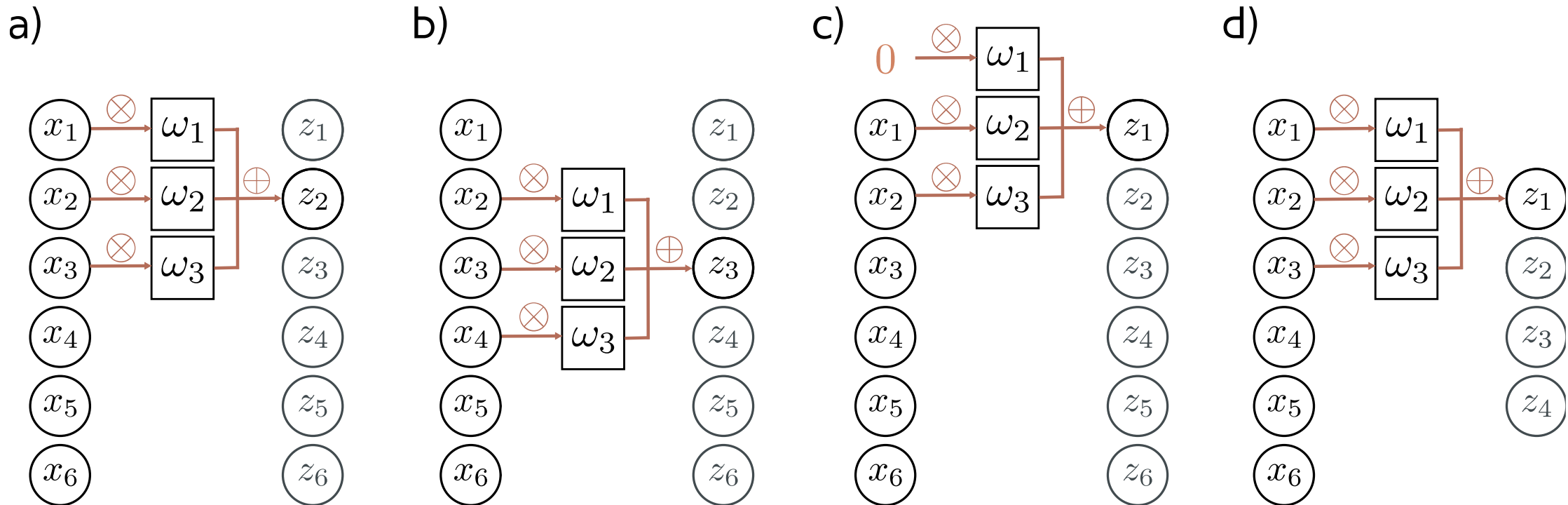


b)



- Slide the kernel over the vector and compute the weighted sum at each position

Visual interpretation of kernel size 3 convolution

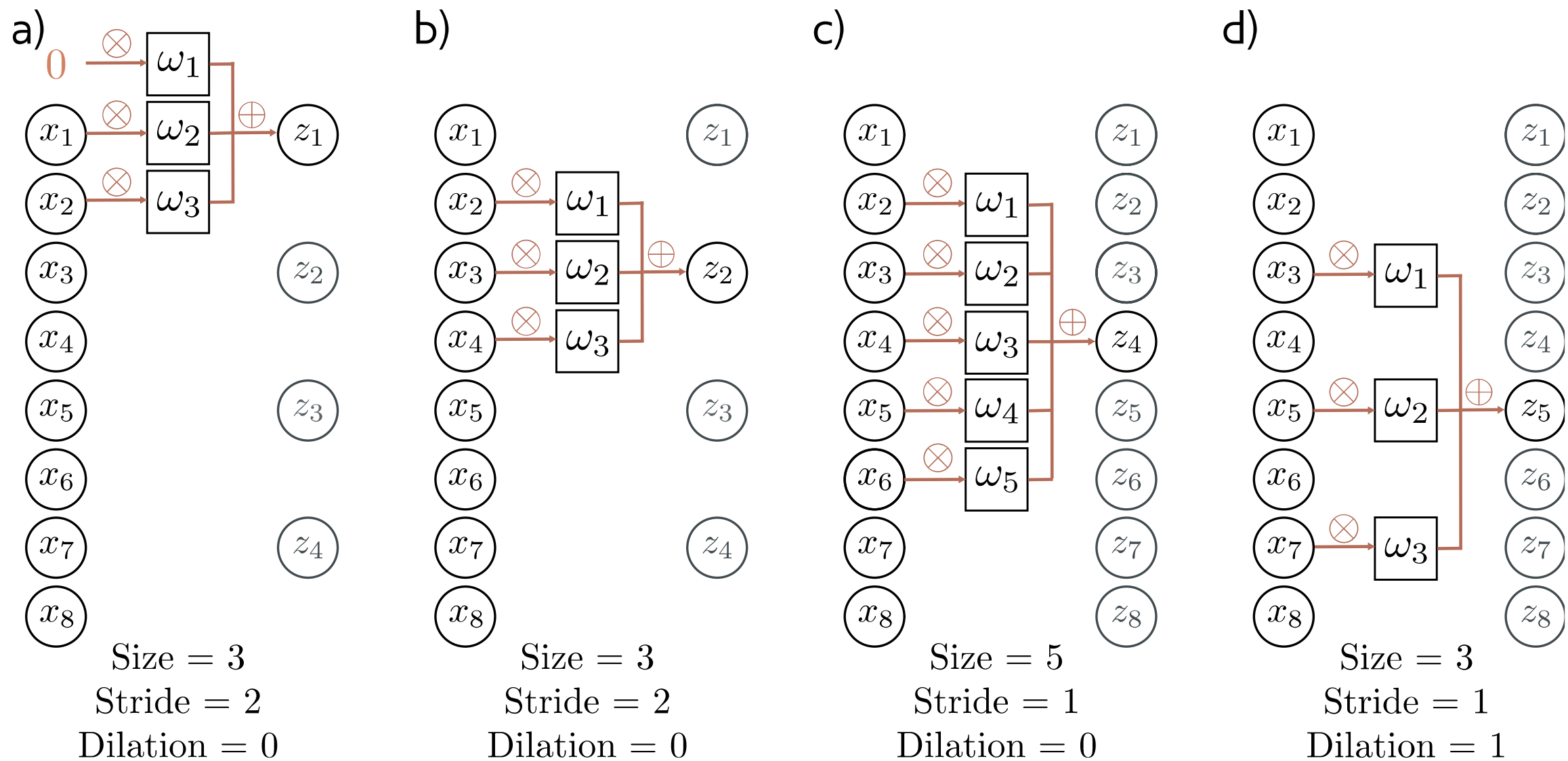


- Slide the kernel over the vector and compute the weighted sum at each position
- At the edges, either pad or diminish the number of outputs

Stride, kernel size and dilation

- Stride = shift by k positions for each output
 - Decreases size of output relative to input
- Kernel size = weight a different number of inputs for each output
 - Combine information from a larger area
 - But kernel size 5 uses 5 parameters
- Dilated or atrous convolutions = intersperse kernel values with zeros
 - Combine information from a larger area
 - Fewer parameters

Stride, kernel size and dilation



PyTorch and Keras use dilation = 1 for *no* dilation, take into account in the lab.

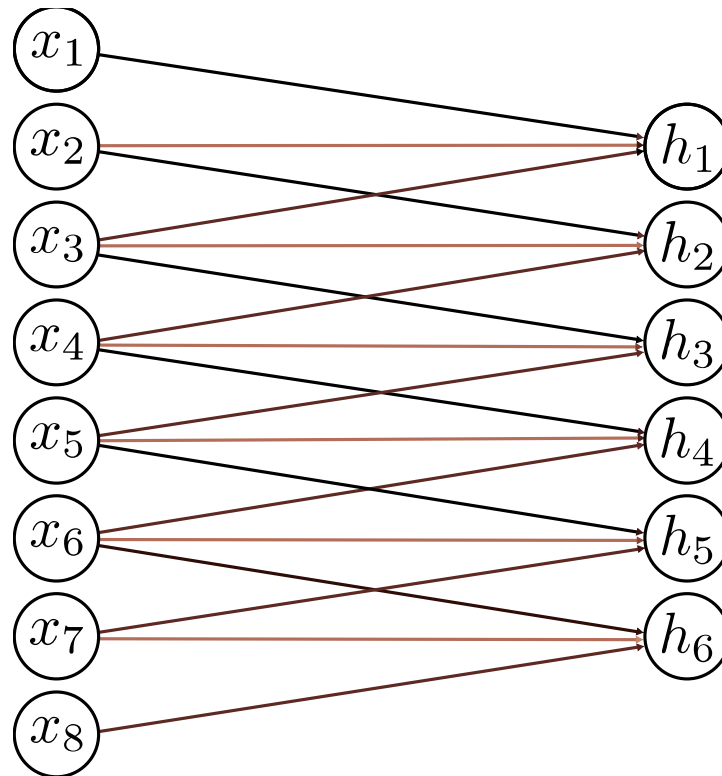
Output size

$$\text{Output size} = \lfloor (W - K + 2P) / S \rfloor + 1$$

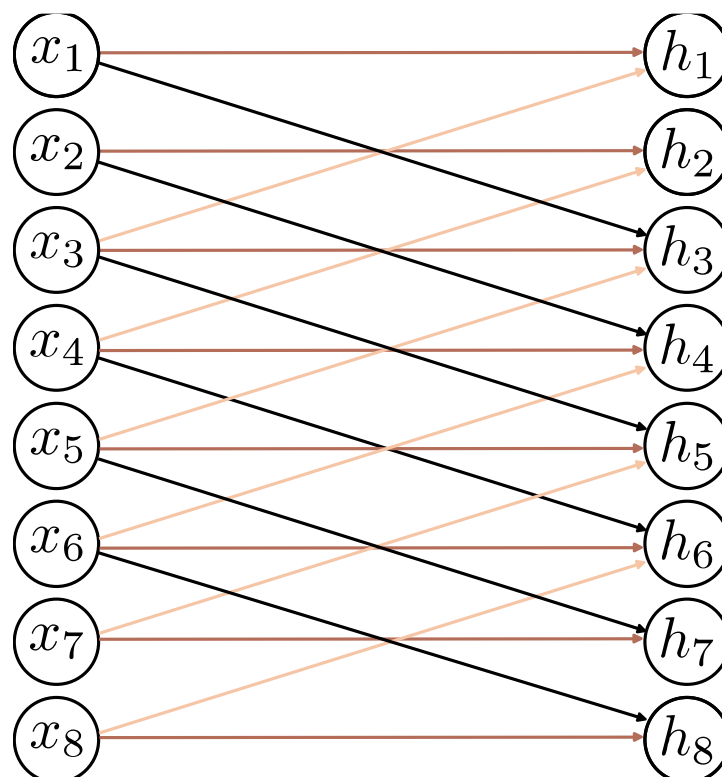
- W is the input size along one spatial dimension (width or height), e.g. 240 for a 240×240 image
- K is the kernel (filter) size, e.g. 3 for a 3×3 kernel
- P is the padding: how many zeros are added on *each* side of the input. That's why it appears as $2P$
- S is the stride: how many positions the kernel moves between outputs
- **With dilation:** replace K by the effective size $D(K - 1) + 1$. In PyTorch/Keras, $D = 1$ means no dilation
- **Check with slide 16:**
 - (a) 8 inputs, $K = 3$, $S = 2$, $P = 1$: $\lfloor 7/2 \rfloor + 1 = 4$ outputs
 - (c) $K = 5$, $S = 1$, $P = 2$: 8 outputs, so the size is kept ("same")
 - (d) $K = 3$, $D = 2$ gives an effective kernel of 5

Questions

- Kernel size?
- Stride?
- Dilation?
- Zero padding / valid?



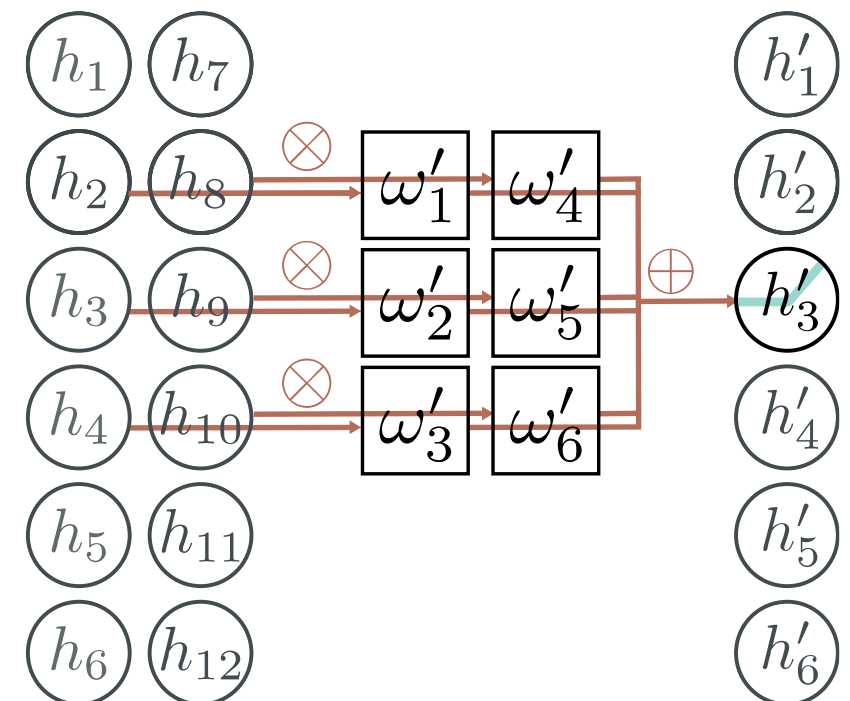
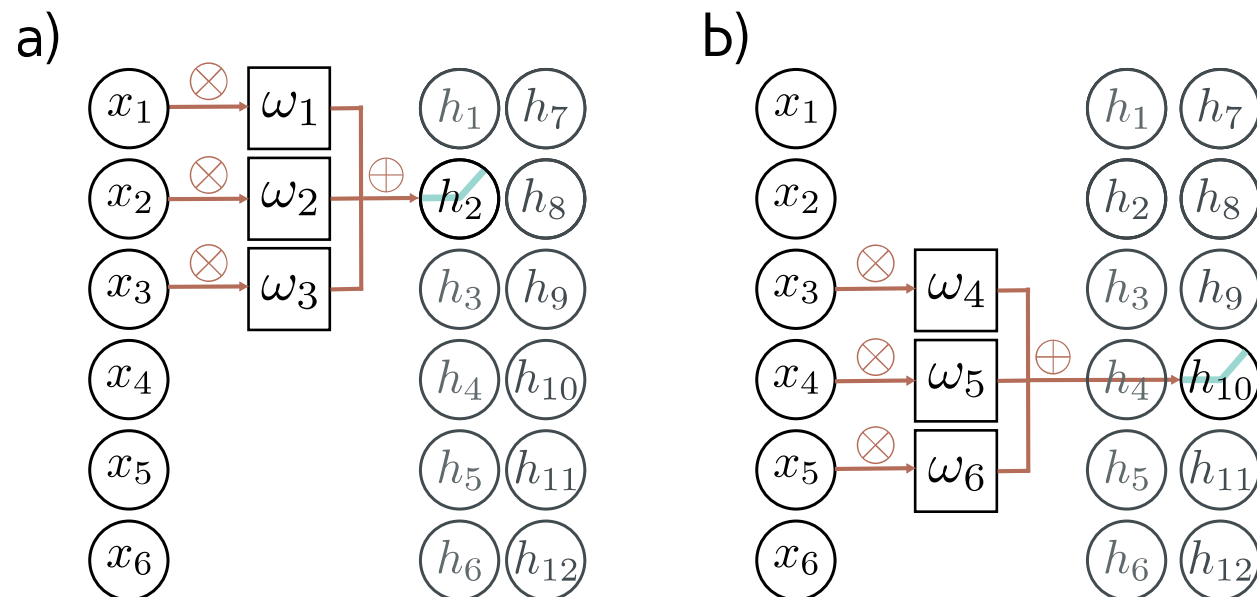
	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
h_1								
h_2								
h_3								
h_4								
h_5								
h_6								



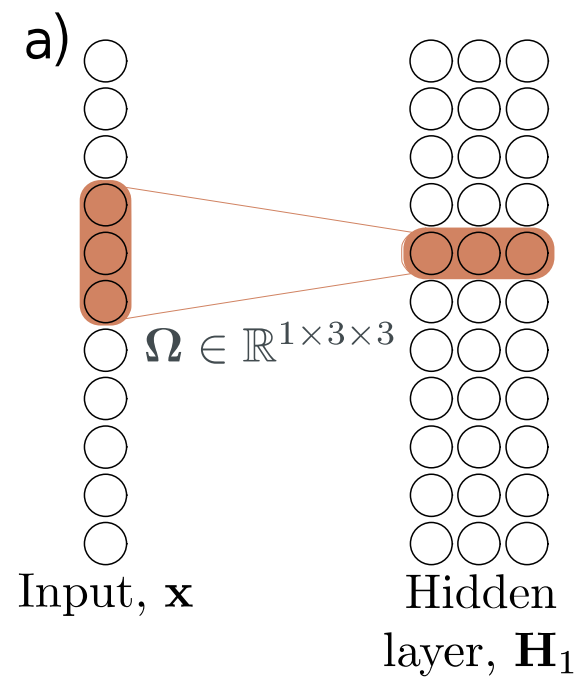
	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
h_1								
h_2								
h_3								
h_4								
h_5								
h_6								
h_7								
h_8								

Channels (or feature maps)

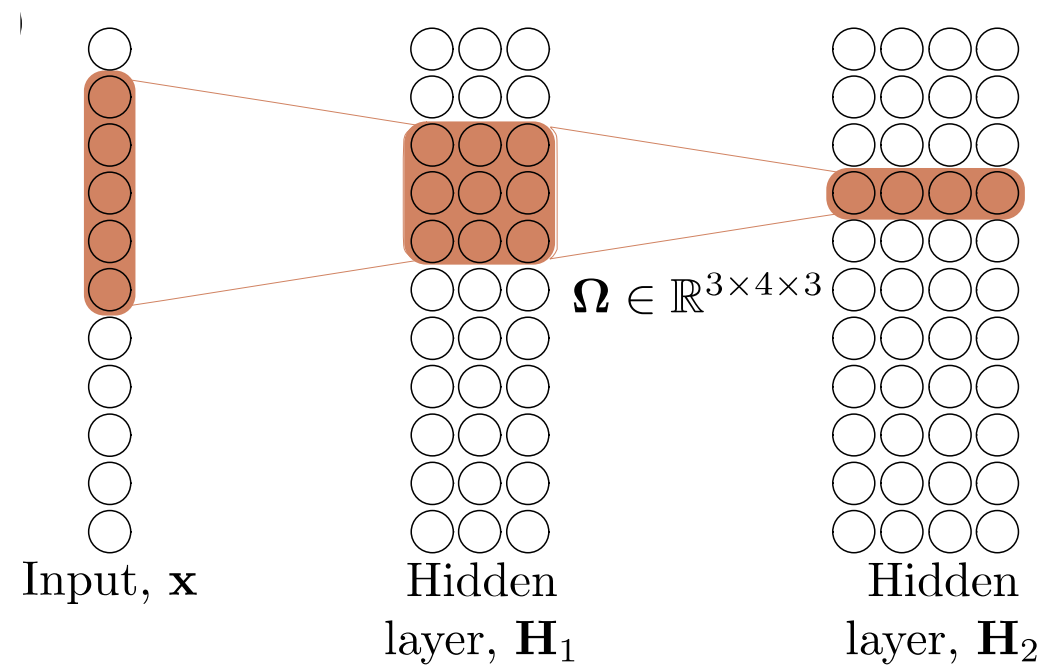
- Loss of information (hopefully finding some interesting feature)
- We need multiple of them to combine them later
- Apply several convolutions to compute multiple outputs called channels or feature maps



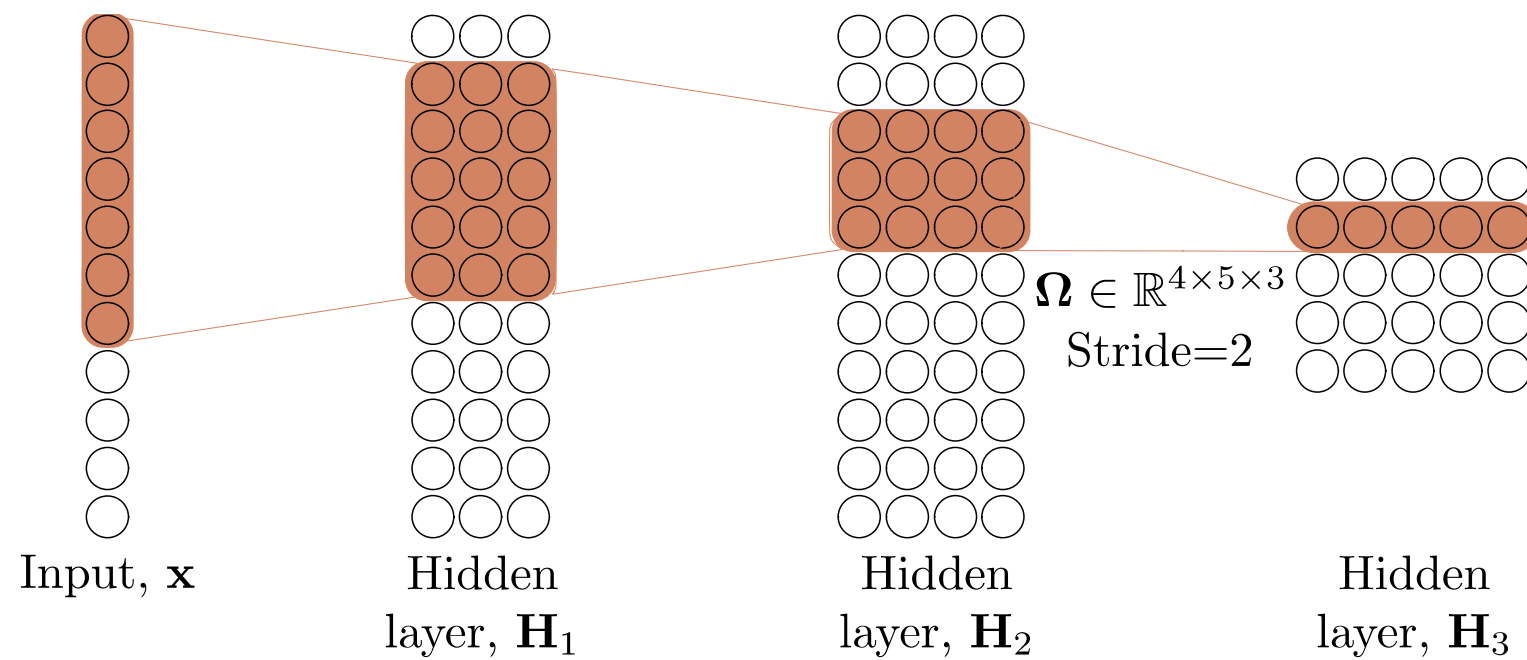
Receptive fields



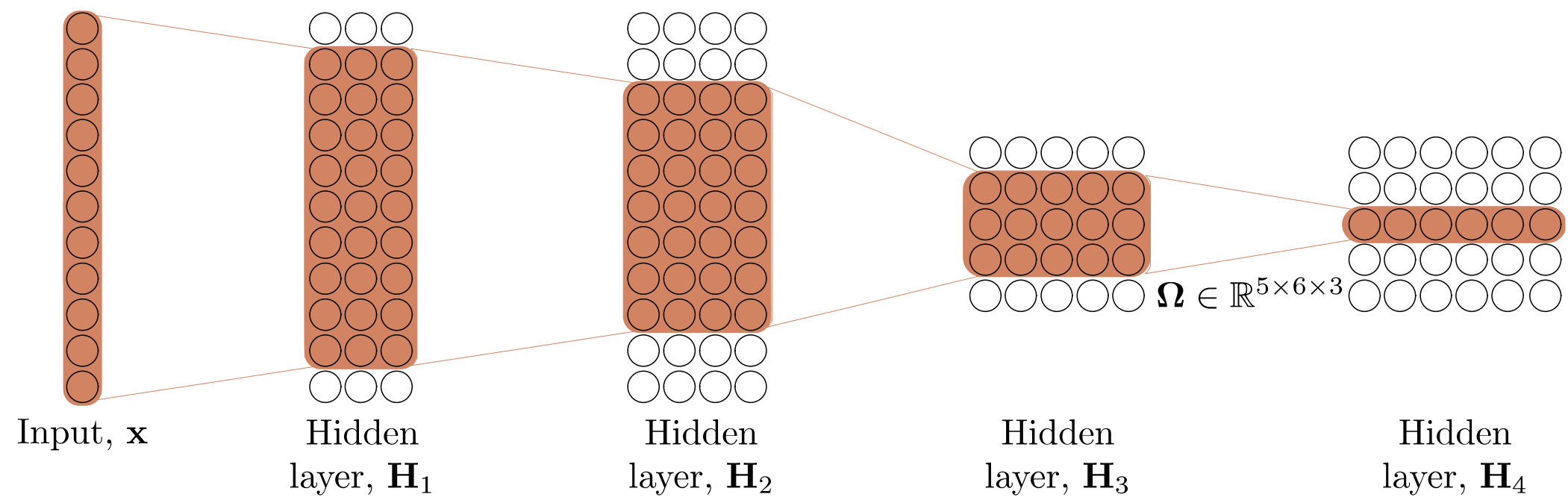
Receptive fields



Receptive fields



Receptive fields

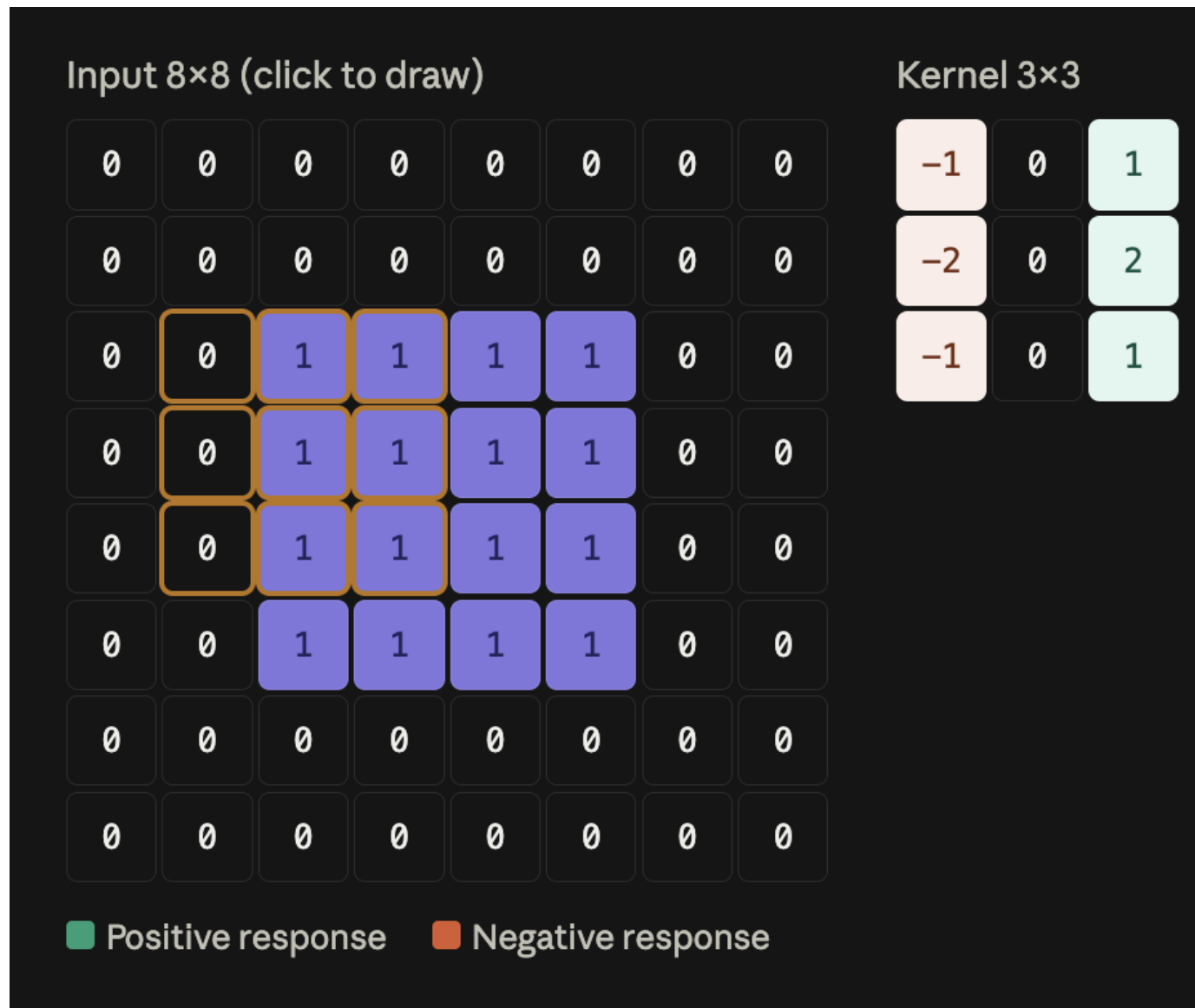


2D convolution

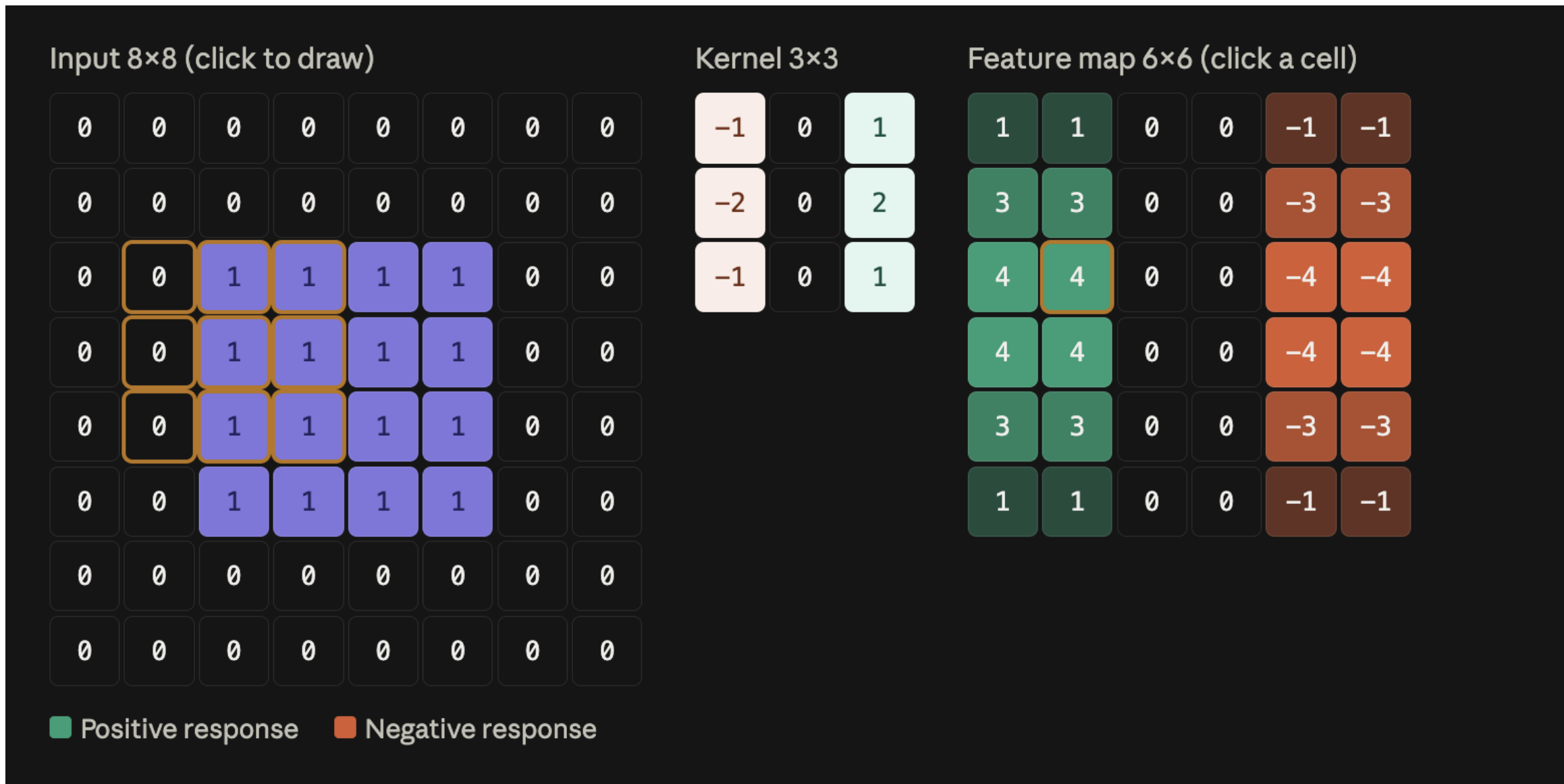
- Weighted sum over a KxK region
- Add bias and pass through an activation function

$$h_{i,j} = a \left[\beta + \sum_{m=1}^3 \sum_{n=1}^3 \omega_{m,n} x_{i+m-2,j+n-2} \right]$$

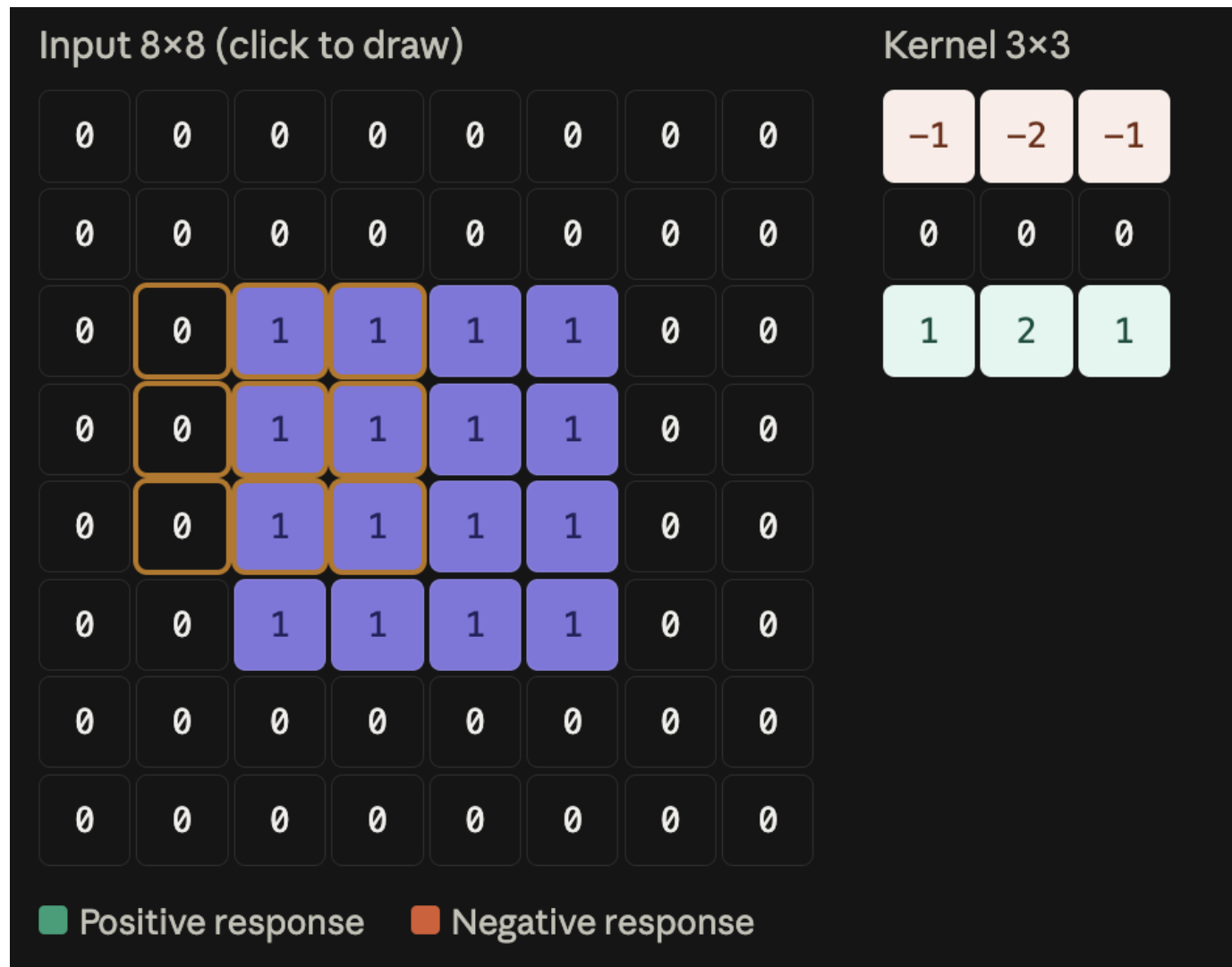
Simple examples of useful handmade 2D convolutions



Simple examples of useful handmade 2D convolutions

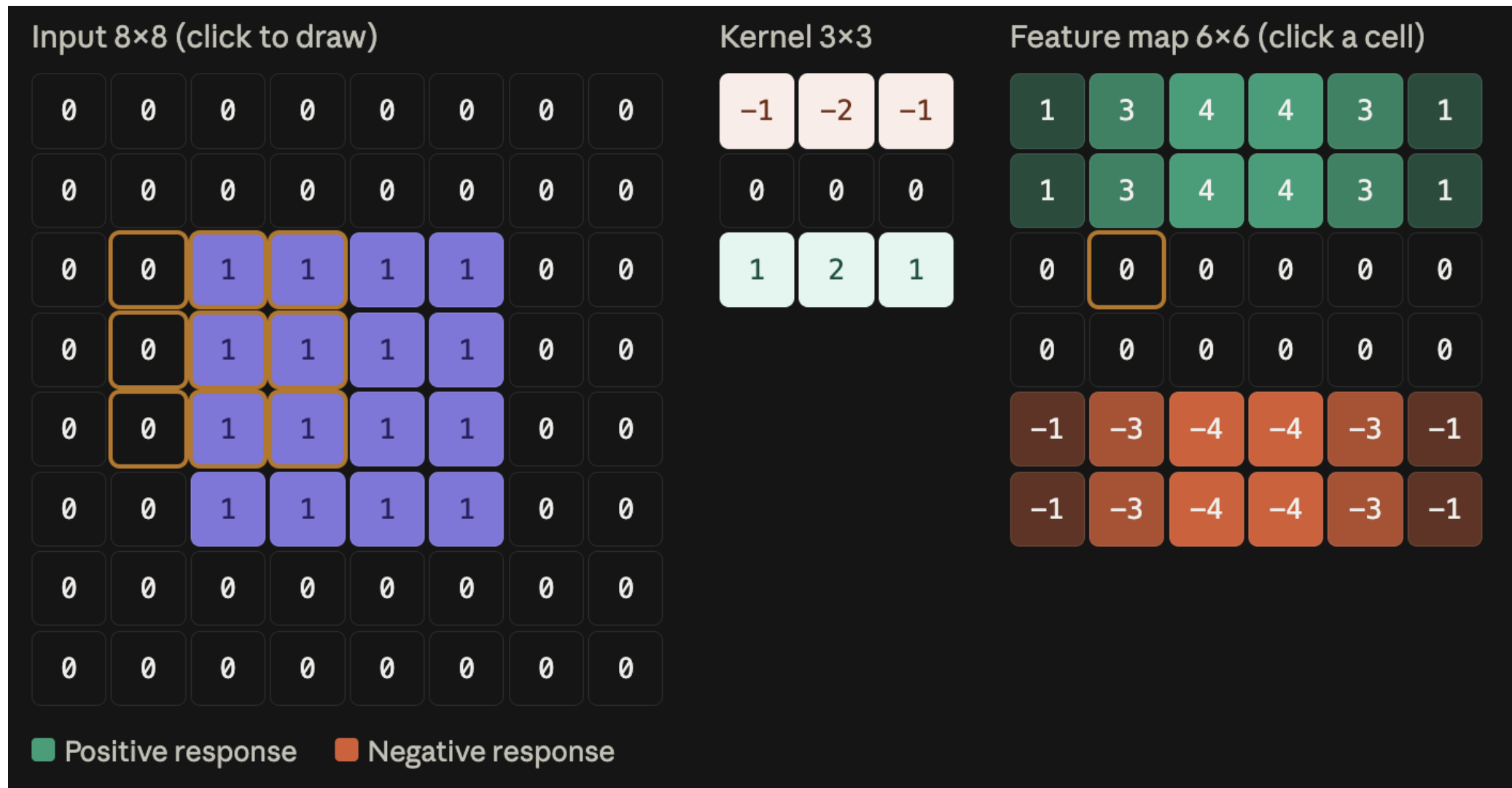


Simple examples of useful handmade 2D convolutions



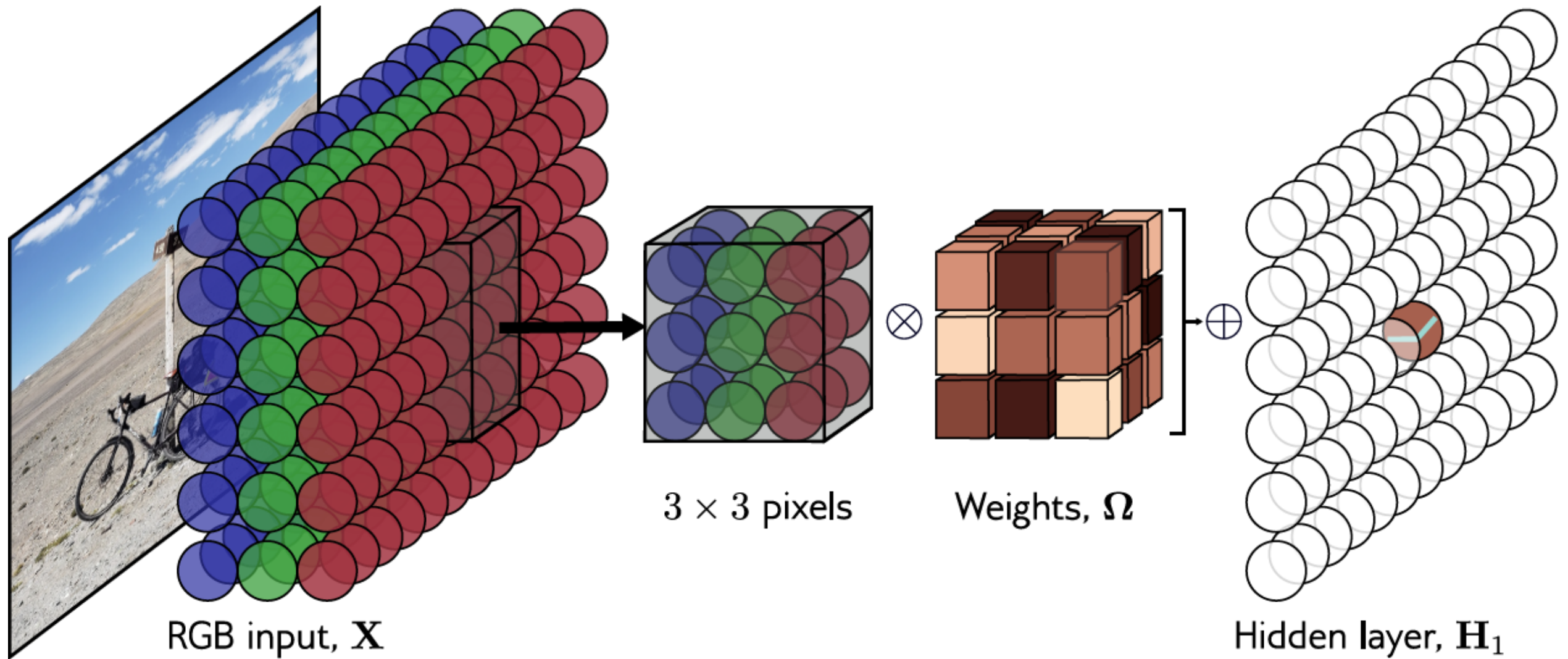
What about channels?

Simple examples of useful handmade 2D convolutions



What about channels?

2D convolution for an RGB image



How many parameters?

- C input channels and a kernel of size KxK

$$\omega \in \mathbb{R}^{C_i \times K \times K}$$

$$\beta \in \mathbb{R}$$

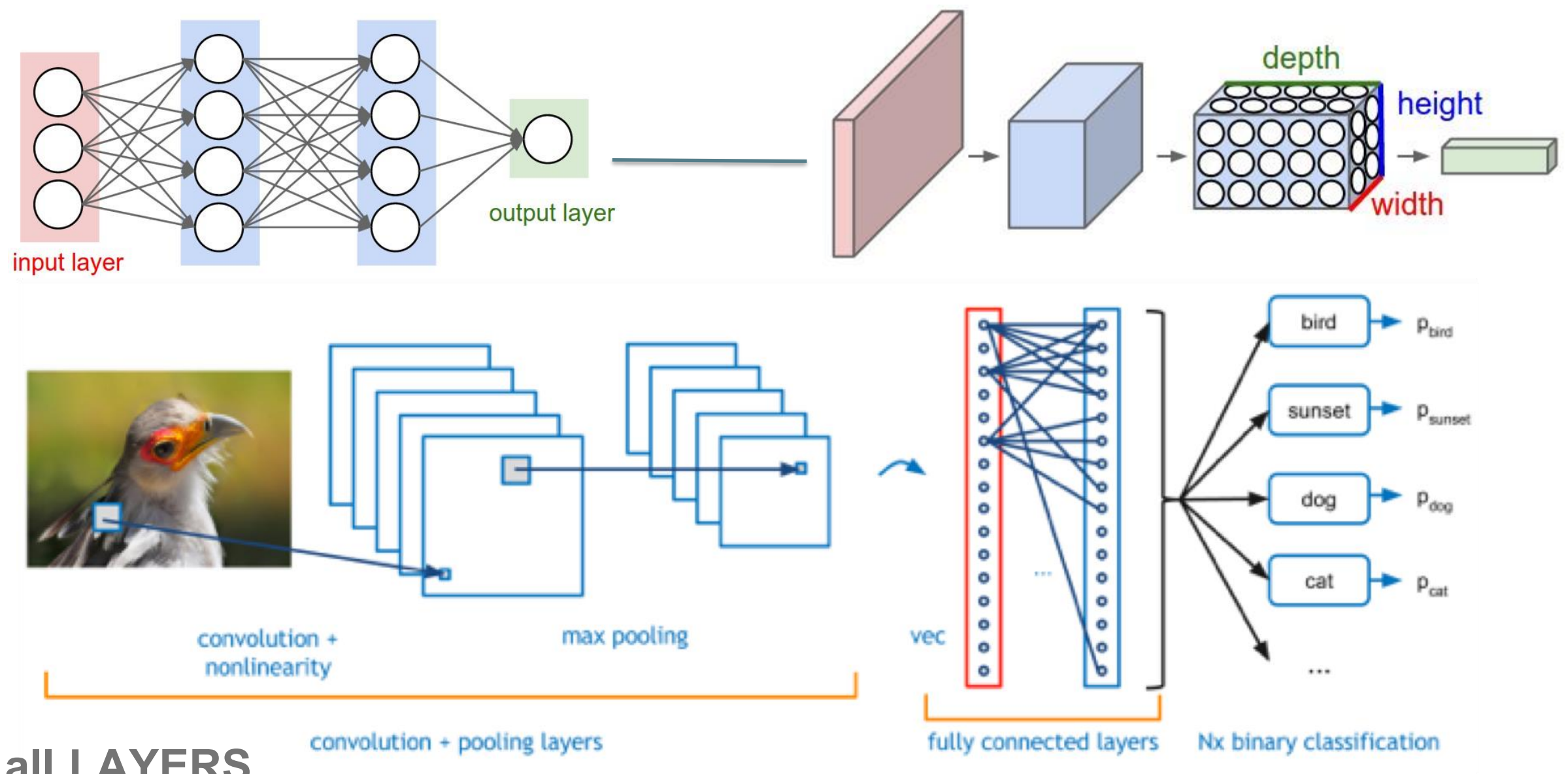
- C input channels, C' output channels and same kernel

$$\omega \in \mathbb{R}^{C_i \times C_o \times K \times K}$$

$$\beta \in \mathbb{R}^{C_o}$$

CNNs

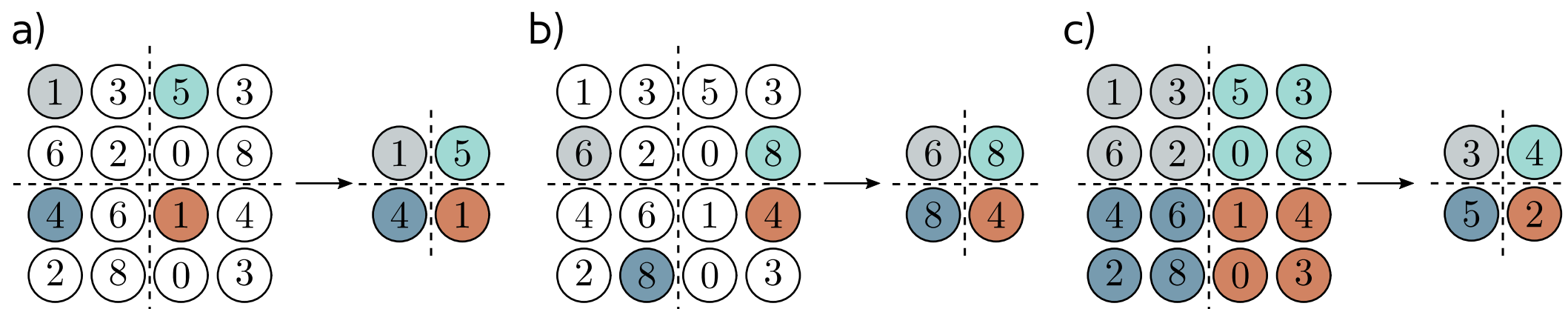
- Convolutional (deep) Neural Networks (CNN)



**NOT all LAYERS
are the same**

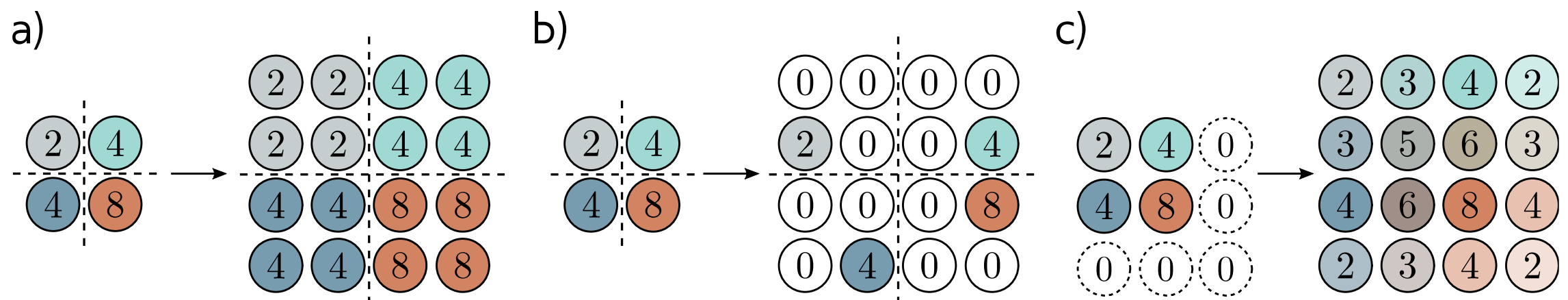
Downsampling and upsampling images

- Downsampling
 - Stride (a)
 - Max pooling (b)
 - Mean pooling (c)

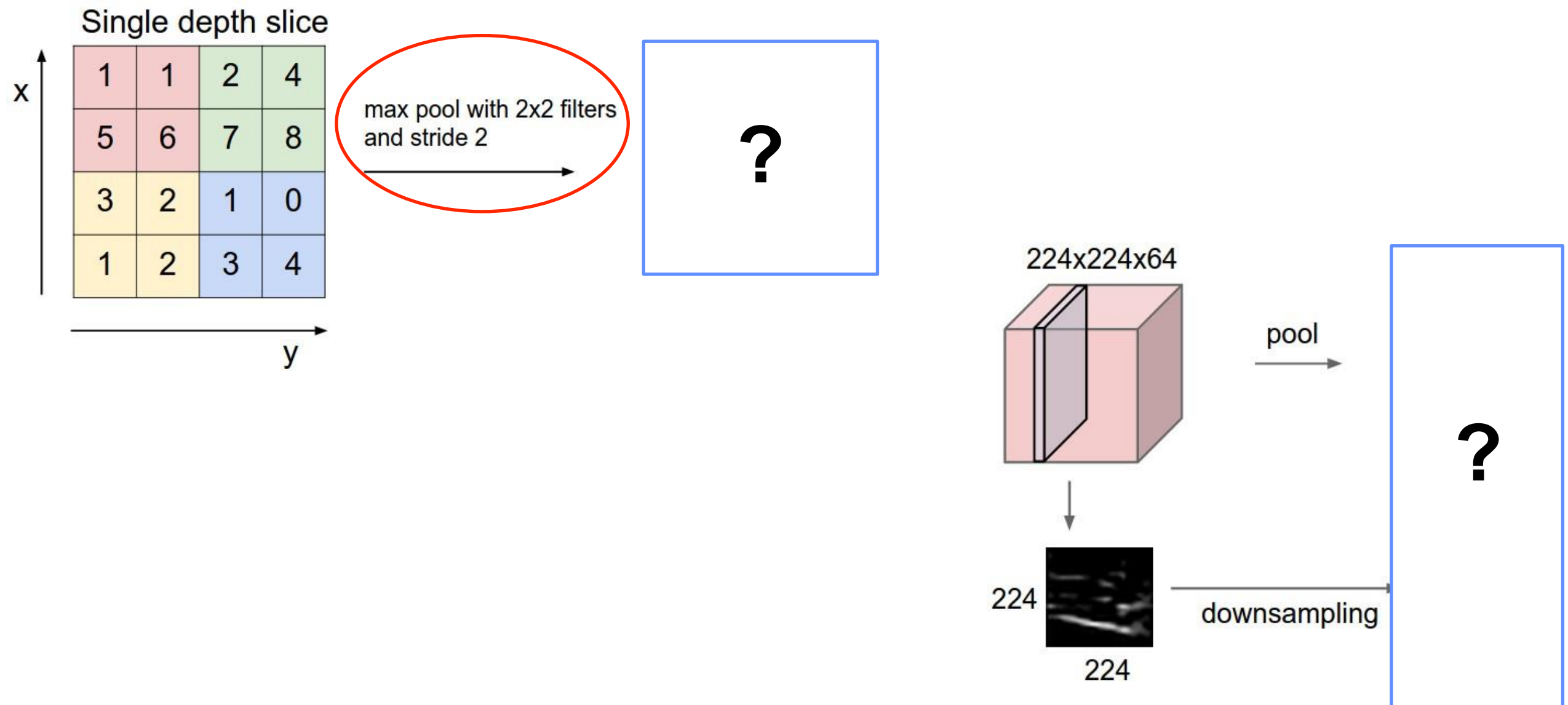


Downsampling and upsampling images

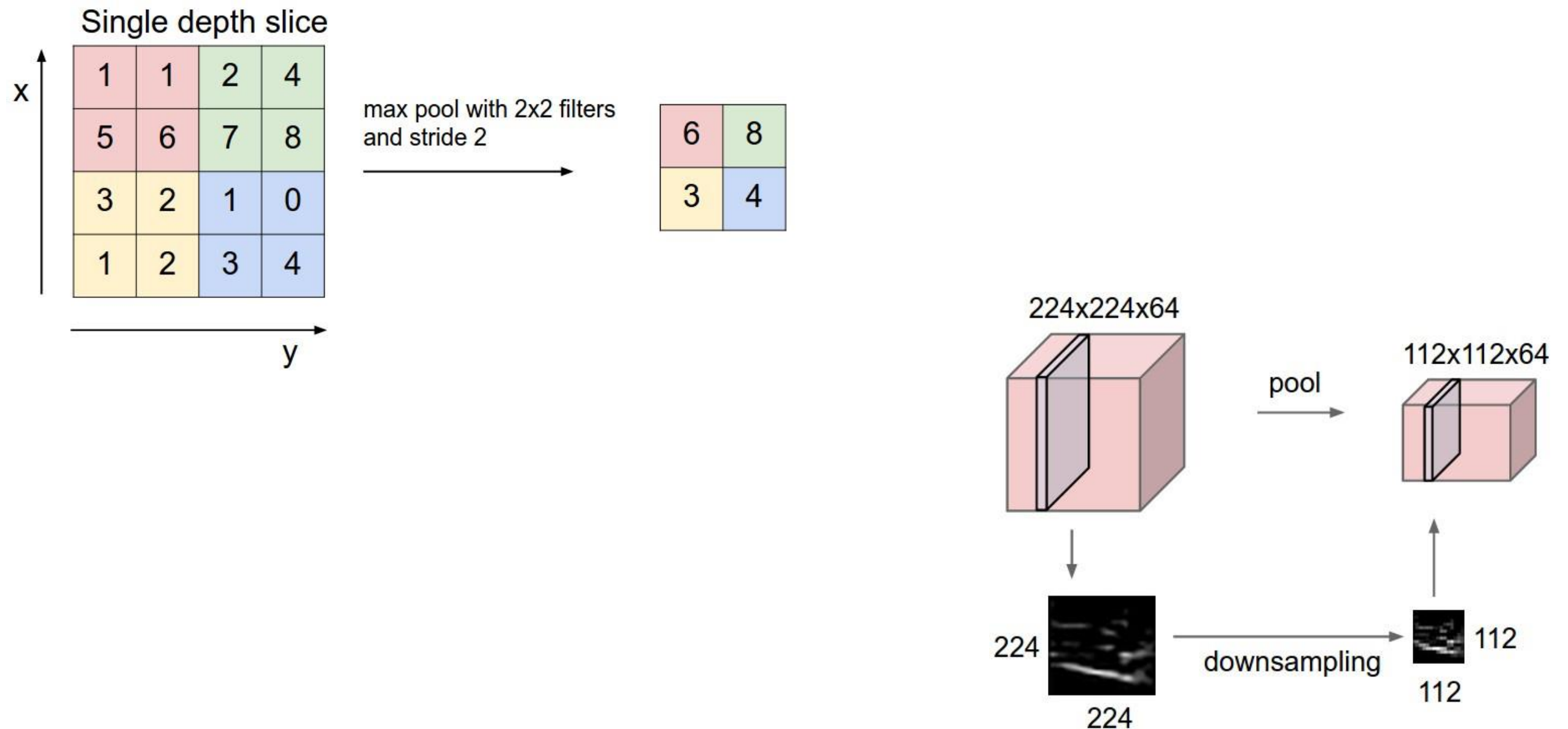
- Upsampling
 - Duplicate (a)
 - Max unpooling (b)
 - Interpolation (c)
 - Transposed convolution



CNNs - pooling example

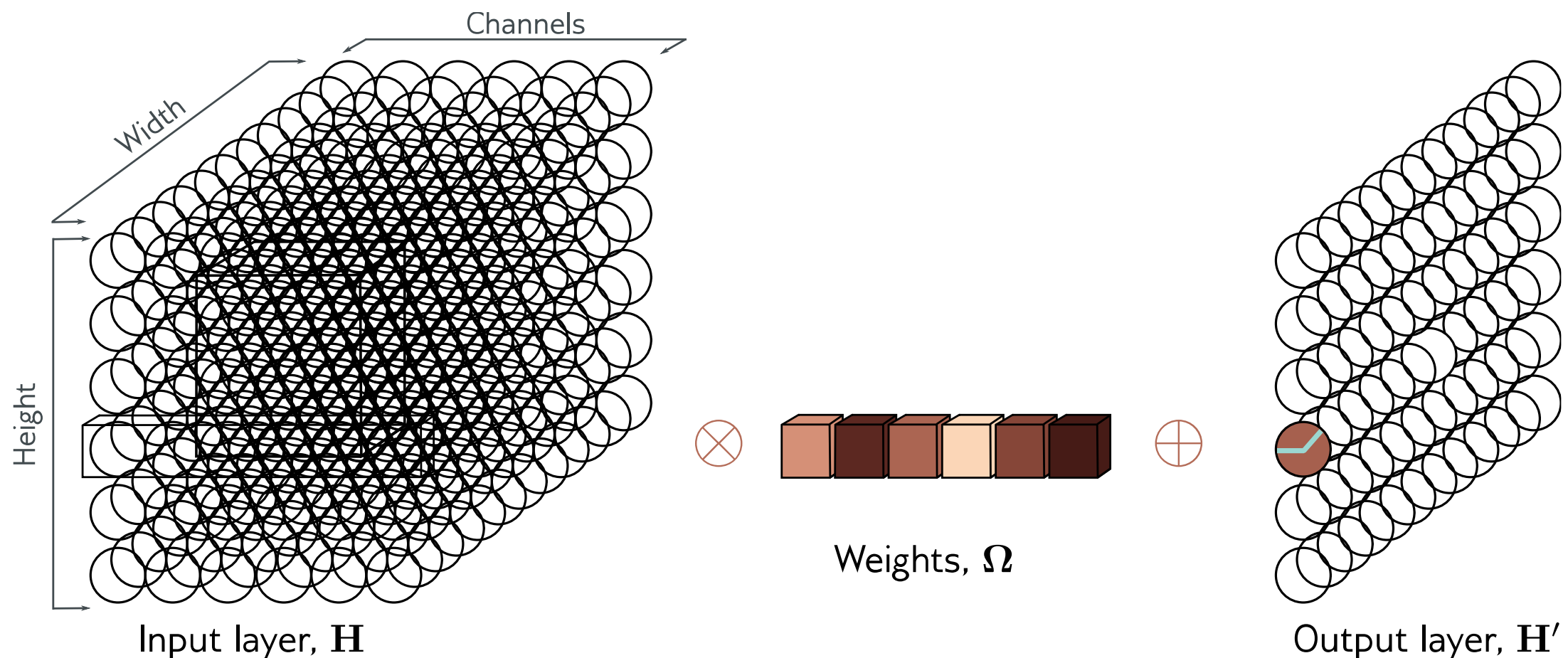


CNNs - pooling example



Adjusting the number of channels

- 1x1 convolution to mix channels
- Equivalent to running the same fully connected network at each position

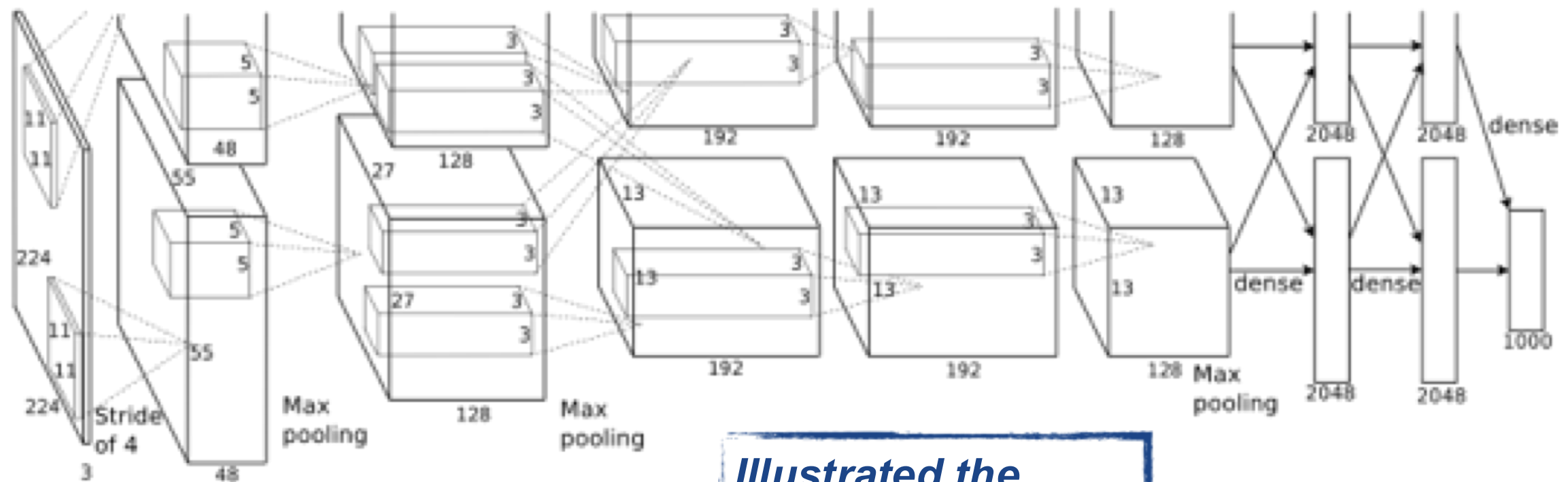


CNNs

- A bit of “history” about CNNs

CNNs - Image Classification

CNNs basics: AlexNet (Beginning of the *new CNN boom*):
a layered model composed of convolution, subsampling, and further operations followed by a holistic representation.



*Illustrated the
benefits of CNNs*

- ImageNet: 15M annotated images over 22,000 classes

- **Most Used ILSVRC subset of 1.2M images and 1000 classes**

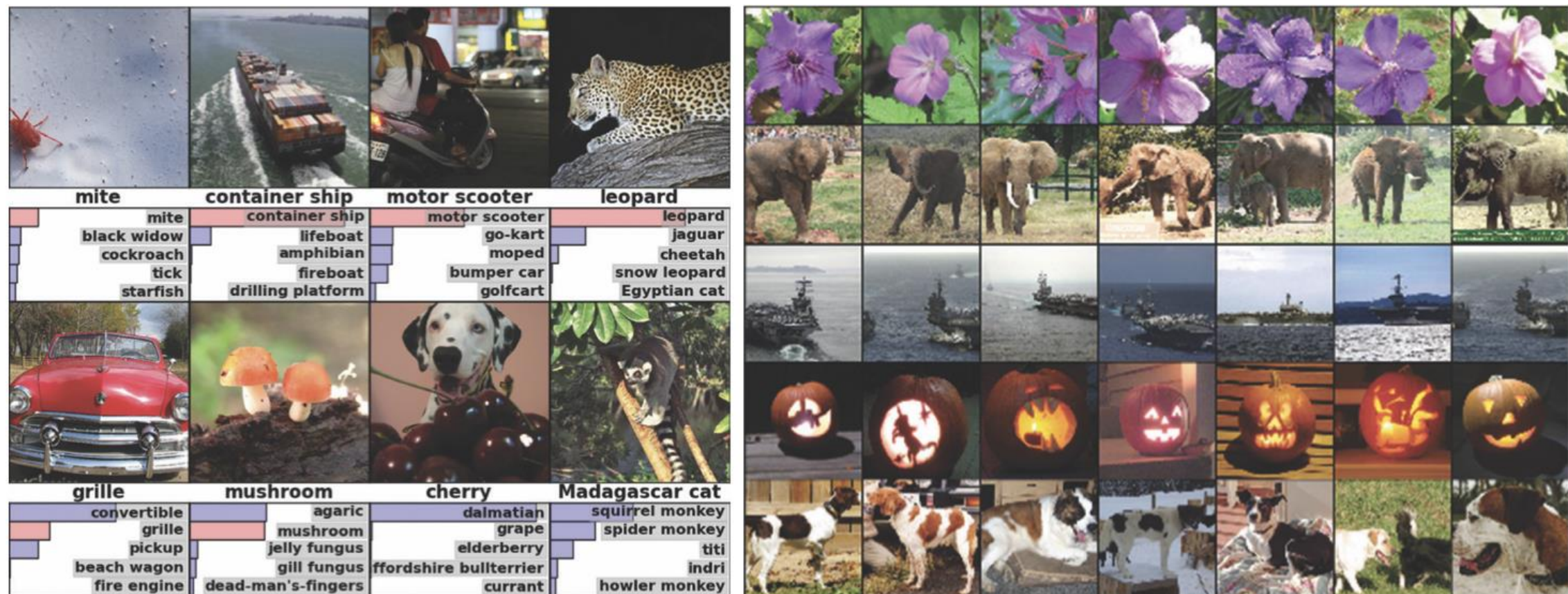
- **ReLU, data augmentation, dropout**

IMAGENET

ImageNet Classification with Deep Convolutional Neural Networks
[Alex Krizhevsky](#), [Ilya Sutskever](#), [Geoffrey E. Hinton](#). NIPS 2012

CNNs - Image Classification

CNNs basics: AlexNet (Beginning of the *new CNN boom*):
a layered model composed of convolution, subsampling, and further operations followed by a holistic representation.



- ImageNet: 15M annotated images over 22,000 classes
- **Most Used ILSVRC subset of 1.2M images and 1000 classes**
- **ReLU, data augmentation, dropout**

IMAGENET

ImageNet Classification with Deep Convolutional Neural Networks
[Alex Krizhevsky](#), [Ilya Sutskever](#), [Geoffrey E. Hinton](#). NIPS 2012

CNNs

CNNs *basics*: DeConvNet

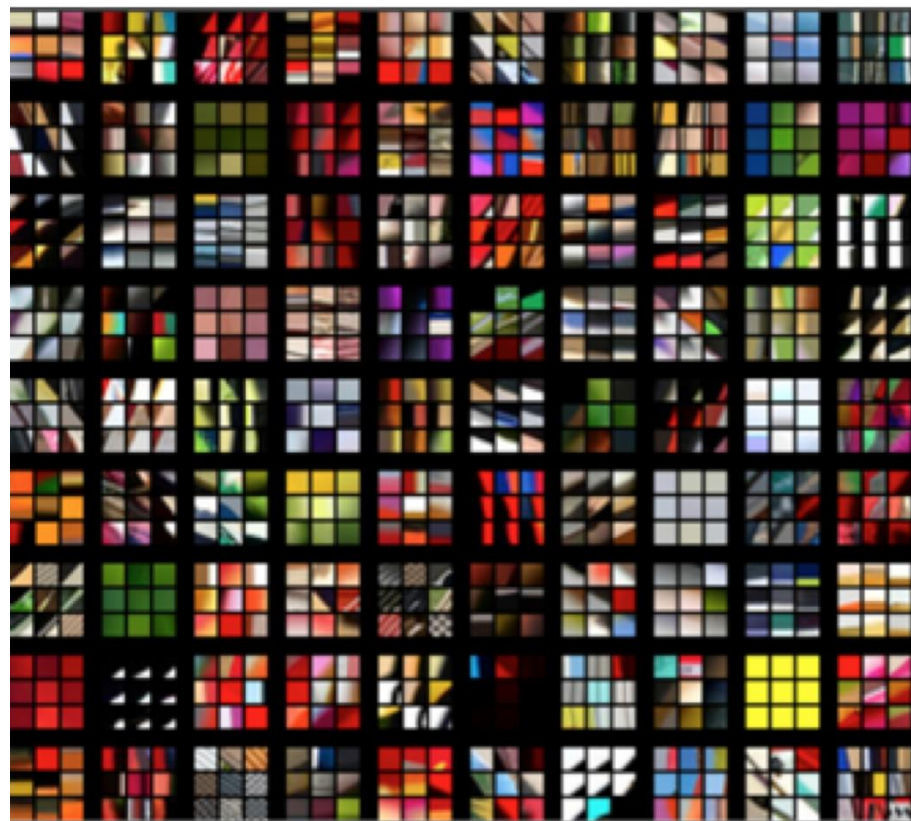
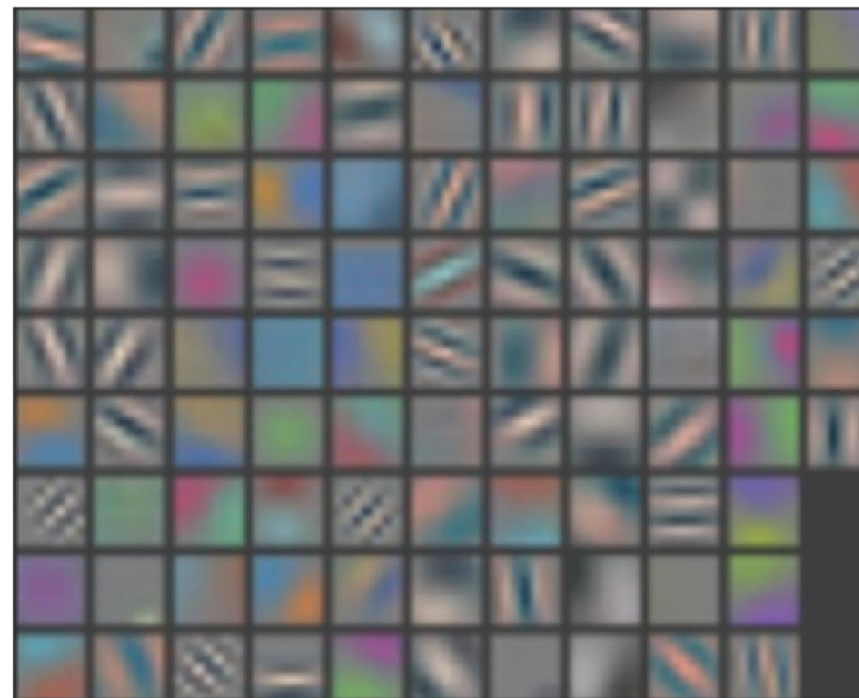


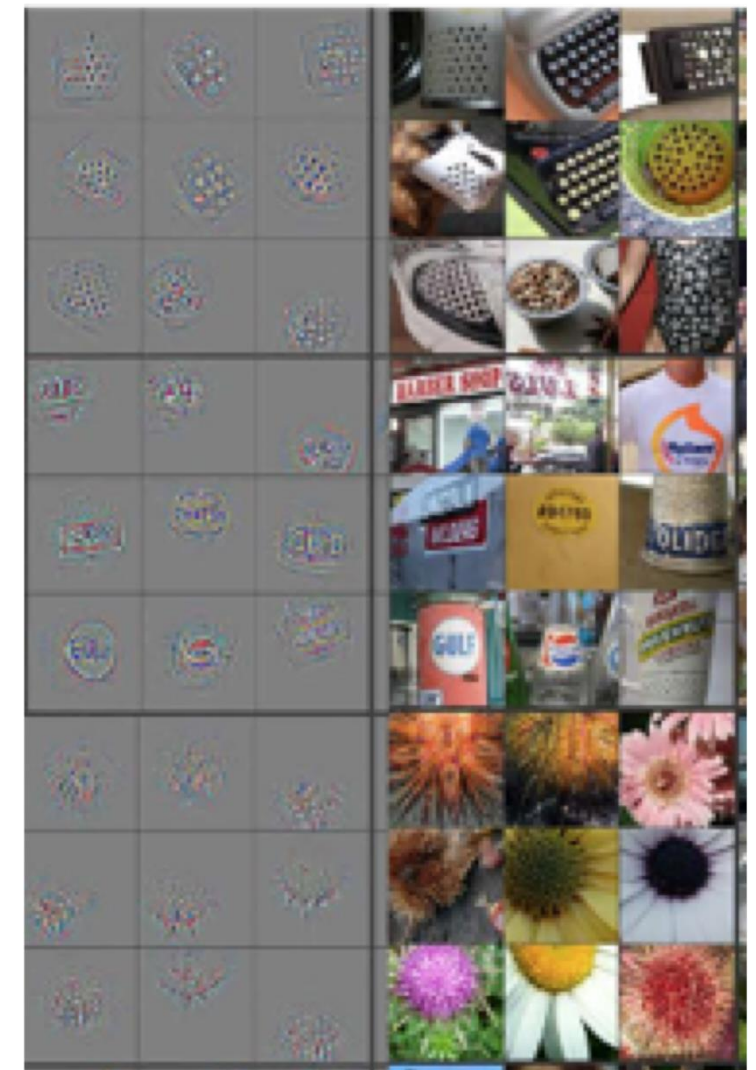
image patches that strongly activate 1st layer filters



1st layer filters

***better understanding:
earlier vs later layers***

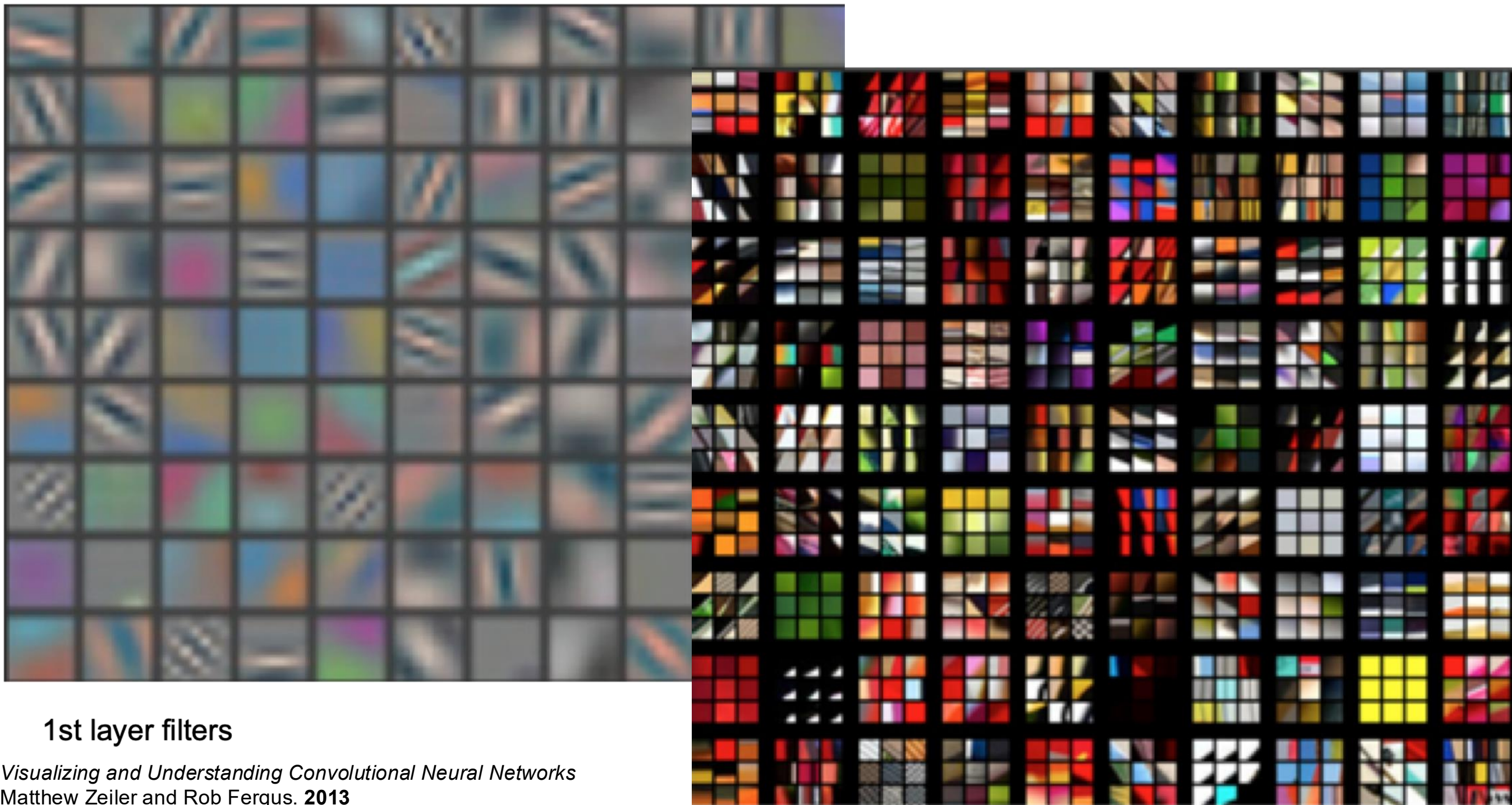
- Similar architecture to AlexNet
- smaller filters and stride
- Deconvolutional Network - **visualization**



conv₅ DeConv visualization
[Zeiler-Fergus]

CNNs

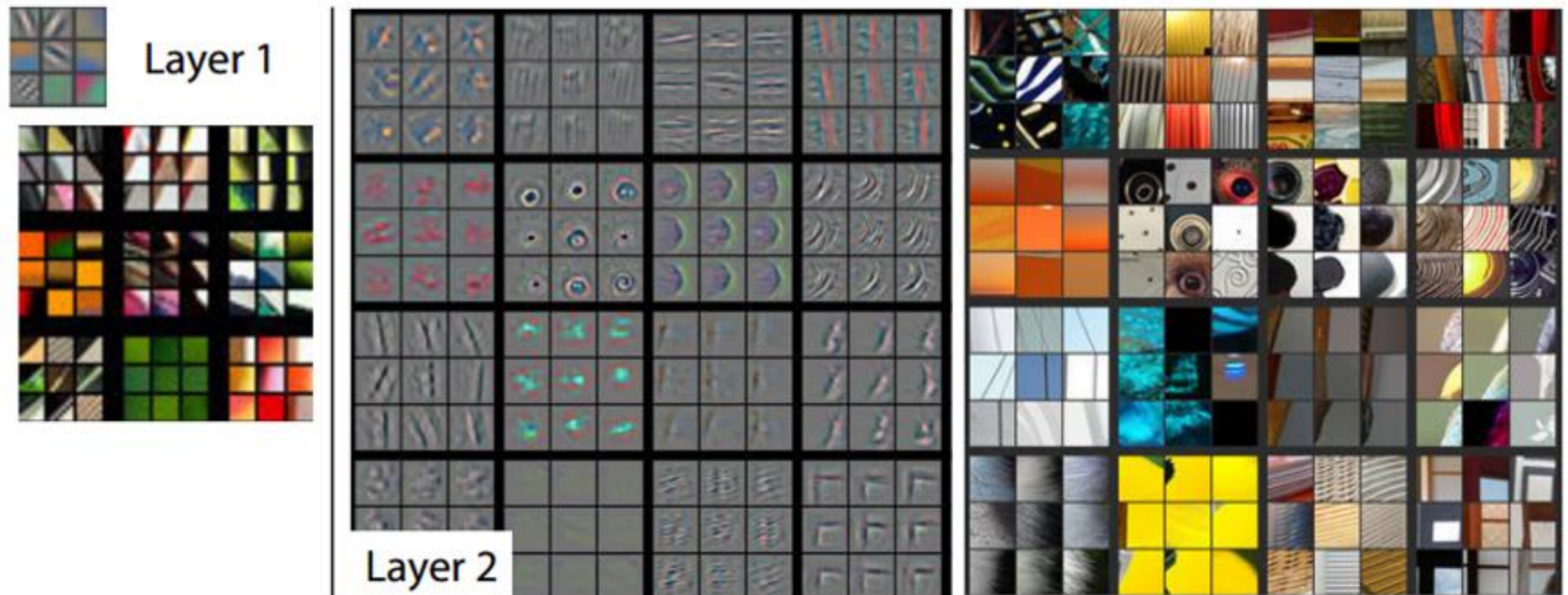
CNNs *basics*: DeConvNet



1st layer filters

CNNs

CNNs *basics*: DeConvNet

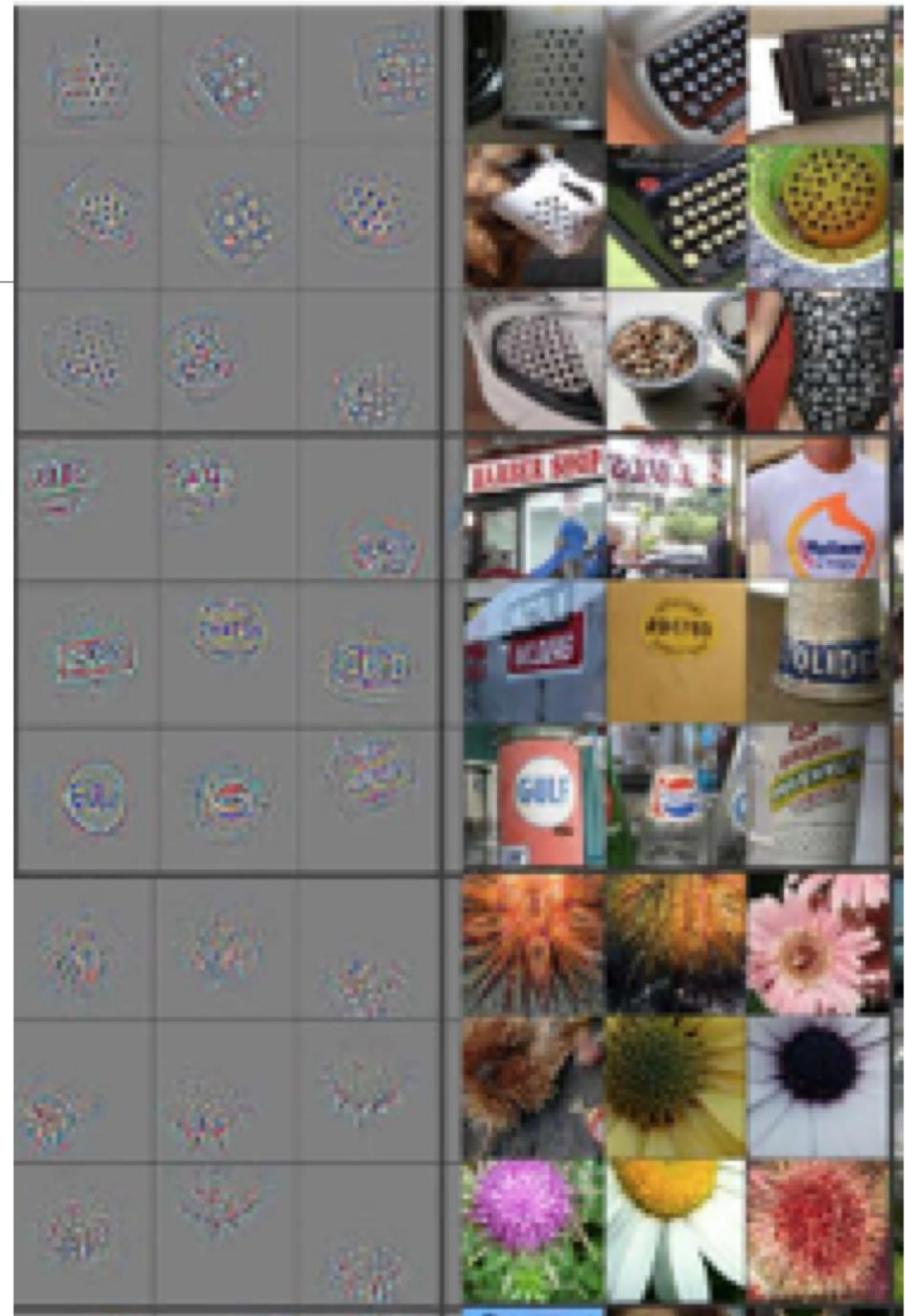


Visualizations of Layer 1 and 2. Each layer illustrates 2 pictures, one which shows the filters themselves and one that shows what part of the image are most strongly activated by the given filter. For example, in the space labeled Layer 2, we have representations of the 16 different filters (on the left)

CNNs

CNNs *basics*: DeConvNet

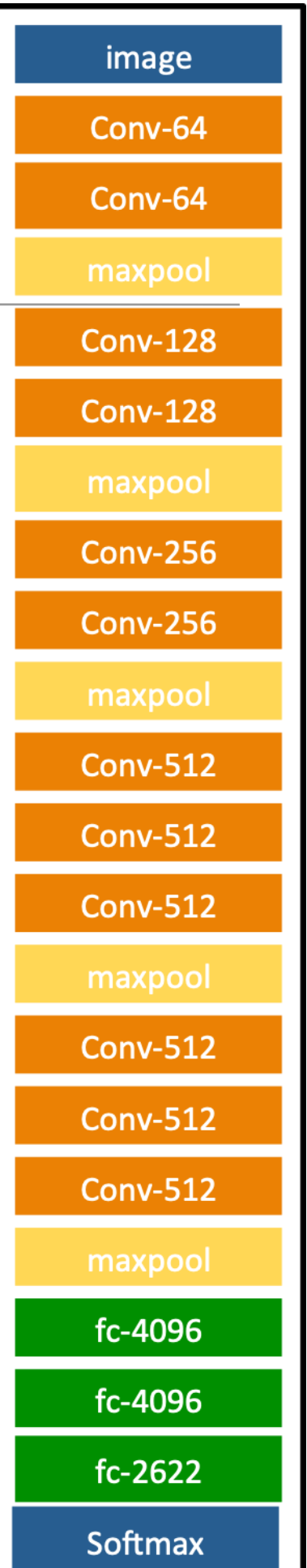
top 9 activations for a few feature maps
(projection to pixel space using DeConv
and actual image patches)



conv₅ DeConv visualization
[Zeiler-Fergus]

CNNs - Image classification

- *Deeper* classification: VGG - **very deep** ConvNets
 - Smaller filters - less params (per receptive field)
 - more depth! —> always good (?)
 - Consecutive conv. layers (smaller filters).
 - Compare to AlexNet 5 conv. layers



CNNs - Image classification

- *Deeper* classification: VGG - **very deep** ConvNets
 - Smaller filters - less params (per receptive field)
 - more depth! —> always good (?)
 - Consecutive conv. layers (smaller filters).
 - Compare to AlexNet 5 conv. layers



How deep?

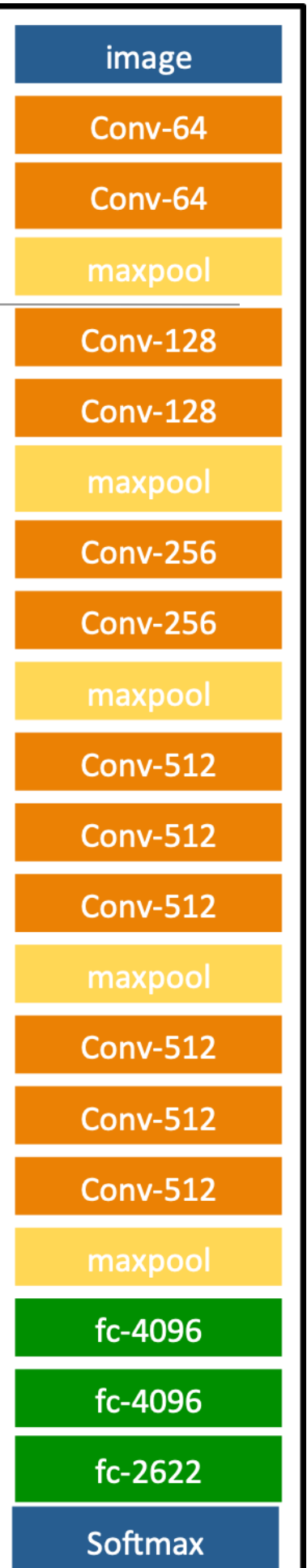


Image classification (deeper)

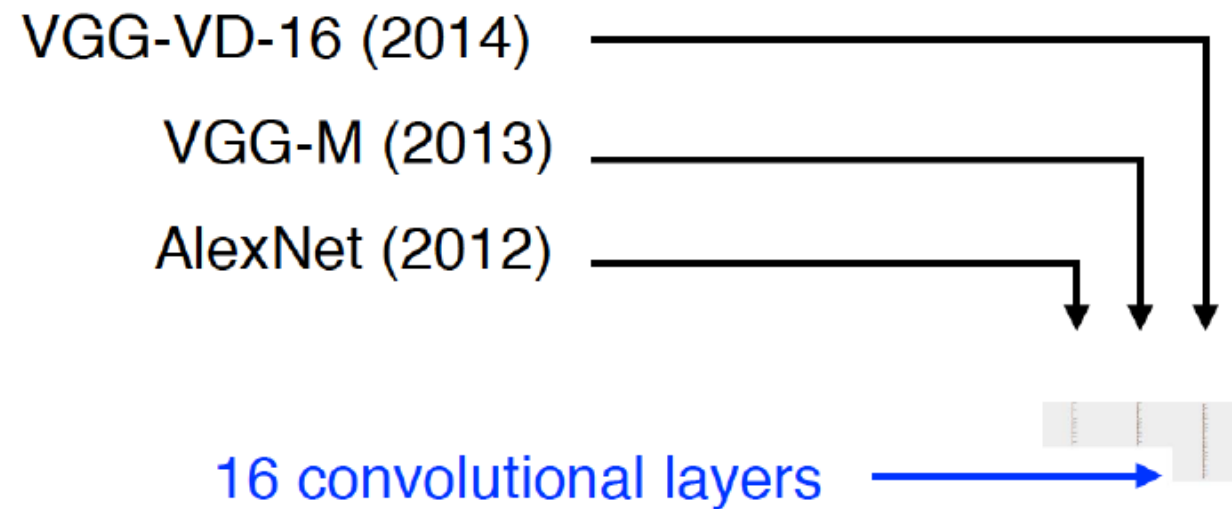
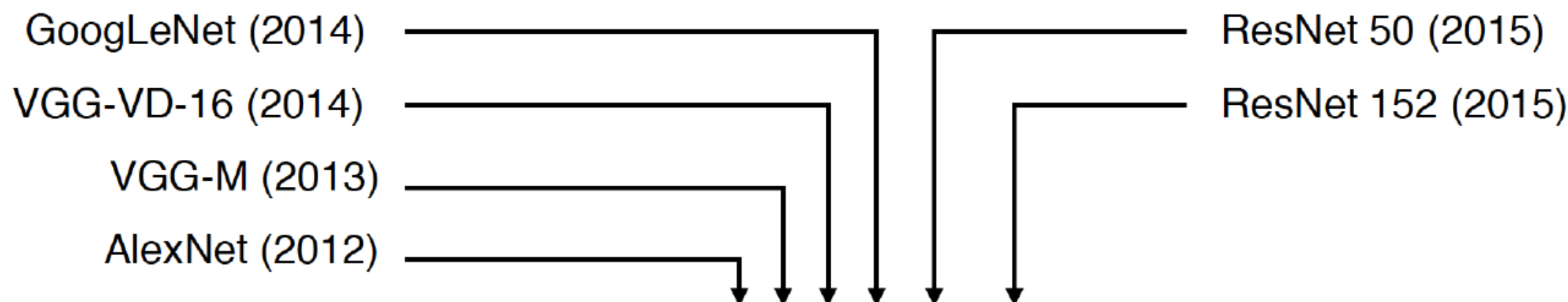


Image classification (deeper)



16 convolutional layers

50 convolutional layers

152 convolutional layers

Krizhevsky, I. Sutskever, and G. E. Hinton. *ImageNet classification with deep convolutional neural networks*. In Proc. NIPS, 2012.

C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. *Going deeper with convolutions*. In Proc. CVPR, 2015.

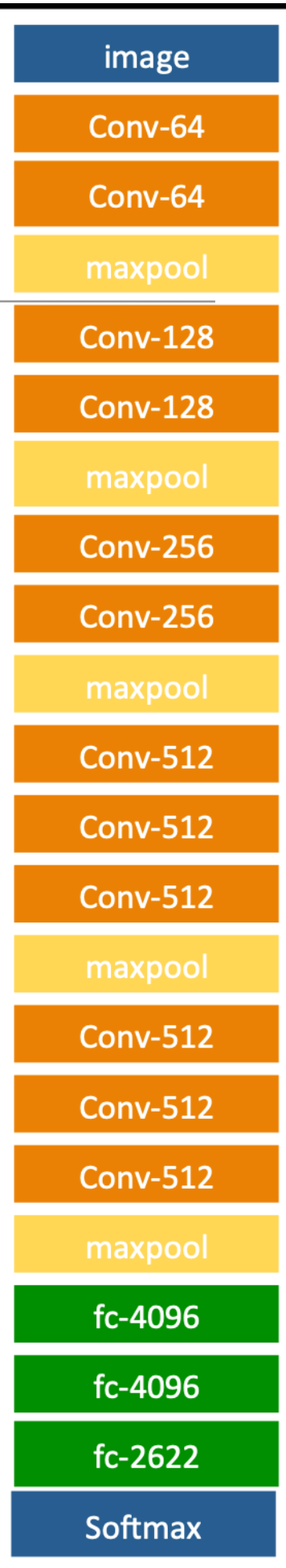
K. Simonyan and A. Zisserman. *Very deep convolutional networks for large-scale image recognition*. In Proc. ICLR, 2015.

K. He, X. Zhang, S. Ren, and J. Sun. *Deep residual learning for image recognition*. In Proc. CVPR, 2016.

Are they just deeper versions of the same?

Deeper: How does memory get affected?

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)
CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$

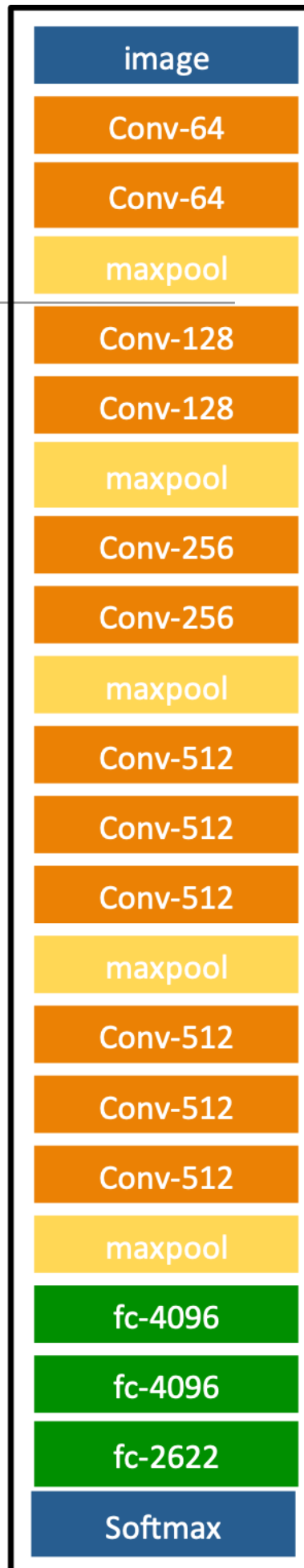


Deeper: How does memory get affected?

INPUT: [224x224x3] memory: $224*224*3=150\text{K}$ params: 0 (not counting biases)
 CONV3-64: [224x224x64] memory: $224*224*64=3.2\text{M}$ params: $(3*3*3)*64 = 1,728$
 CONV3-64: [224x224x64] memory: $224*224*64=3.2\text{M}$ params: $(3*3*64)*64 = 36,864$
 POOL2: [112x112x64] memory: $112*112*64=800\text{K}$ params: 0
 CONV3-128: [112x112x128] memory: $112*112*128=1.6\text{M}$ params: $(3*3*64)*128 = 73,728$
 CONV3-128: [112x112x128] memory: $112*112*128=1.6\text{M}$ params: $(3*3*128)*128 = 147,456$
 POOL2: [56x56x128] memory: $56*56*128=400\text{K}$ params: 0
 CONV3-256: [56x56x256] memory: $56*56*256=800\text{K}$ params: $(3*3*128)*256 = 294,912$
 CONV3-256: [56x56x256] memory: $56*56*256=800\text{K}$ params: $(3*3*256)*256 = 589,824$
 CONV3-256: [56x56x256] memory: $56*56*256=800\text{K}$ params: $(3*3*256)*256 = 589,824$
 POOL2: [28x28x256] memory: $28*28*256=200\text{K}$ params: 0
 CONV3-512: [28x28x512] memory: $28*28*512=400\text{K}$ params: $(3*3*256)*512 = 1,179,648$
 CONV3-512: [28x28x512] memory: $28*28*512=400\text{K}$ params: $(3*3*512)*512 = 2,359,296$
 CONV3-512: [28x28x512] memory: $28*28*512=400\text{K}$ params: $(3*3*512)*512 = 2,359,296$
 POOL2: [14x14x512] memory: $14*14*512=100\text{K}$ params: 0
 CONV3-512: [14x14x512] memory: $14*14*512=100\text{K}$ params: $(3*3*512)*512 = 2,359,296$
 CONV3-512: [14x14x512] memory: $14*14*512=100\text{K}$ params: $(3*3*512)*512 = 2,359,296$
 CONV3-512: [14x14x512] memory: $14*14*512=100\text{K}$ params: $(3*3*512)*512 = 2,359,296$
 POOL2: [7x7x512] memory: $7*7*512=25\text{K}$ params: 0
 FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$
 FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$
 FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

TOTAL memory: $24\text{M} * 4 \text{ bytes} \approx 96\text{MB}$ / image (only forward! $\sim *2$ for bwd)

TOTAL params: 138M parameters



Deeper: How does memory get affected?

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$

POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$

POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0

FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$

FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$

FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

Note:

Most memory is in early CONV

Most params are in late FC

image
Conv-64
Conv-64
maxpool
Conv-128
Conv-128
maxpool
Conv-256
Conv-256
maxpool
Conv-512
Conv-512
Conv-512
maxpool
Conv-512
Conv-512
Conv-512
maxpool
fc-4096
fc-4096
fc-2622
Softmax

TOTAL memory: $24M * 4 \text{ bytes} \approx 96MB$ / image (only forward! $\sim *2$ for bwd)

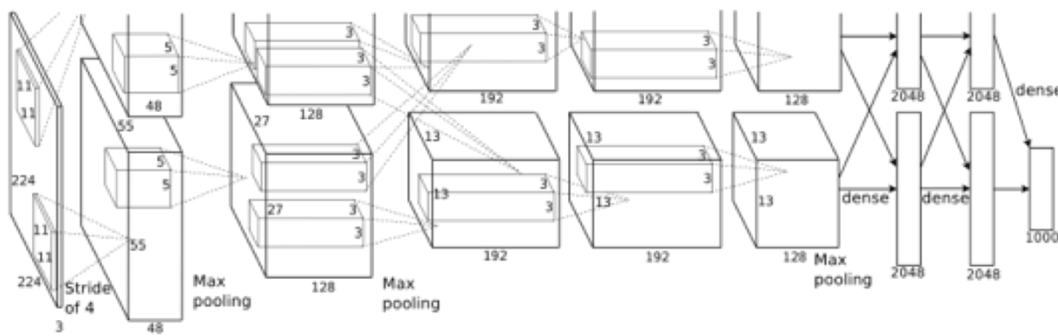
TOTAL params: 138M parameters

slide from
Fei-Fei Li & Justin Johnson & Serena Yeung
<http://cs231n.stanford.edu>

What is more relevant for batch size design decisions?

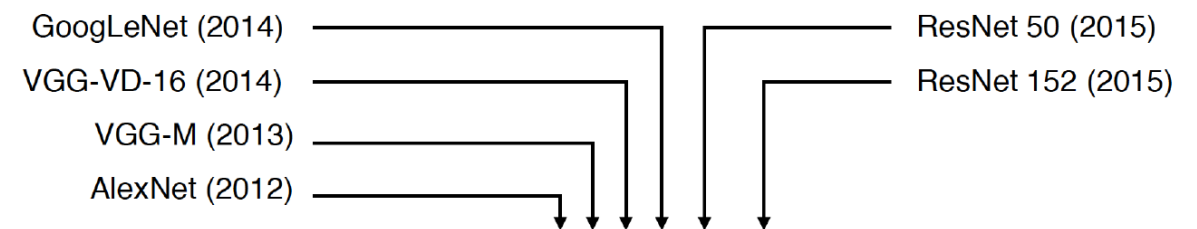
Image classification (deeper)

- Not only more layers! **it's getting hard(er) to train ...**



Vanishing gradients!

ImageNet Classification with Deep Convolutional Neural Networks
[Alex Krizhevsky](#), [Ilya Sutskever](#), [Geoffrey E. Hinton](#). NIPS 2012



16 convolutional layers

50 convolutional layers

152 convolutional layers

Krizhevsky, I. Sutskever, and G. E. Hinton. *ImageNet classification with deep convolutional neural networks*. In Proc. NIPS, 2012.

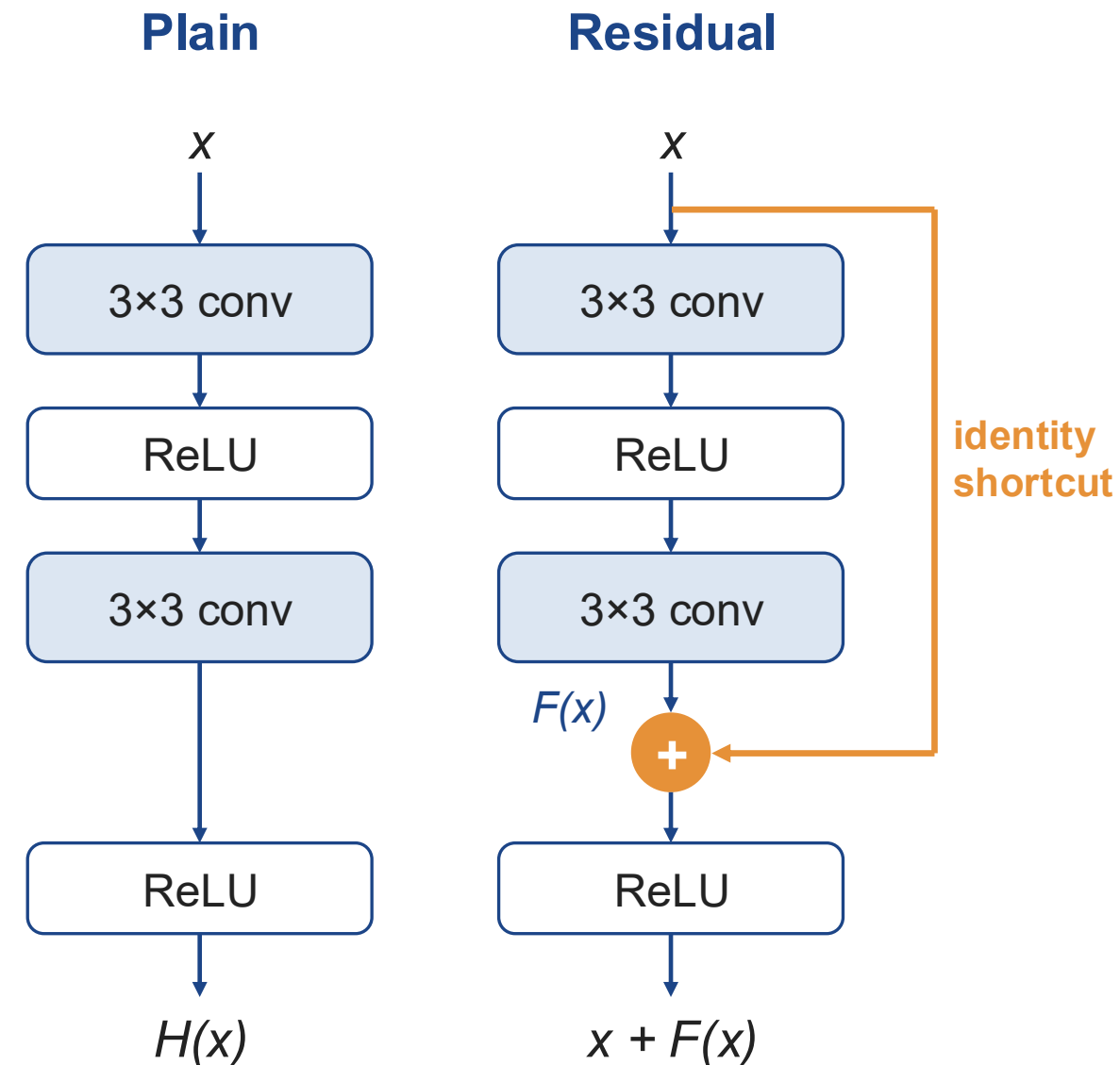
C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. *Going deeper with convolutions*. In Proc. CVPR, 2015.

K. Simonyan and A. Zisserman. *Very deep convolutional networks for large-scale image recognition*. In Proc. ICLR, 2015.

K. He, X. Zhang, S. Ren, and J. Sun. *Deep residual learning for image recognition*. In Proc. CVPR, 2016.

Residual connections (ResNet)

- Plain block: layers learn the mapping $H(x)$ directly
- Residual block: layers learn only the residual
 - $F(x) = H(x) - x$
- Shortcut (identity) adds the input back:
 - $y = x + F(x)$
- If identity is the best thing to do, just push $F(x) \rightarrow 0$
 - no parameters, one addition



Deeper should not be worse!

Why it works: a gradient highway

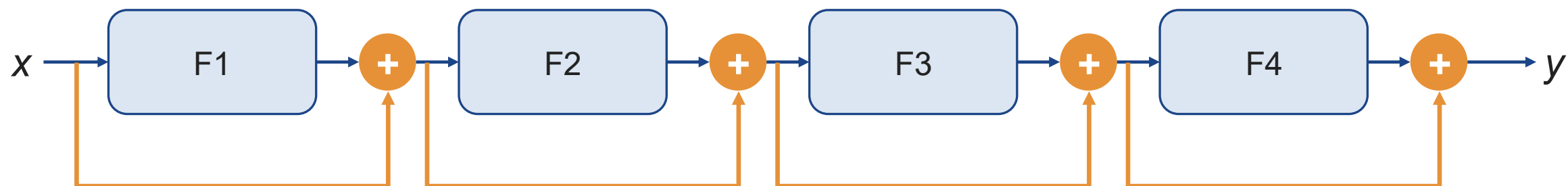
$$\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l) \Rightarrow \mathbf{h}_L = \mathbf{h}_l + \sum_{i=l}^{L-1} F(\mathbf{h}_i)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_L} \left(\mathbf{1} + \frac{\partial}{\partial \mathbf{h}_l} \sum_{i=l}^{L-1} F(\mathbf{h}_i) \right)$$

the **1**: gradient reaches early layers directly,
no product of Jacobians

plain : $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_L} \prod_{i=l}^{L-1} \frac{\partial \mathbf{h}_{i+1}}{\partial \mathbf{h}_i}$

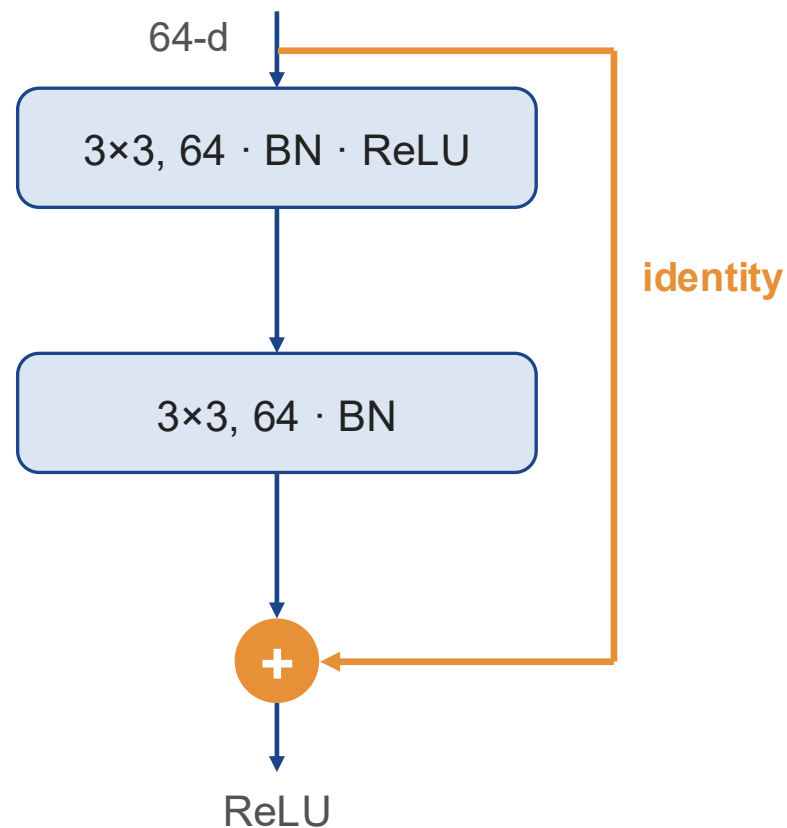
product of many Jacobians $\rightarrow$ vanishes or
explodes (slide 51)



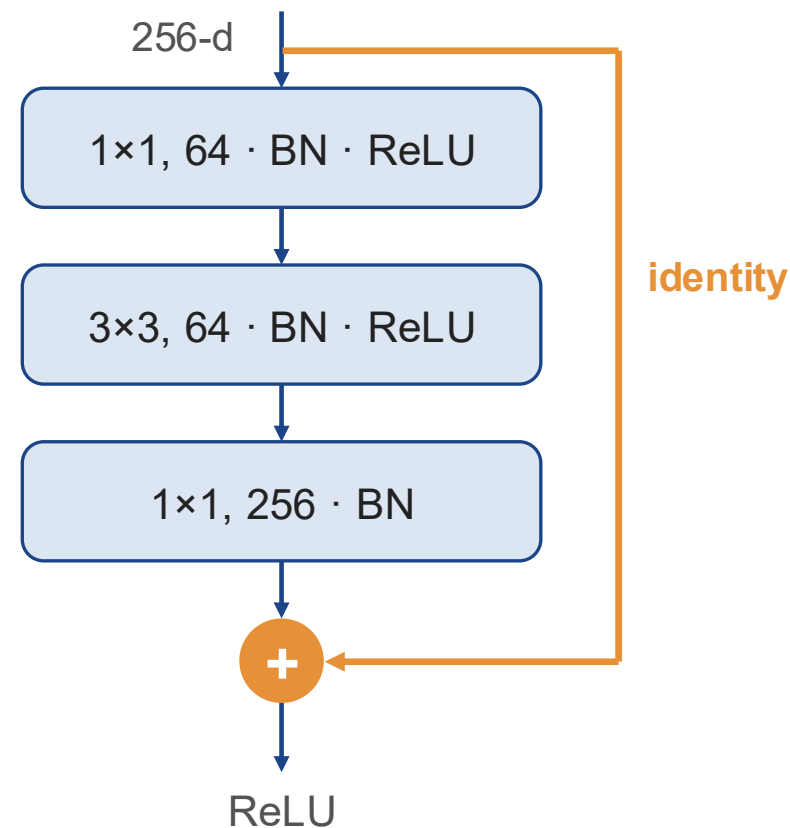
Unrolled view: n blocks = 2^n paths of different lengths $\rightarrow$ behaves like an ensemble of shallower nets

ResNet building blocks

Basic (ResNet-18/34)



Bottleneck (ResNet-50+)



Shapes must match

When a stage halves H×W and doubles C:

shortcut = 1×1 conv, stride 2
(projection shortcut; 1×1 conv, slide 36)

17×

fewer weights

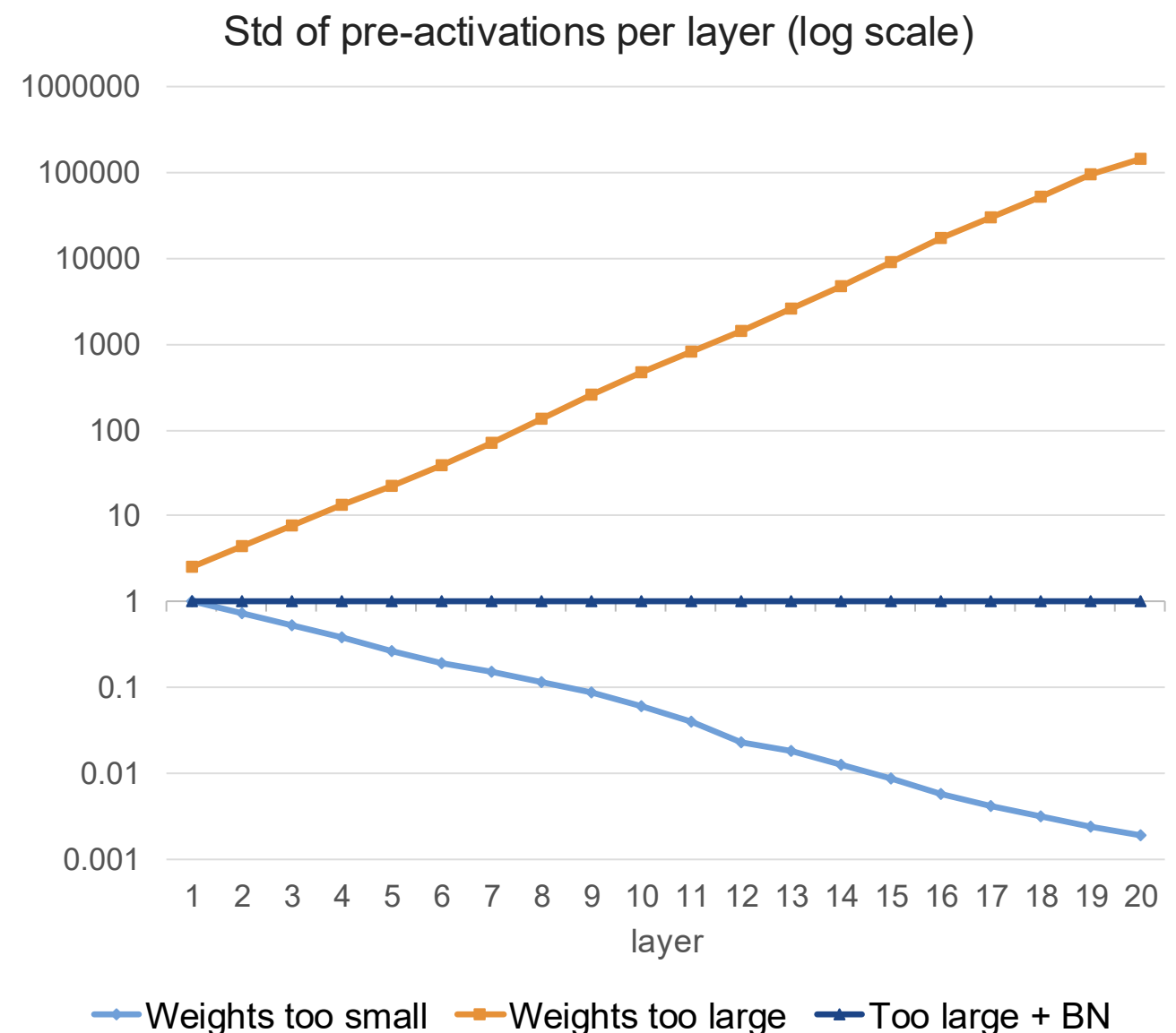
$2 \cdot 9 \cdot 256^2 = 1.18\text{M}$ vs.
 $256 \cdot 64 + 9 \cdot 64^2 + 64 \cdot 256 = 70\text{K}$

conv → **BN** → **ReLU** everywhere ... what is BN?

Batch normalization: motivation

- We normalize the input data ...
 - ... every layer is the input of the next one
- Without control, activation scale shrinks or grows layer after layer
 - good init (He) only fixes the start of training
- BN: normalize each feature with the statistics of the current mini-batch

Normalize inside the network!



Batch normalization: the computation

For each channel, over a mini-batch of m values:

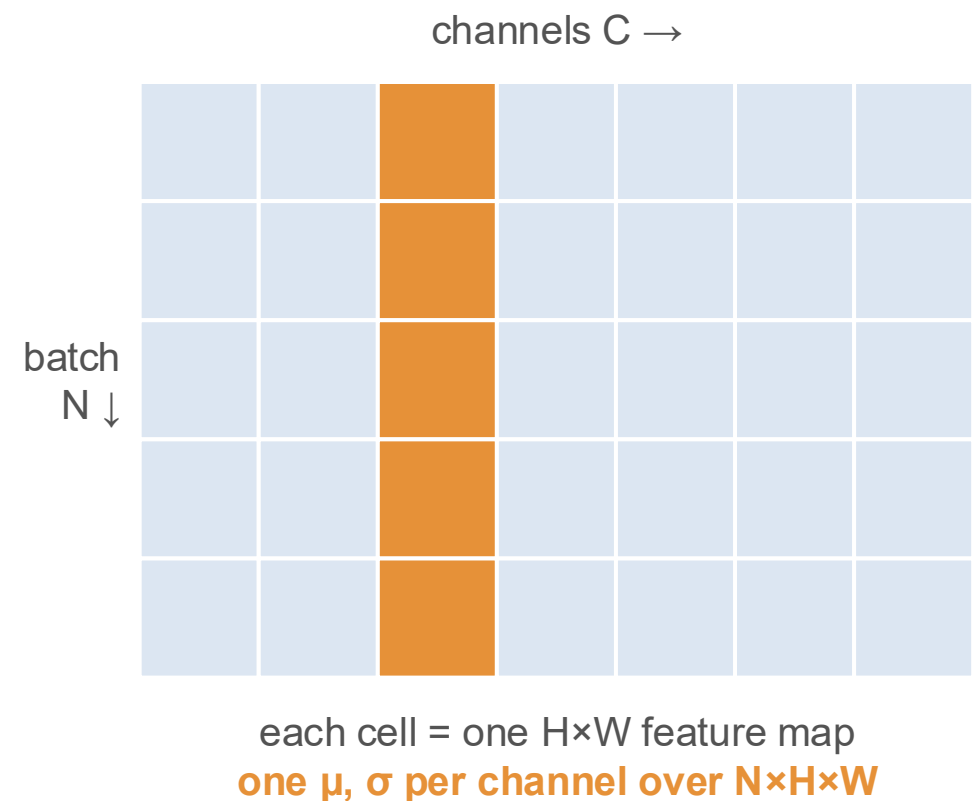
$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$
$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad y_i = \gamma \hat{x}_i + \beta$$

γ, β learnable: the net can rescale, or even undo, the normalization

Parameters: $2C$ learnable + $2C$ running statistics

Conv bias is redundant (β does it) $\rightarrow$ `bias=False`

In a CNN: statistics per channel



Exercise: 3×3 conv $64 \rightarrow 128$ channels followed by BN. How many learnable parameters?

Batch normalization: training vs. inference

Training · `model.train()`

- uses statistics of the current mini-batch
- updates running averages:

$$\mu_{run} \leftarrow (1 - m) \mu_{run} + m \mu_B$$

$m = 0.1$ in PyTorch

Inference · `model.eval()`

- uses the stored running μ , σ^2
- output independent of the batch
- BN = fixed affine map $\rightarrow$ fold into conv

```
model.train() # batch stats
loss.backward(); opt.step()
model.eval() # running stats
with torch.no_grad(): y = model(x)
```

Always check `train()` / `eval()`!

Classic pitfalls

Forgot `model.eval()` $\rightarrow$ results depend on the batch

Tiny batches ($< 8-16$) $\rightarrow$ noisy statistics

Fine-tuning on little data $\rightarrow$ freeze BN statistics (transfer learning)

Why does batch normalization help?

14×

fewer training steps
same accuracy (Inception + BN)

*BN made very deep nets
(ResNet-152) practical*

1

Faster, more stable training

higher learning rates; much less sensitive to initialization

2

Smoother loss landscape

gradients more predictive → larger steps are safe (the 'covariate shift' explanation is debated)

3

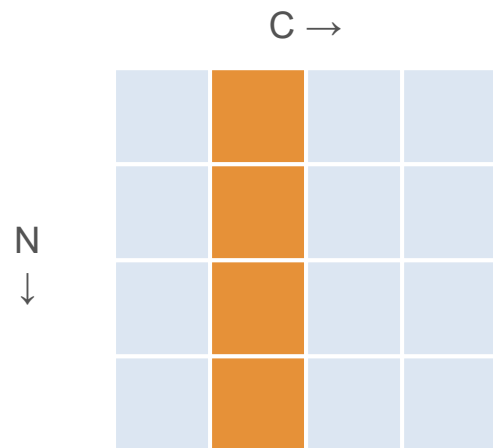
Mild regularization

batch statistics add noise → often less dropout needed

Works very well ... and is only partly understood

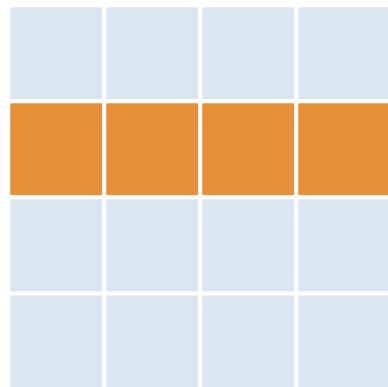
Normalization variants

Batch Norm



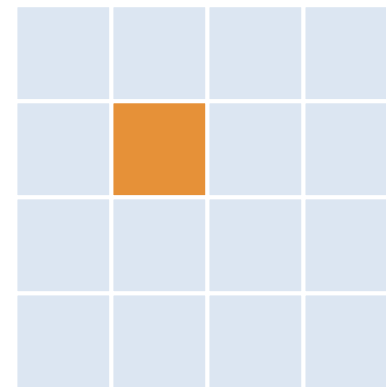
CNNs, batch ≥ 16

Layer Norm



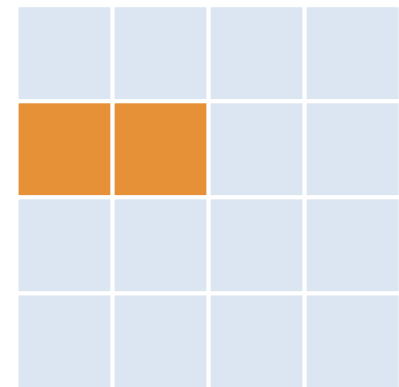
Transformers, RNNs
(module 5)

Instance Norm



style transfer

Group Norm

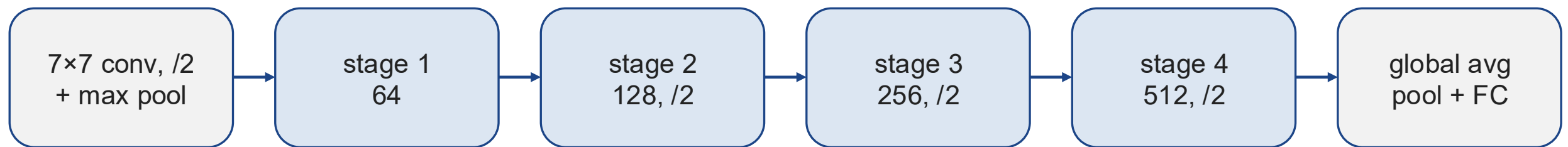


small batches: detection,
segmentation

Same formula. Only changes which values share one μ and σ (orange = one group)

Layer Norm: you will see it in every Transformer

The ResNet family

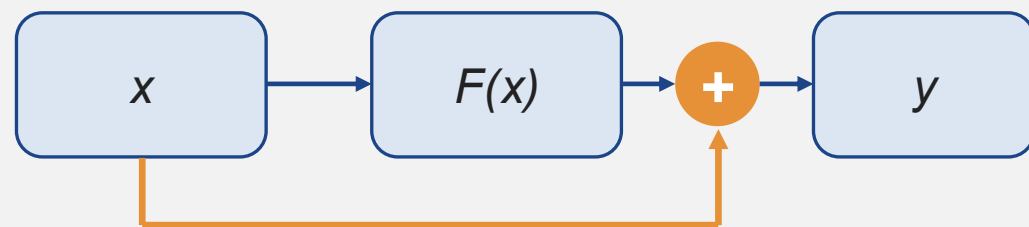


Model	Block	Blocks per stage	Params	GFLOPs	Top-1 (%)
ResNet-18	basic	2 · 2 · 2 · 2	11.7 M	1.8	69.8
ResNet-34	basic	3 · 4 · 6 · 3	21.8 M	3.7	73.3
ResNet-50	bottleneck	3 · 4 · 6 · 3	25.6 M	4.1	76.1
ResNet-101	bottleneck	3 · 4 · 23 · 3	44.5 M	7.8	77.4
ResNet-152	bottleneck	3 · 8 · 36 · 3	60.2 M	11.5	78.3
<i>VGG-16 (ref.)</i>	—	—	<i>138 M</i>	<i>15.5</i>	<i>71.6</i>

ResNet-50: 5× fewer parameters than VGG-16 and more accurate → default backbone

Skip connections everywhere

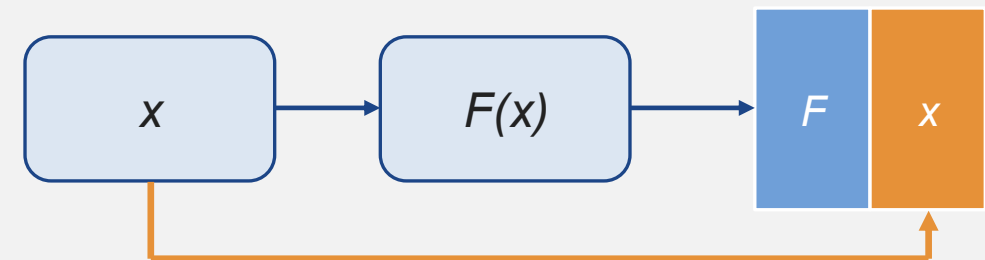
Add (ResNet)



$$y = x + F(x)$$

- same shape required
- channels stay constant
- free gradient path

Concatenate (DenseNet, U-Net)



$$y = [F(x), x] \text{ (along channels)}$$

- channels grow
- keeps earlier features intact
- fine detail for segmentation (slides 90–92)

Transformers: each sub-layer is $x + \text{Sublayer}(x)$, then Layer Norm $\rightarrow$ 'Add & Norm' (module 5)

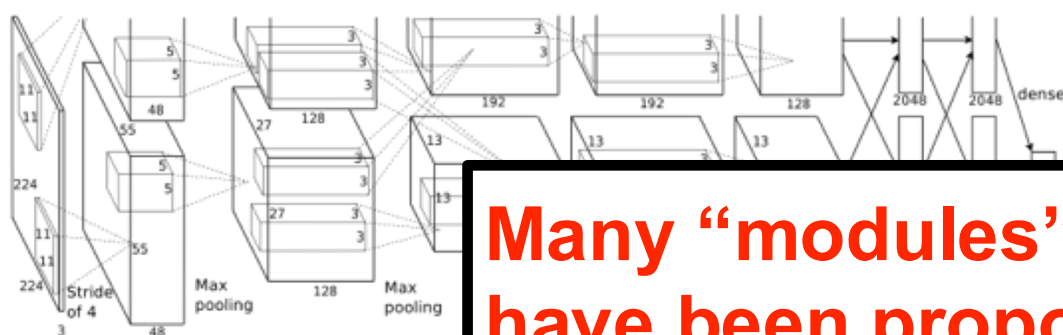
Quick check

1. A 56-layer plain CNN has higher training error than a 20-layer one. Is it overfitting?
2. Why can adding a residual block never make the network worse (in principle)?
3. 3×3 conv, $64 \rightarrow 128$ channels, followed by BN: how many learnable parameters?
4. Test accuracy changes when you evaluate with batch size 1 instead of 64. What did you forget?
5. Layer Norm instead of Batch Norm. What are the differences?



Some *reference modules* for CNN architectures ...

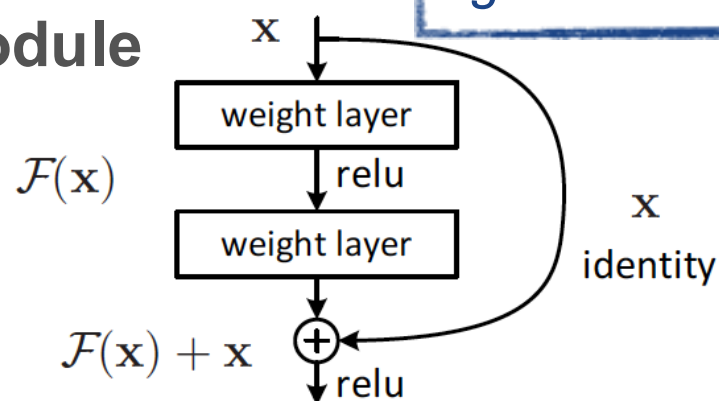
- to train better: GoogLeNet, ResNet, DenseNet, ...



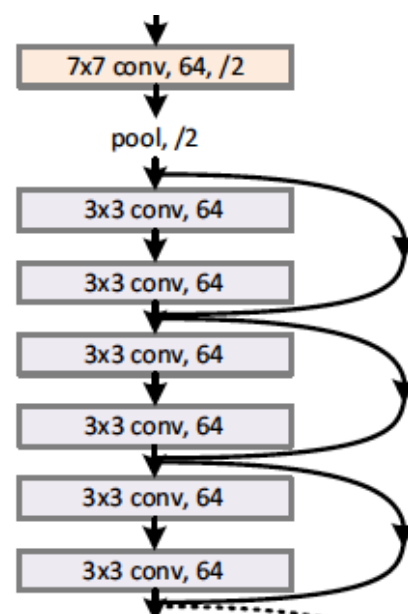
**Many “modules”
have been proposed**

ImageNet Classification with Deep Convolutional Neural Networks
[Alex Krizhevsky](#), [Ilya Sutskever](#), [Geoffrey E. Hinton](#). NIPS 2012

Residual module

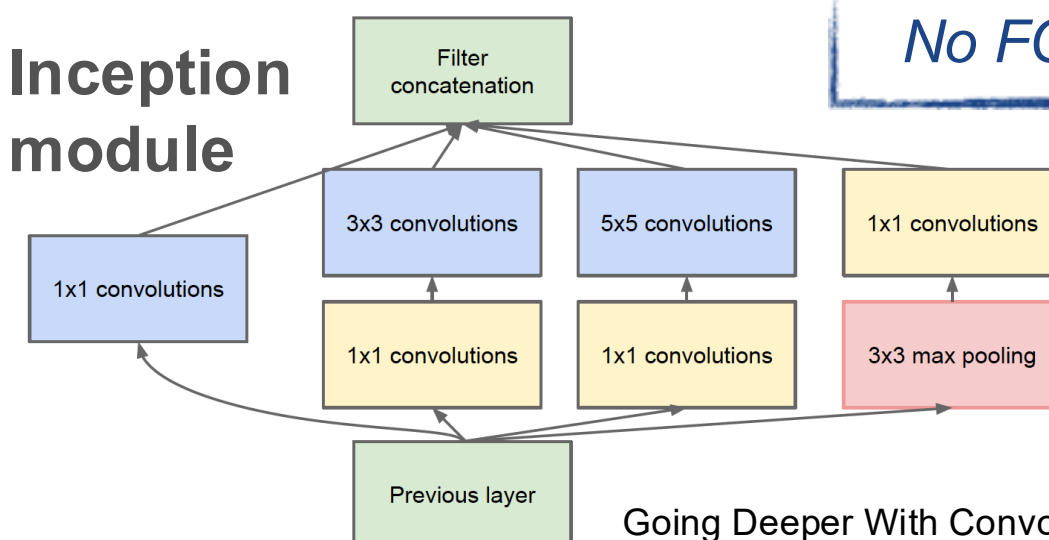


*Vanishing
gradients*



Deep residual learning for image recognition.
He, Kaiming, et al. CVPR 2016

Inception module

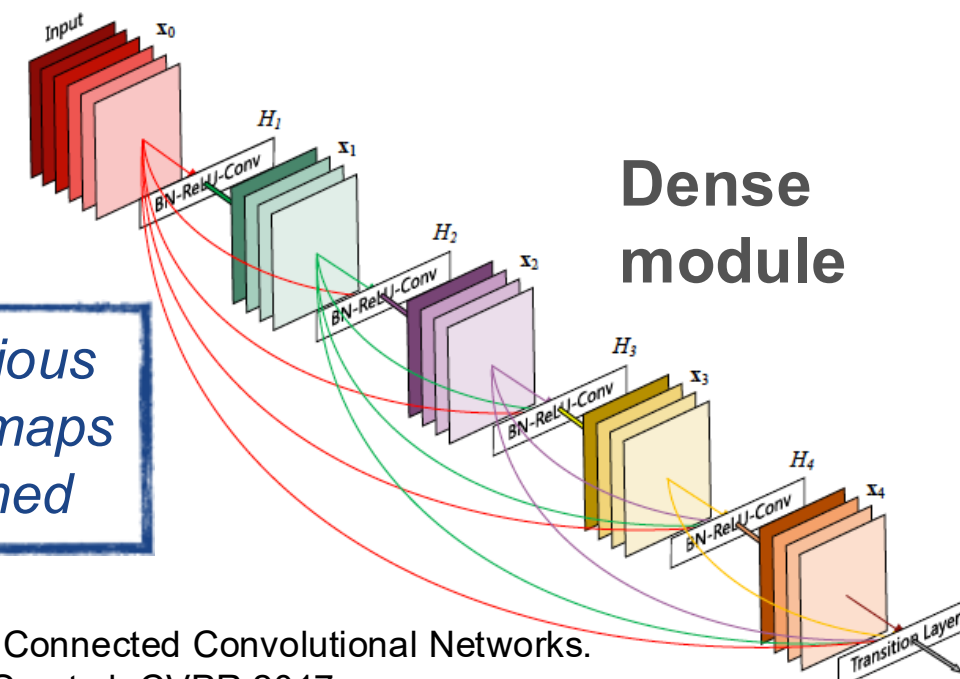


No FC

Going Deeper With Convolutions.
C.Szegedy, et al. CVPR 2015.

*All previous
feature maps
combined*

Dense module



Densely Connected Convolutional Networks.
Huang, G., et al. CVPR 2017.

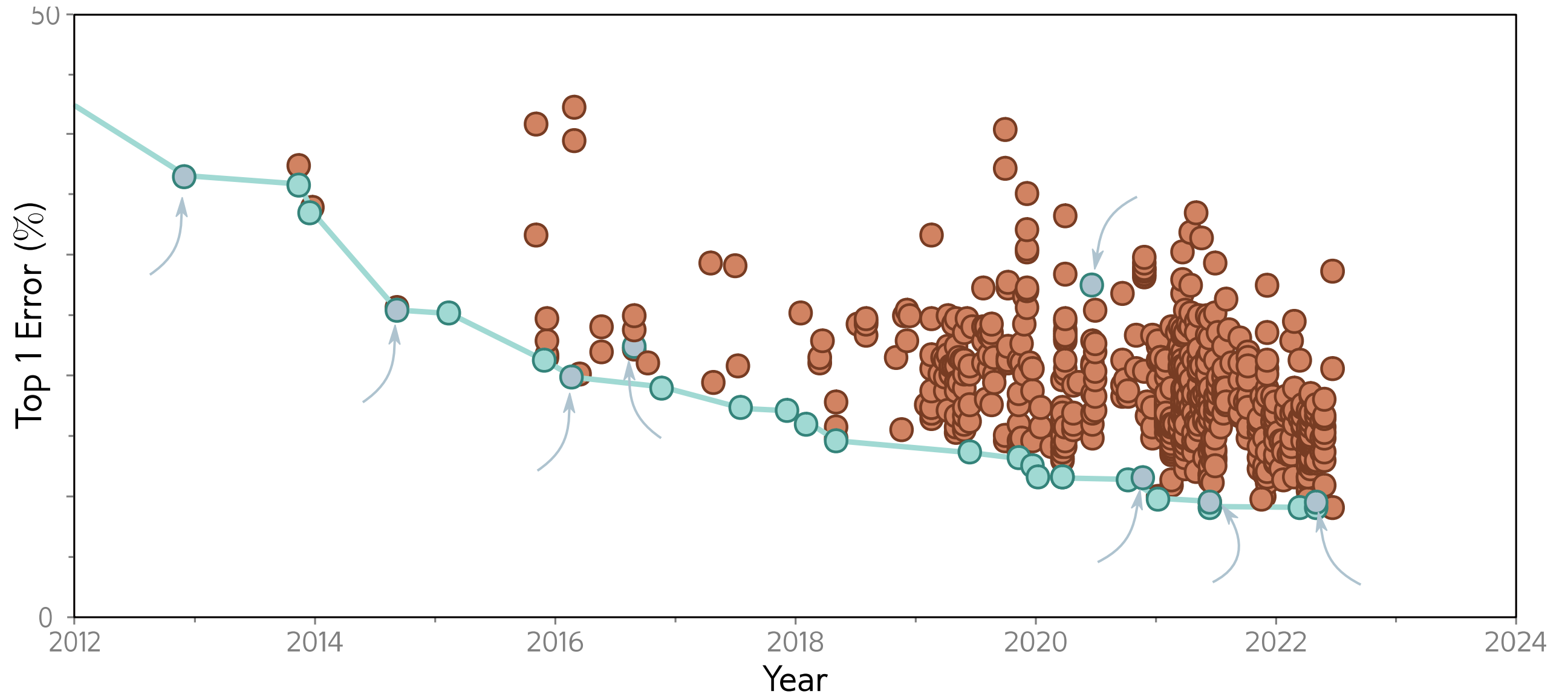
Efficient CNNs

- MobileNet (Howard et al., 2017)
 - Replaces each standard convolution with a depthwise separable convolution.
 - First, one $K \times K$ kernel filters each channel separately. Then a 1×1 convolution mixes the channels
 - Used on phones and embedded devices.
- EfficientNet (Tan & Le, ICML 2019)
 - Efficient way to make a network bigger: deeper, wider (more channels) or higher input resolution.
 - Scale all three together in fixed proportions ("compound scaling") with better performance than much larger earlier models.

CNNs in the transformer era

- Vision Transformers: ViT (Dosovitskiy et al., ICLR 2021)
 - Treats the 16x16 image patches as words in a sentence, no convolutions
- ConvNeXt (Liu et al., CVPR 2022)
 - Implements ideas of transformers (larger kernels, better training recipes and fewer normalization and activation layers)
 - Showed that most of Transformer's advantage in ViT like models came from training and design, not only from attention

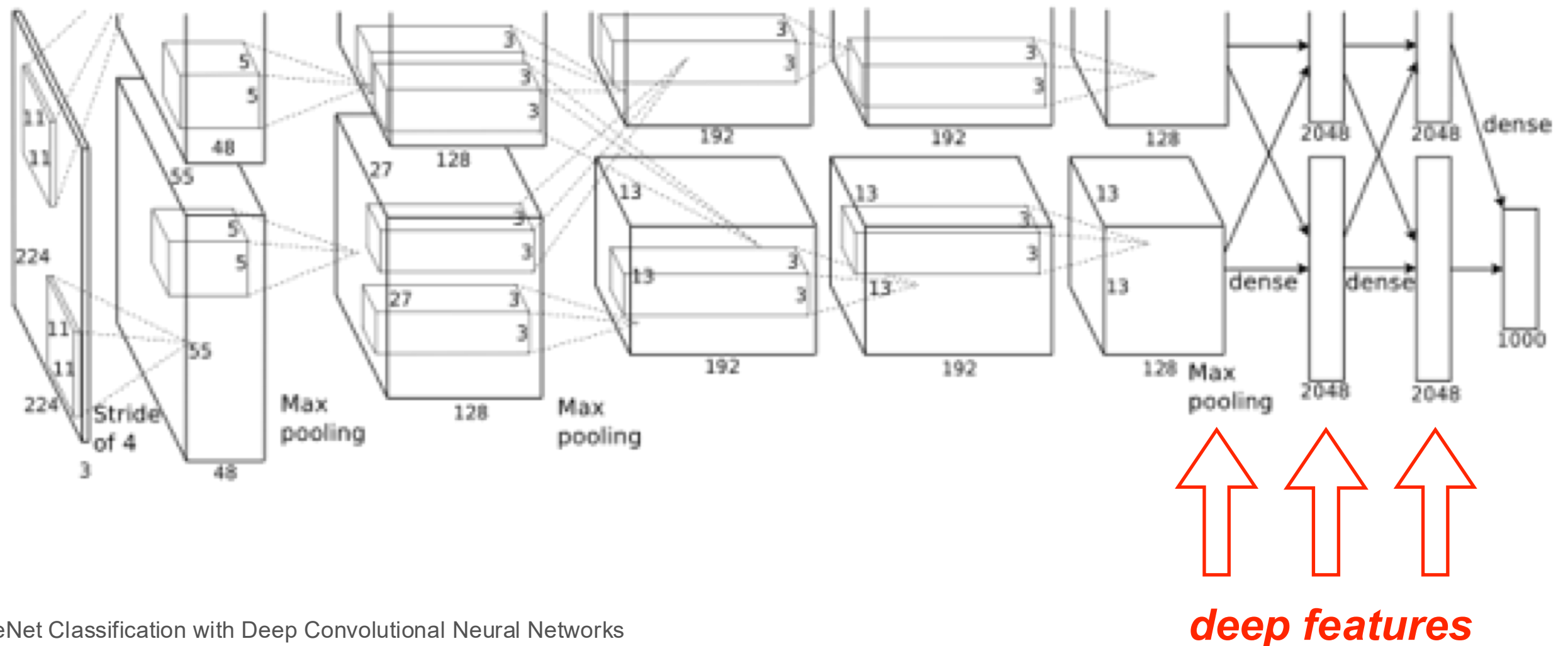
A decade of ImageNet classification



ImageNet top-1 error over time. Understanding Deep Learning, S. J. D. Prince, MIT Press.

CNNs & Transfer Learning

- CNNs are able to generalize well!
 - great **features**
 - **fine-tuning**

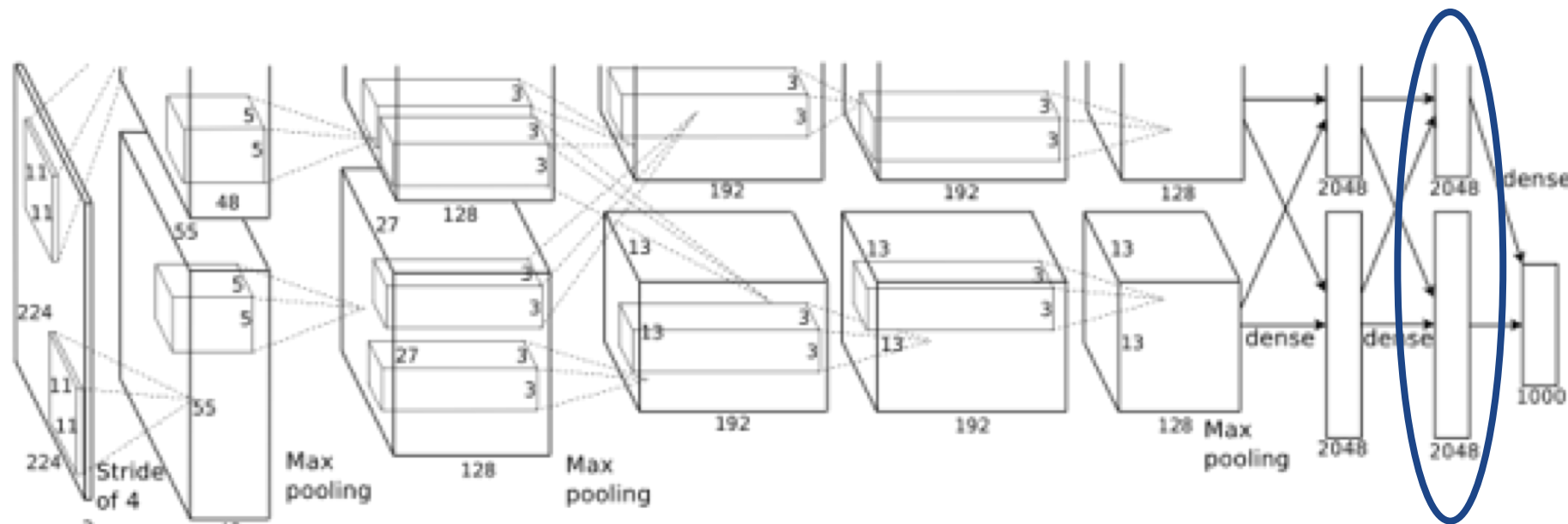


Transfer Learning

- What is it?
- When would you use transfer learning?
- Types of transfer learning?

Transfer Learning: features

- E.g., features from AlexNet last layer: 4096 dims. vector



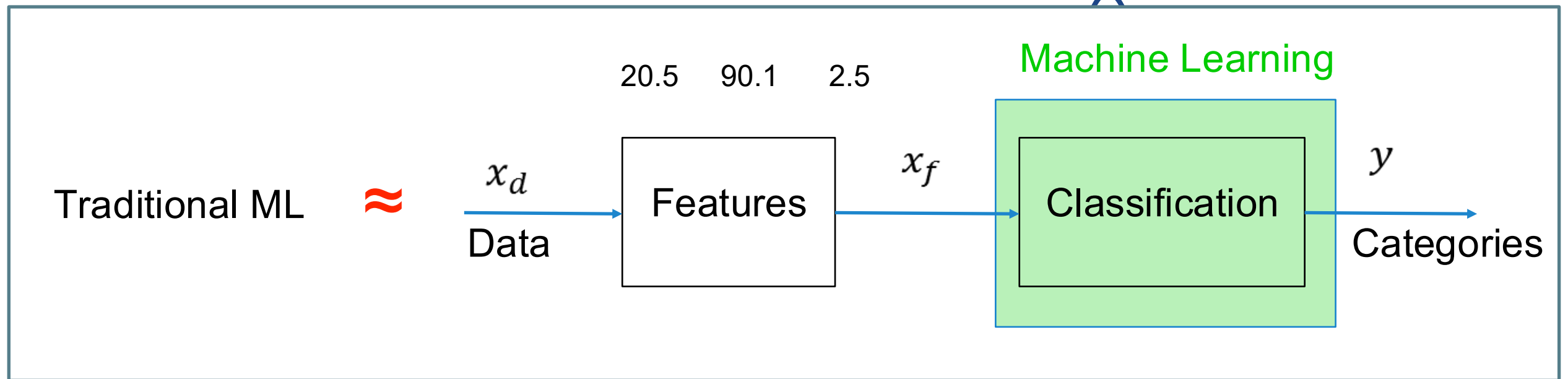
How do we get these features?

Test image L2 Nearest neighbors in feature space



Transfer Learning: features

- E.g., features from AlexNet last layer: 4096 dims. vector



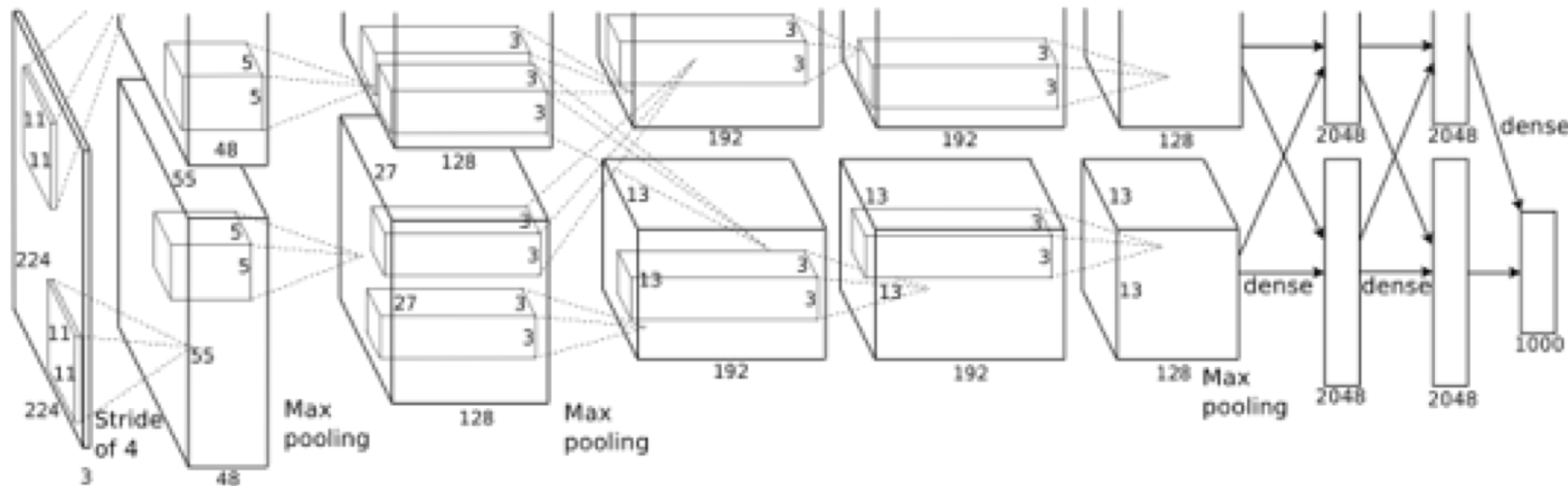
How do we use these features?

Test image L2 Nearest neighbors in feature space



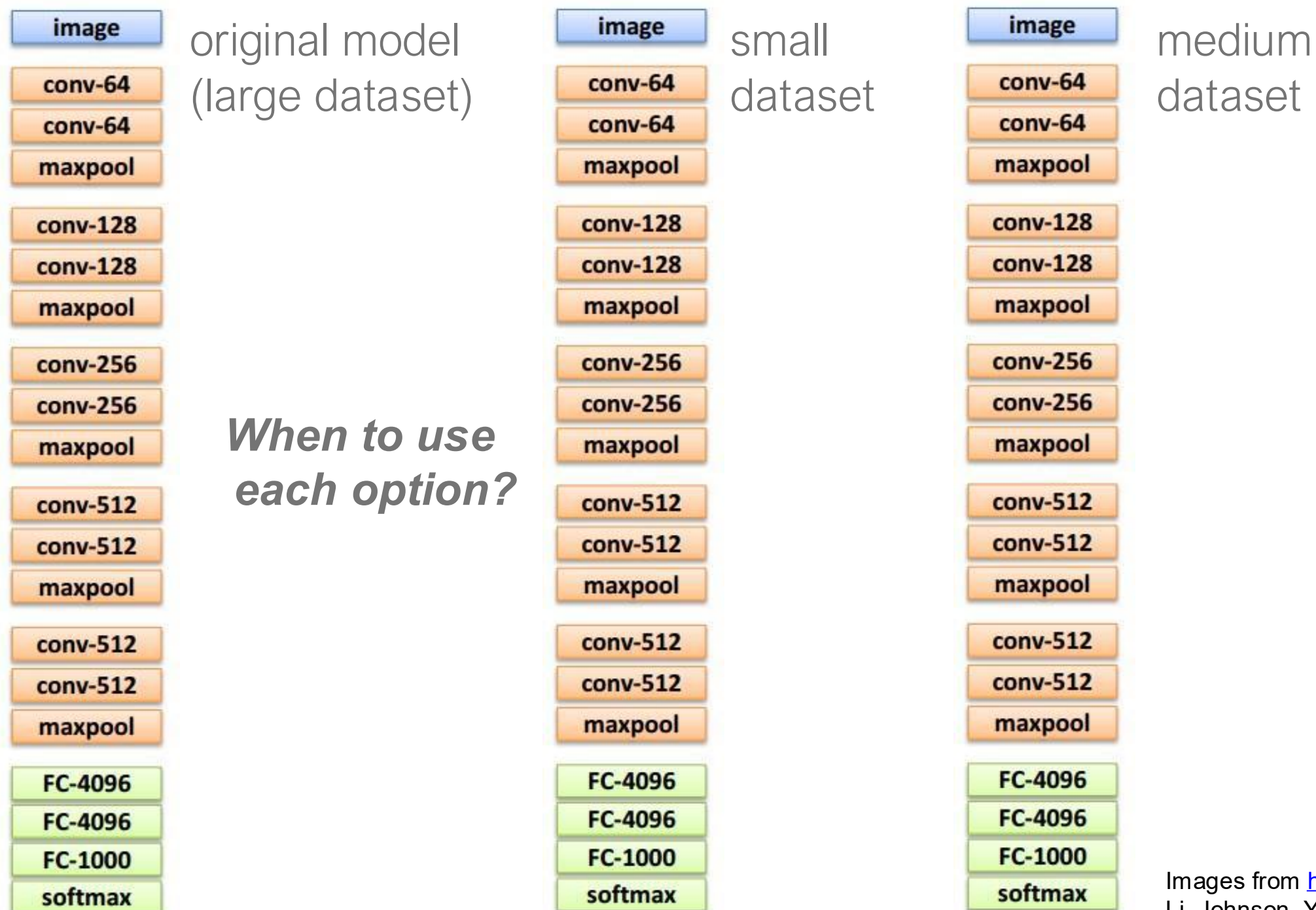
Transfer Learning: fine-tune

- For example, fine-tune ImageNet AlexNet for non ImageNet classes
- Basically, initialise weights to something more “*interesting*” than random (careful also with hyperparameters!)



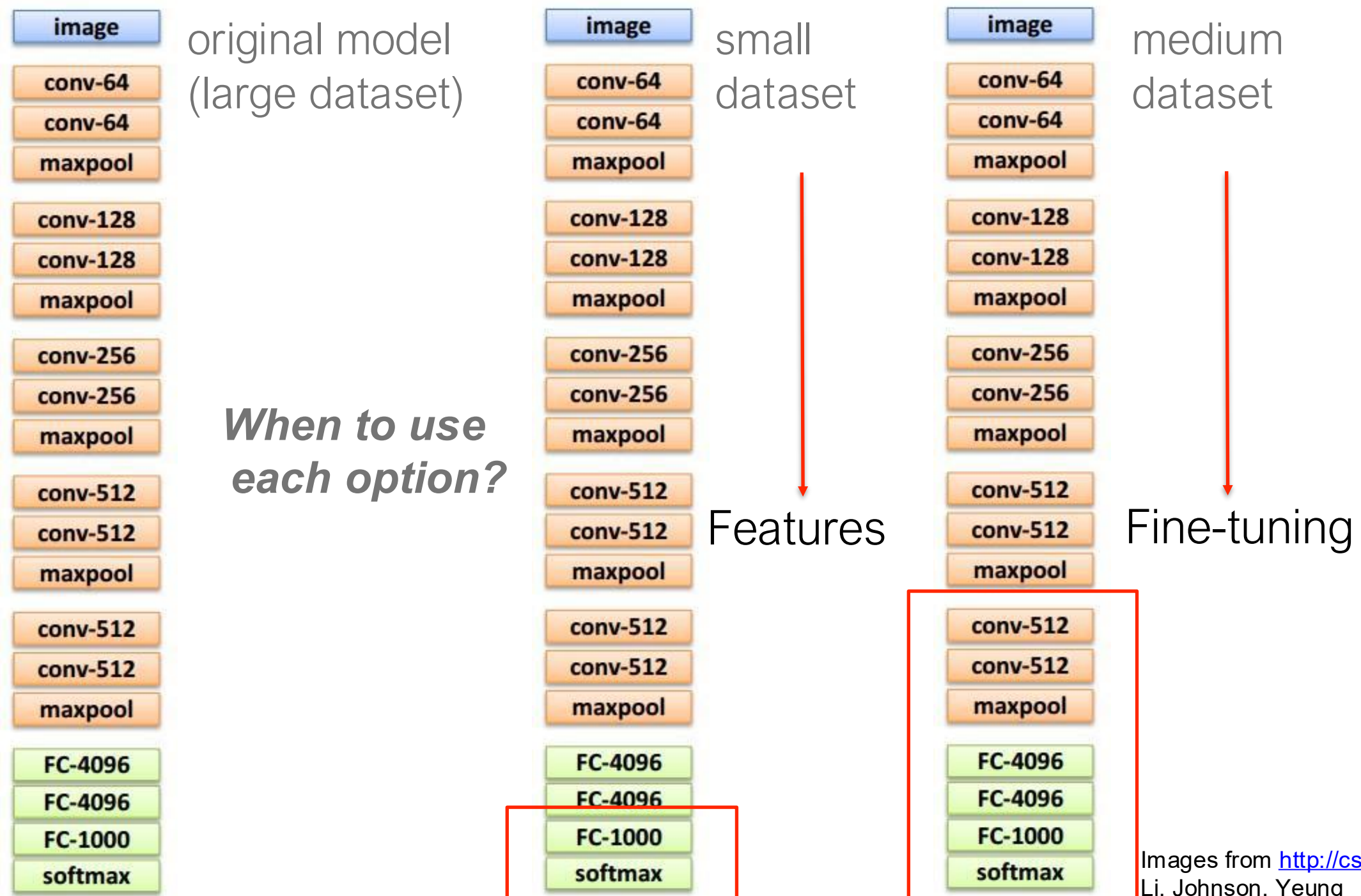
Transfer Learning

... when not enough resources to train from scratch



Transfer Learning

... when not enough resources to train from scratch



Transfer Learning: fine-tuning

- not just that! very very common strategy:

Object Detection (Fast R-CNN)

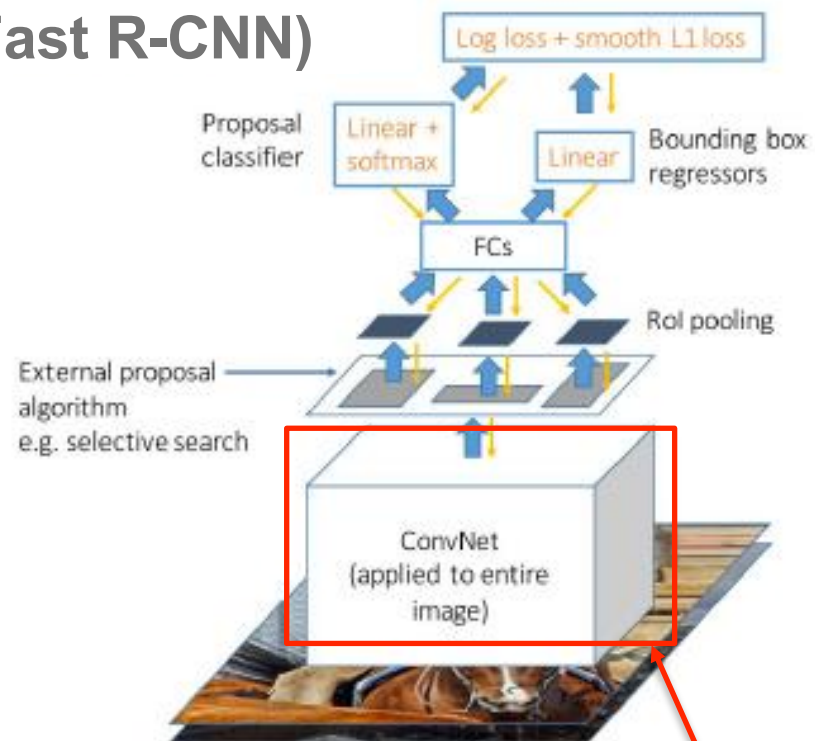
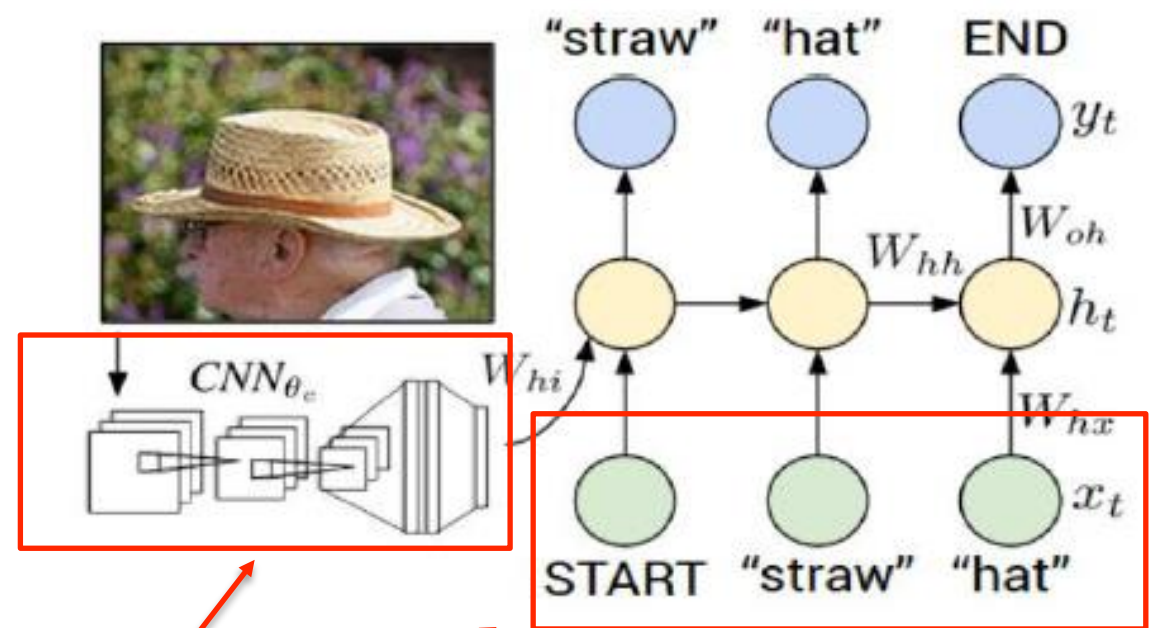


Image captioning: CNN + RNN



*Pre-trained
models to
initialize
certain blocks*

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

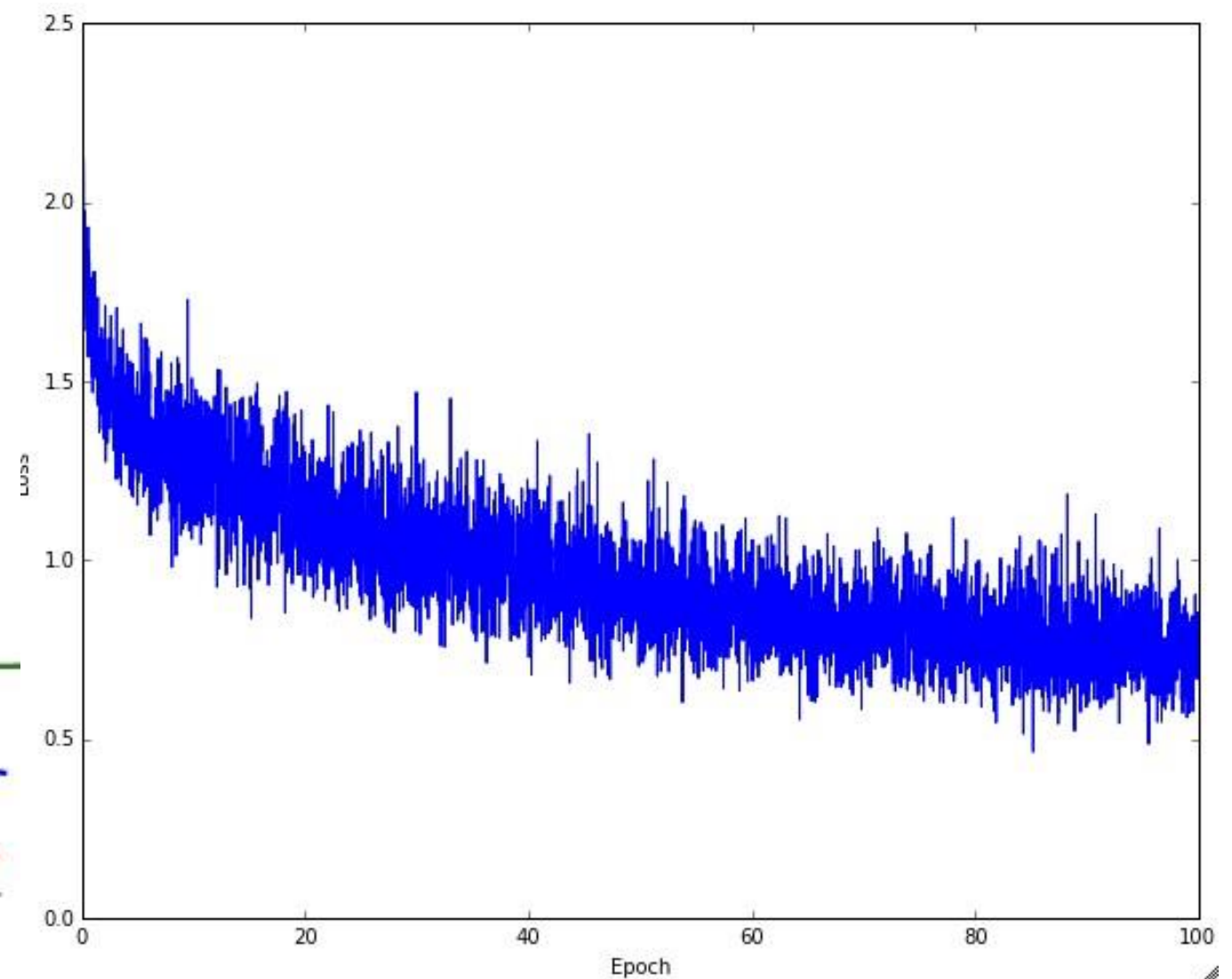
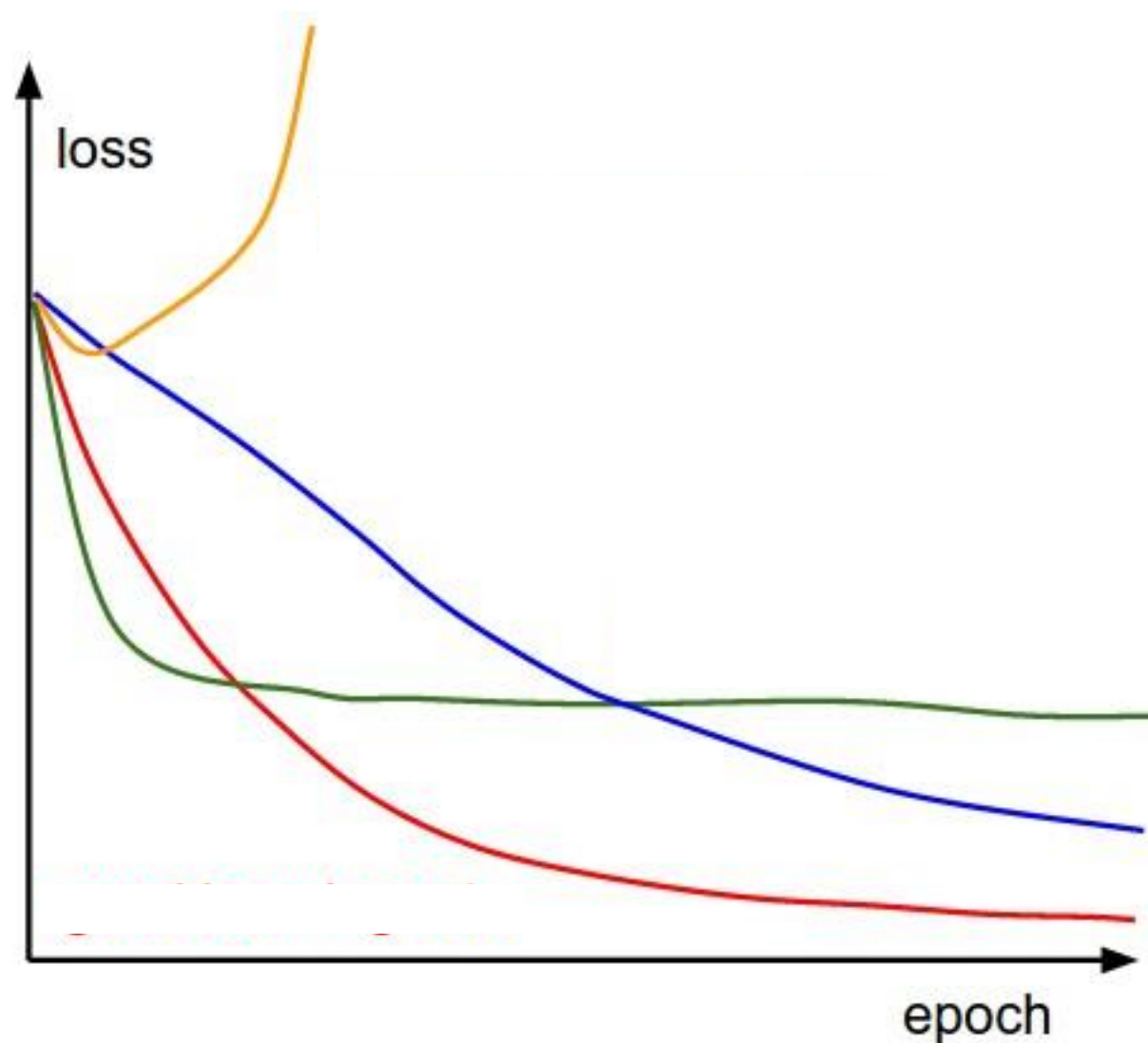
Understanding the training process

Initial **sanity checks** (after a few “toy” iterations)

- ✓ *Correct* loss at chance
- ✓ Increasing regularisation strength increases loss
- ✓ Overfit a tiny training set (you can set regularisation strength to zero to better see this)

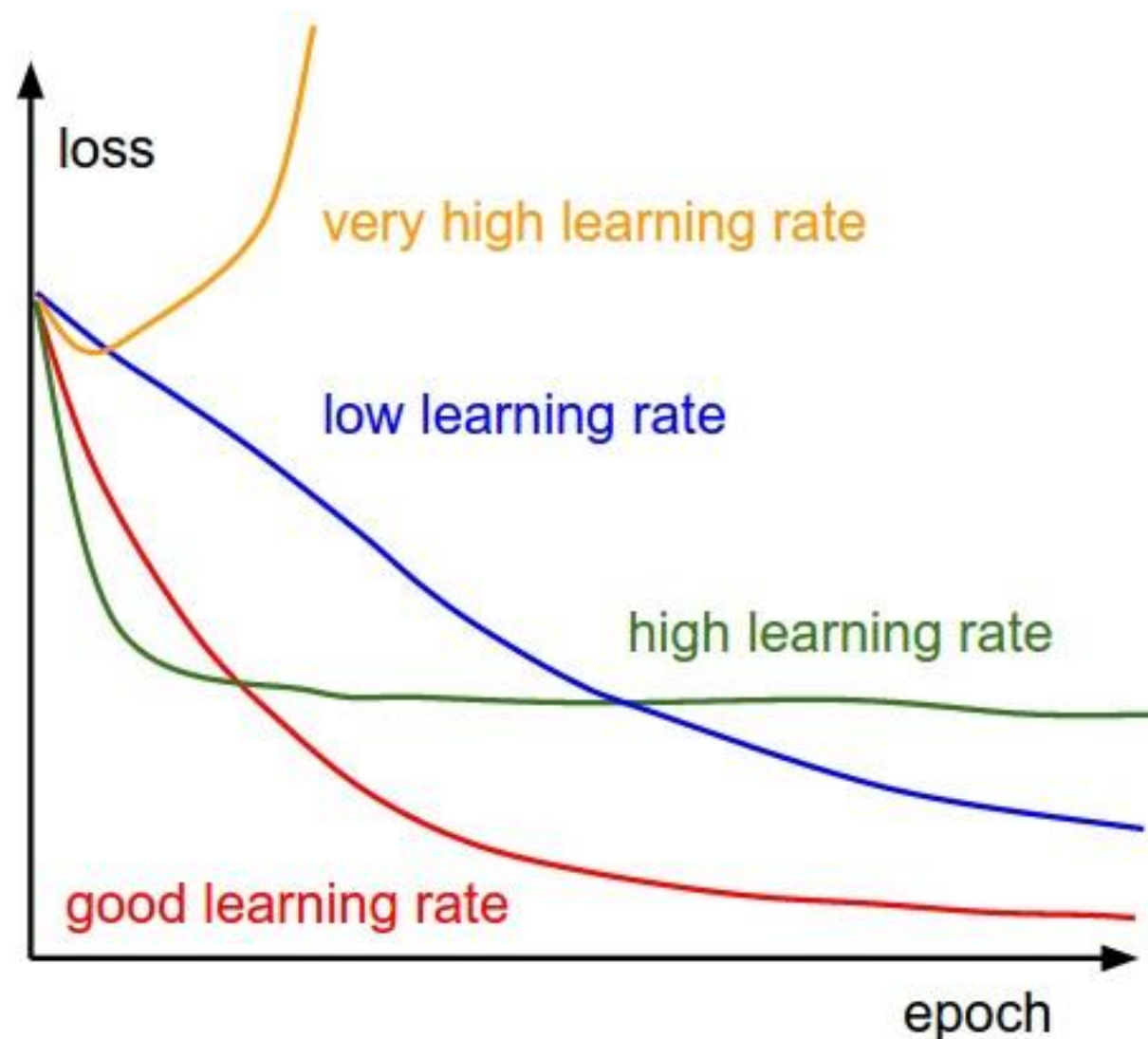
Understanding the training process

- Monitor the loss function

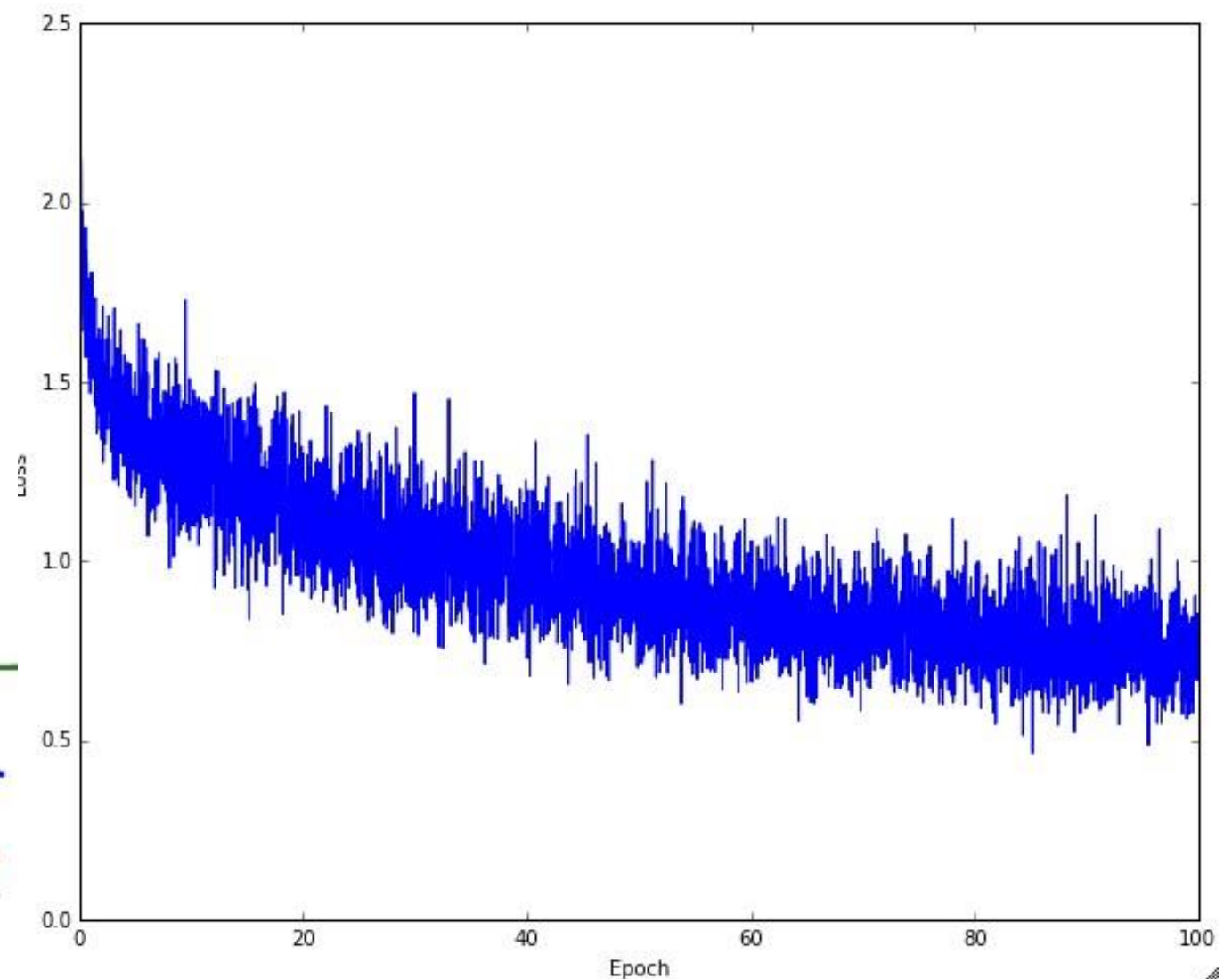


Understanding the training process

- Monitor the loss function

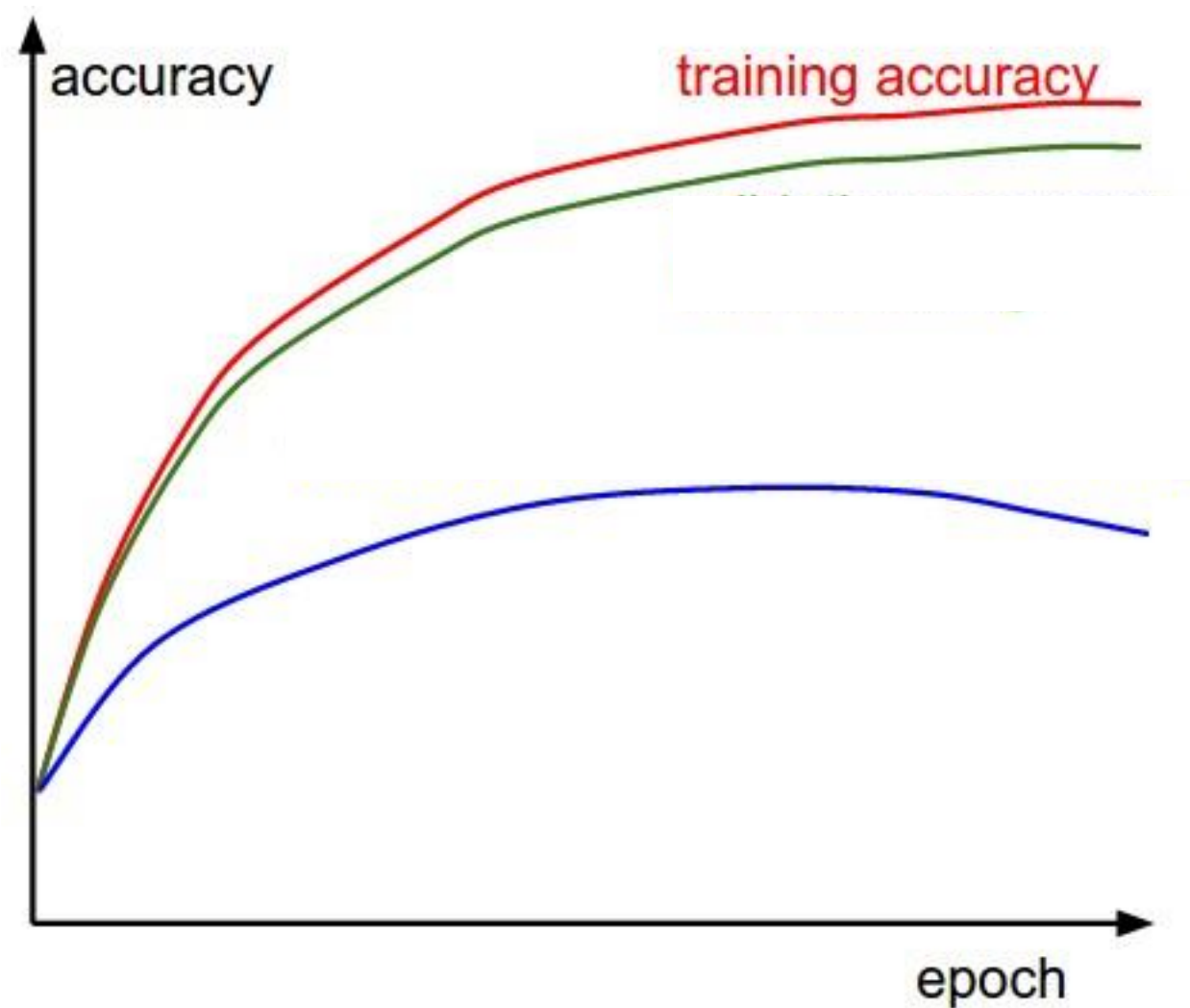


Typical loss function.
Batch size a little low? (noisy plot)



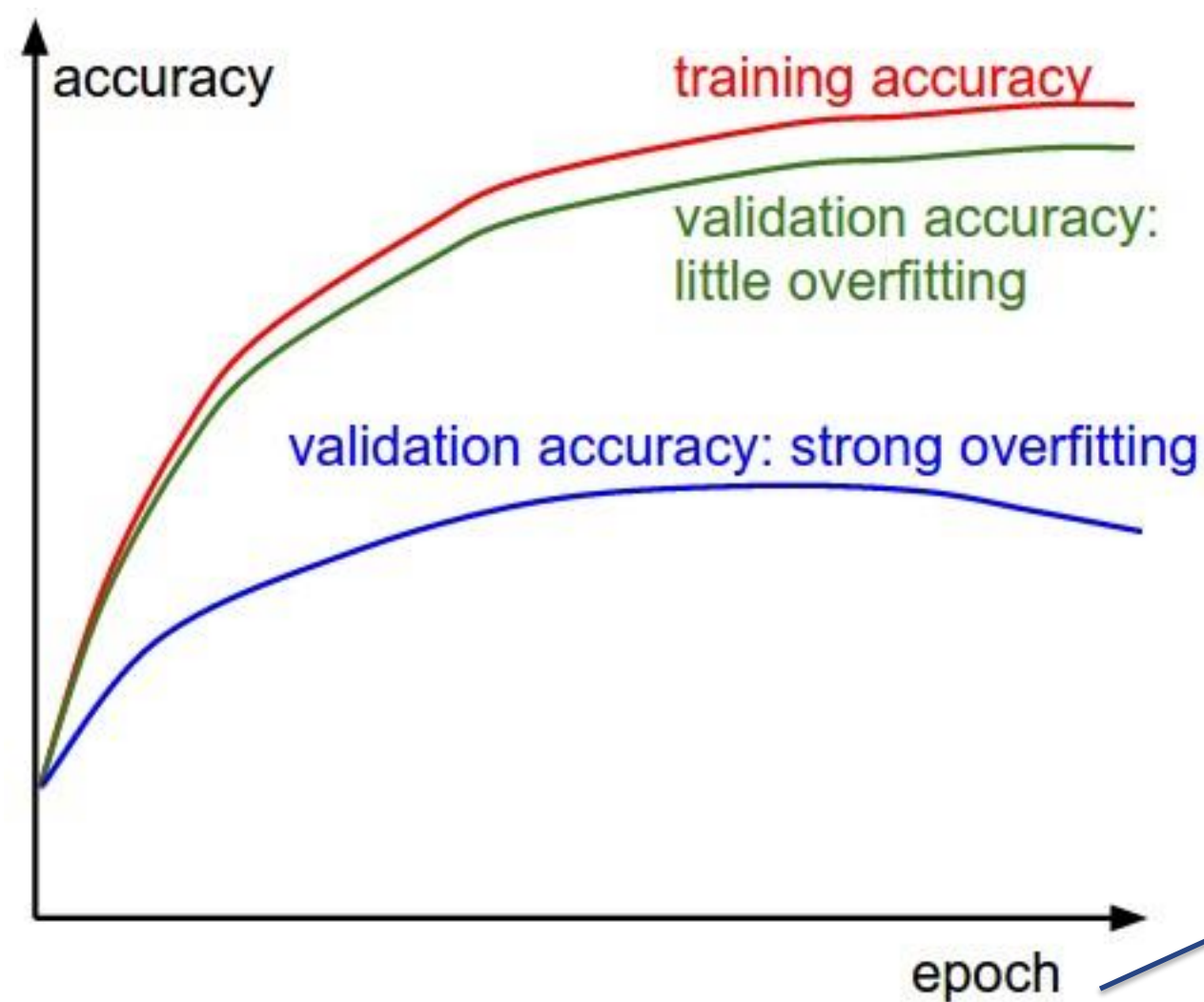
Understanding the training process

- Monitor **training/validation** accuracy



Understanding the training process

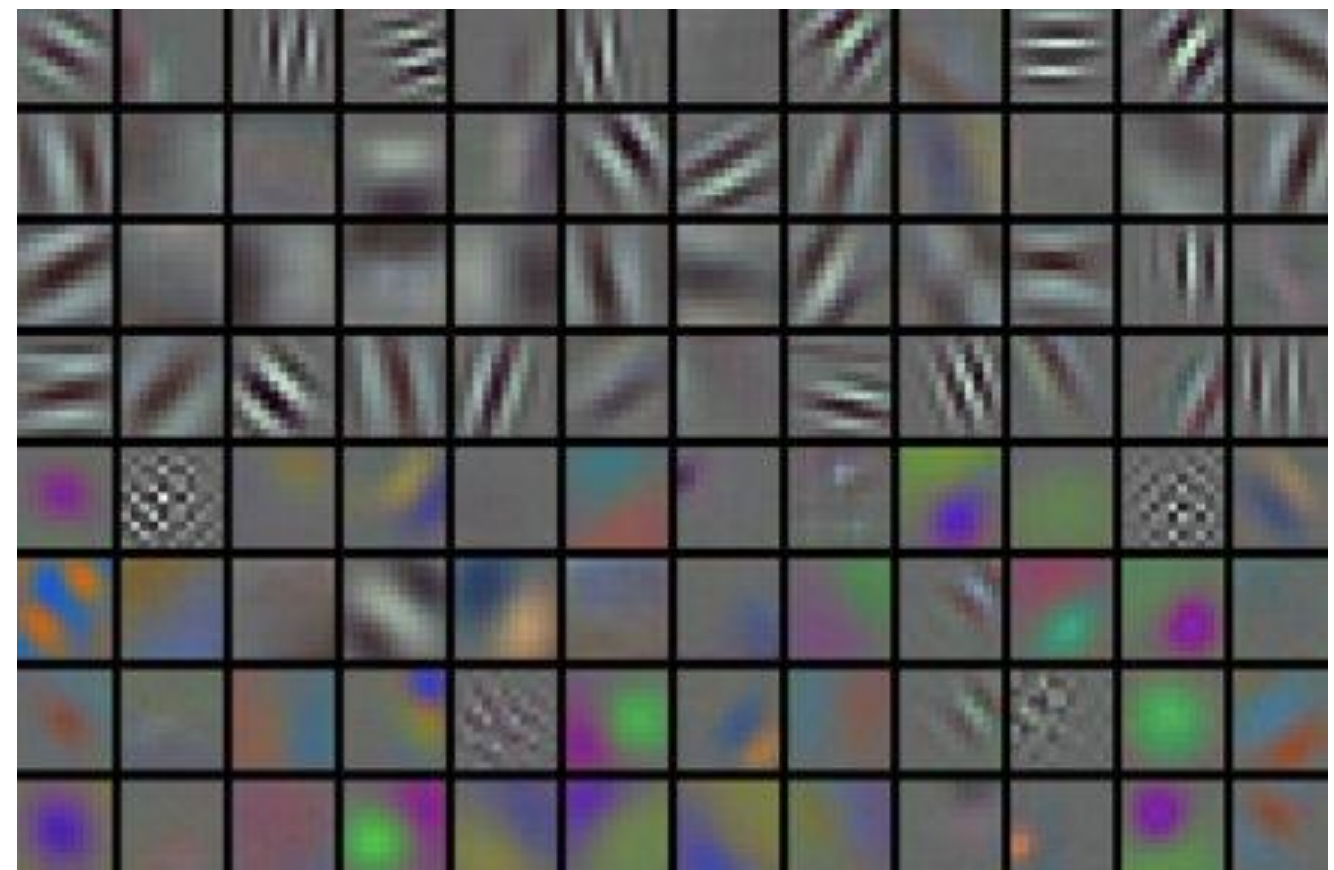
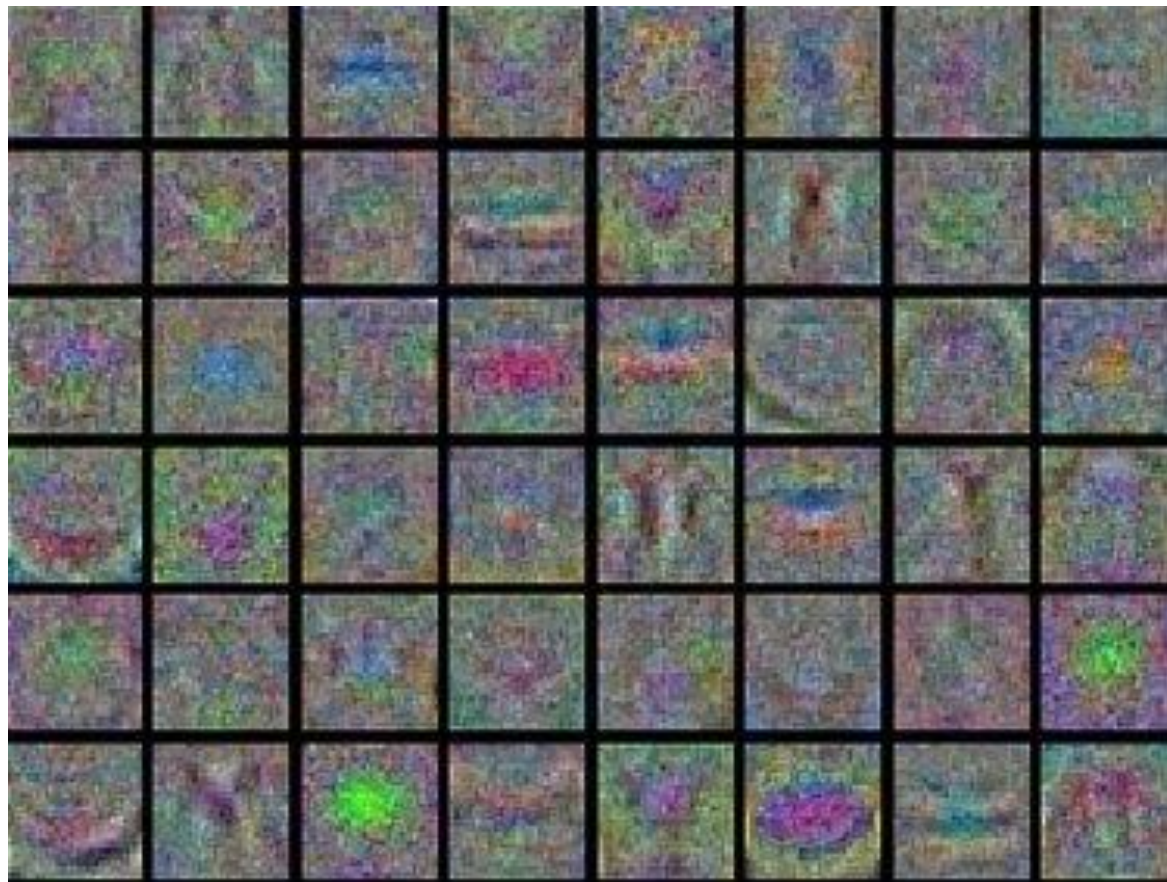
- Monitor **training/validation** accuracy



***What's an
epoch?***

Understanding the training process

- Visualize first conv layer weights



***What's a good
sign?***

Understanding the training process

Additional tips:

- **Decay your learning rate** during training
- If you can afford it
 - **Search** for good **hyperparameters** with random search (not grid search). From coarse to fine
 - Consider **model ensembles** (not in our labs)

Open/Interesting problems
related to deep learning?

**Perform more *complex* tasks
than image classification?**

Deep Learning & Regression

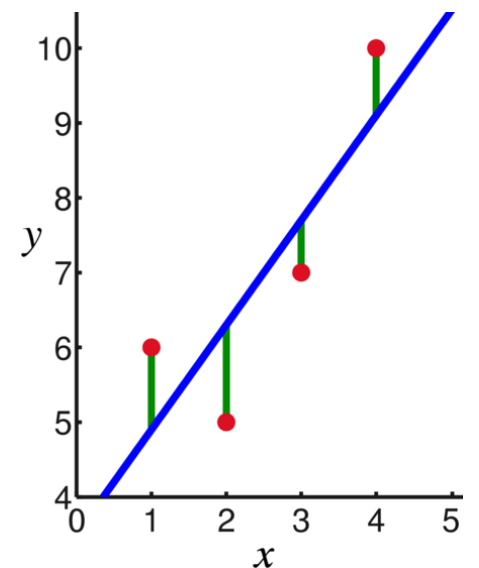
- Linear regression: relationship between a scalar (y) and one or more explanatory variables (x)

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i, \quad i=1, \dots, n$$

“map” to a continuous output (regression)

VS

“map” to a discrete output (classification)



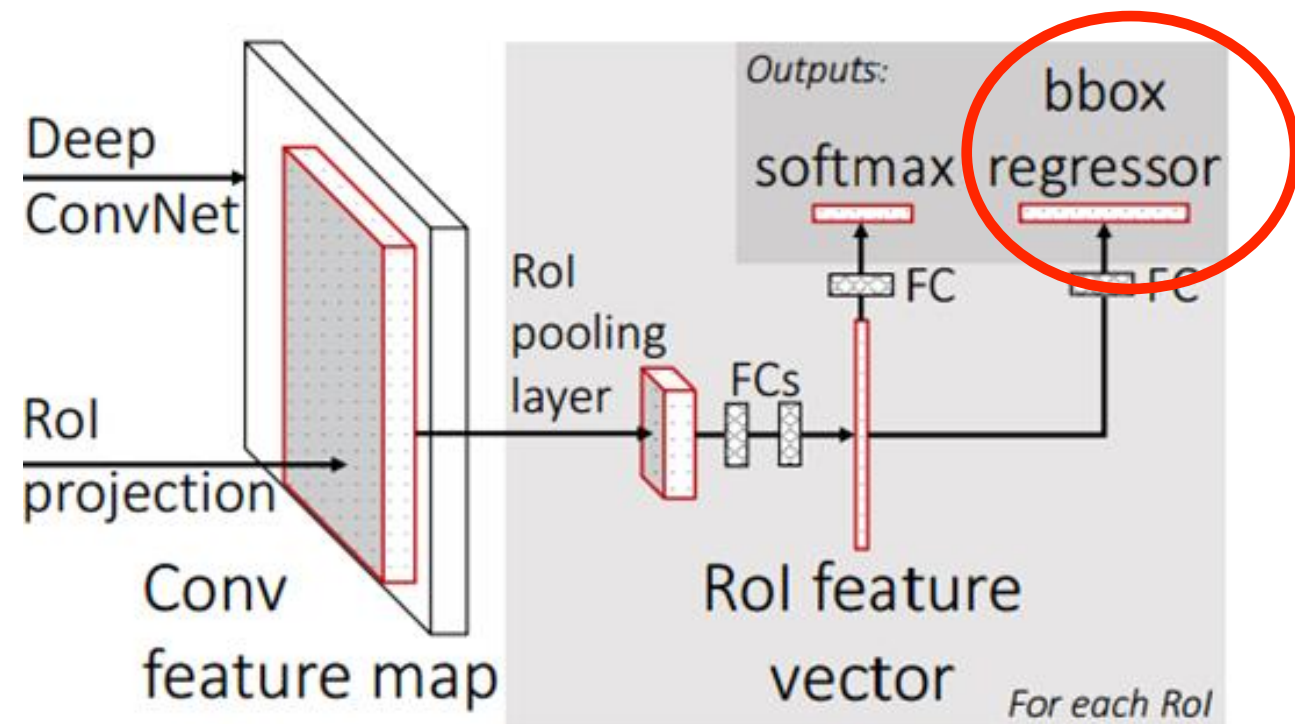
Wikipedia:
File:Linear least squares example2.png

Deep Learning & Regression

- **Detection**

predict normalized b-box coordinates

$$p = (x_1, y_1, x_2, y_2)$$



Fast R-CNN. Ross Girshick. 2015

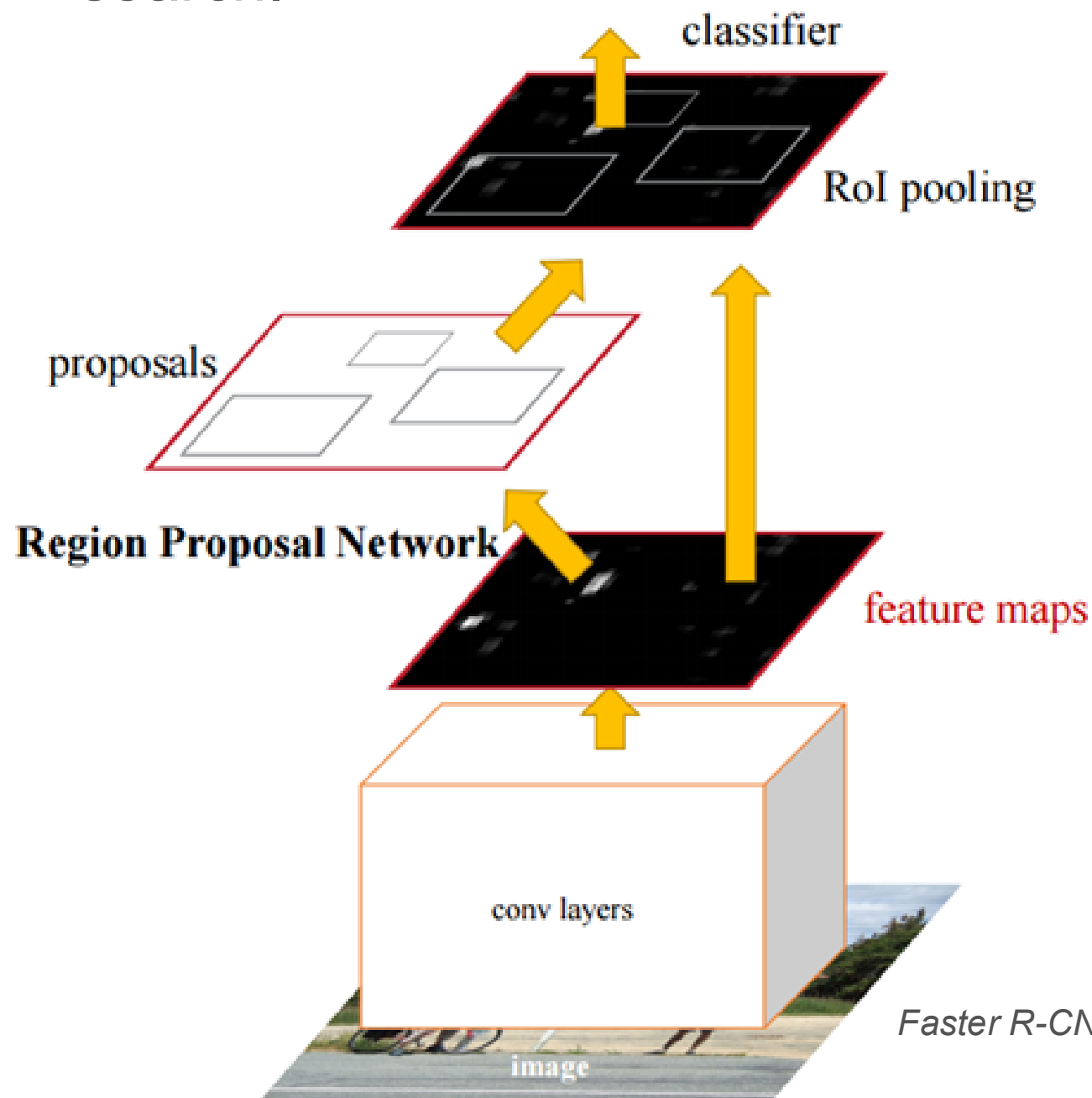
Different versions of Region-CNN:

- R-CNN – 2014 (CVPR)
- Fast R-CNN – 2015 (ICCV)

Detection: Faster R-CNN, learning where to look

- **Analyze the *feature maps*: Region - CNN**

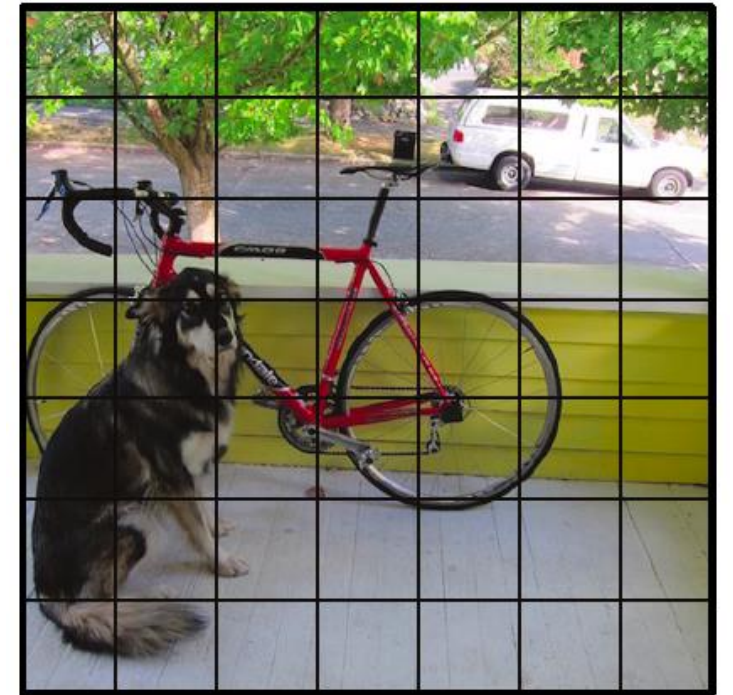
The network also learns the proposals (RPN), with no external region search.



**Analyze feature maps
(activations) to learn
where the objects are**

YOLO – single-shot detection

- Network inspired by GoogLeNet, 448x448 input
- 7×7 grid of locations
- Predict class at each location
- Predict 2 bounding boxes at each location
 - Five parameters – x, y, w, h, and confidence
- Momentum, weight decay, dropout, and data augmentation
- Non maximum suppression: Heuristic at the end to threshold and decide final boxes

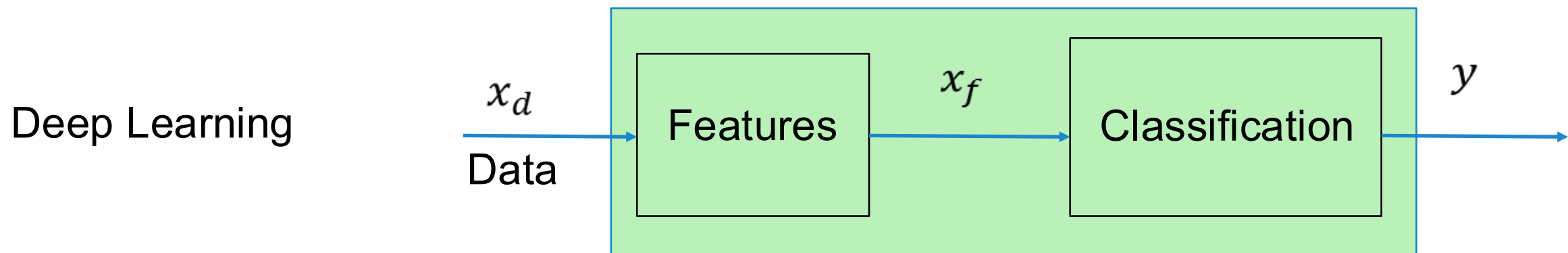


Deep Learning & Regression

- Regression of **Pose** - *PoseNet*
 - Train the network to output 3D position ($\mathbf{x}$) and orientation ($\mathbf{q}$) (7 dims). Loss function:

$$loss(I) = \|\hat{\mathbf{x}} - \mathbf{x}\|_2 + \beta \left\| \hat{\mathbf{q}} - \frac{\mathbf{q}}{\|\mathbf{q}\|} \right\|_2$$

- Modify GoogLeNet:



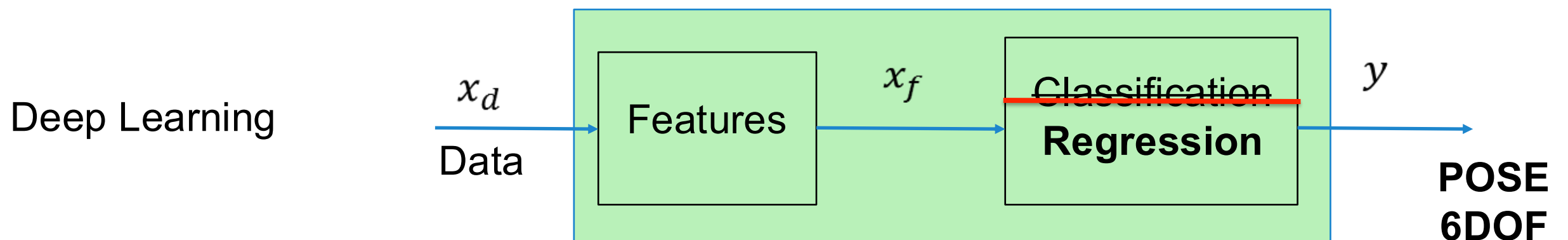
Deep Learning & Regression

- Regression of **Pose** - *PoseNet*

- Train the network to output 3D position ($\mathbf{x}$) and orientation ($\mathbf{q}$) (7 dims). Loss function:

$$loss(I) = \|\hat{\mathbf{x}} - \mathbf{x}\|_2 + \beta \left\| \hat{\mathbf{q}} - \frac{\mathbf{q}}{\|\mathbf{q}\|} \right\|_2$$

- Modify GoogLeNet. Key change: replace softmax classifiers with affine regressors (each final fully connected layer now outputs a pose vector of 7-dims).

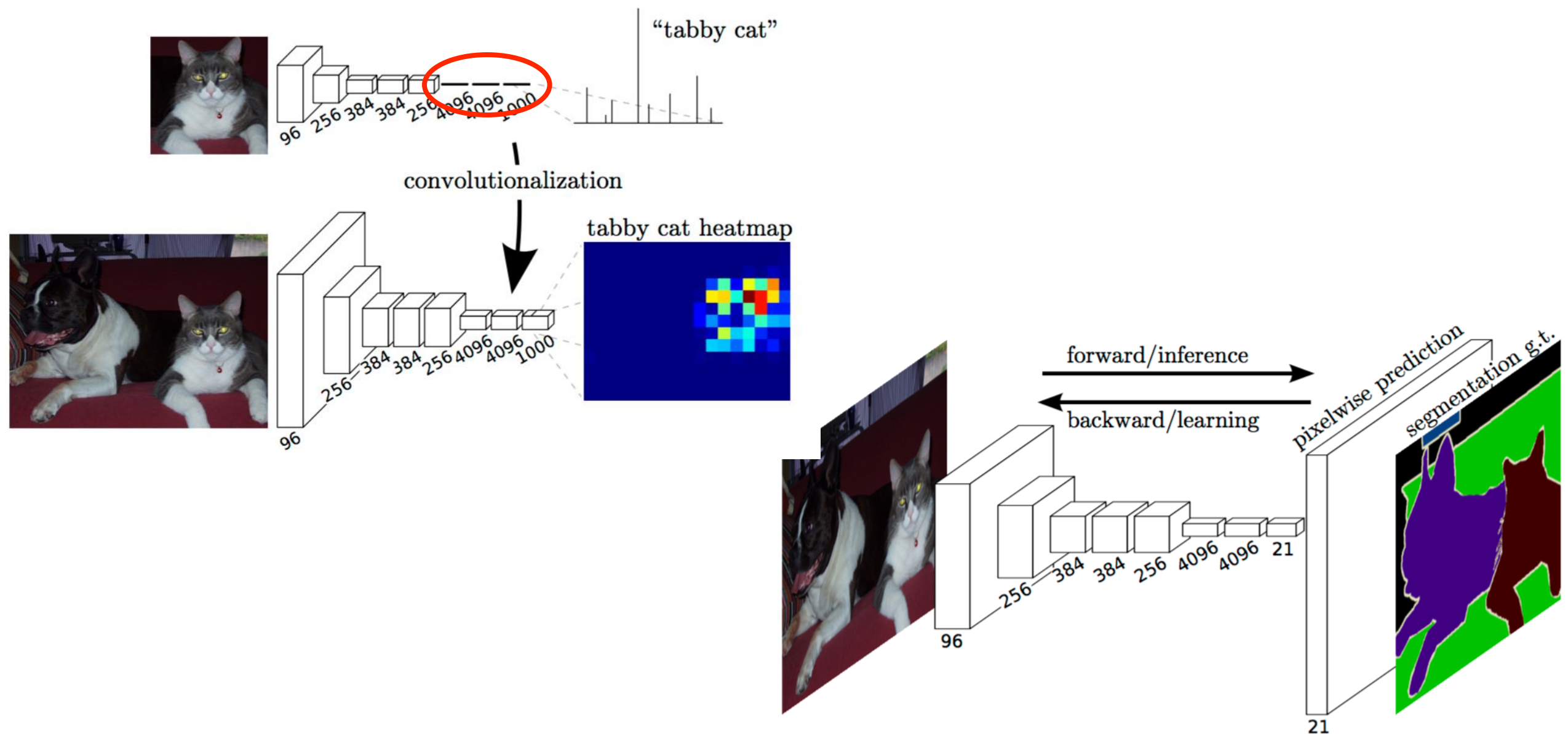


More architectures ...

More accurate image understanding?

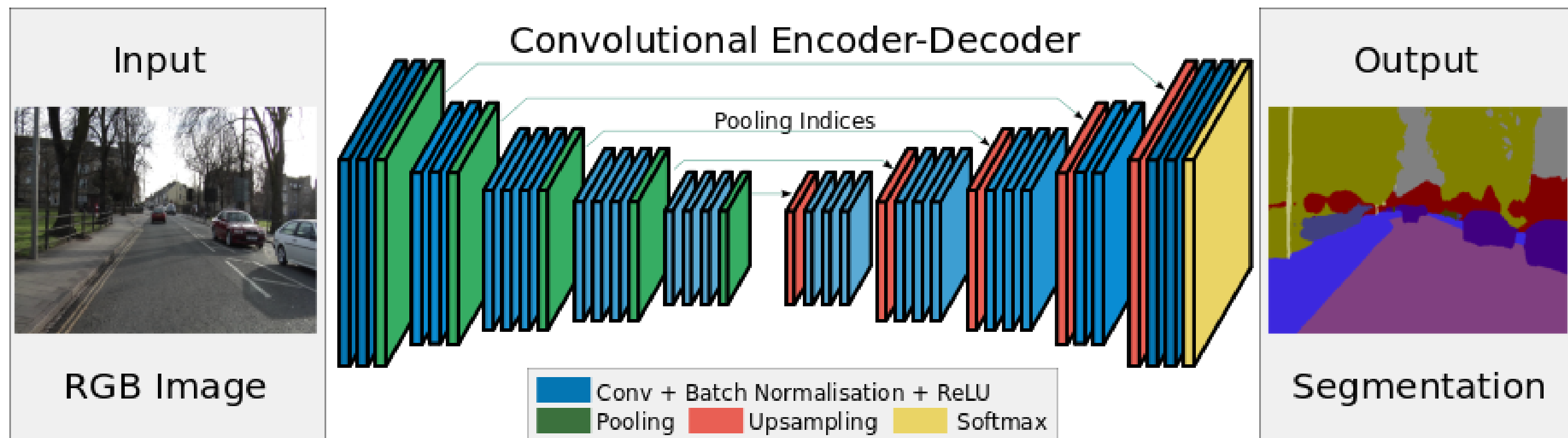
Deep Learning & pixel classification

- Dense labeling/**Segmentation** - Fully Convolutional Net (FCN)



Deep Learning & pixel classification

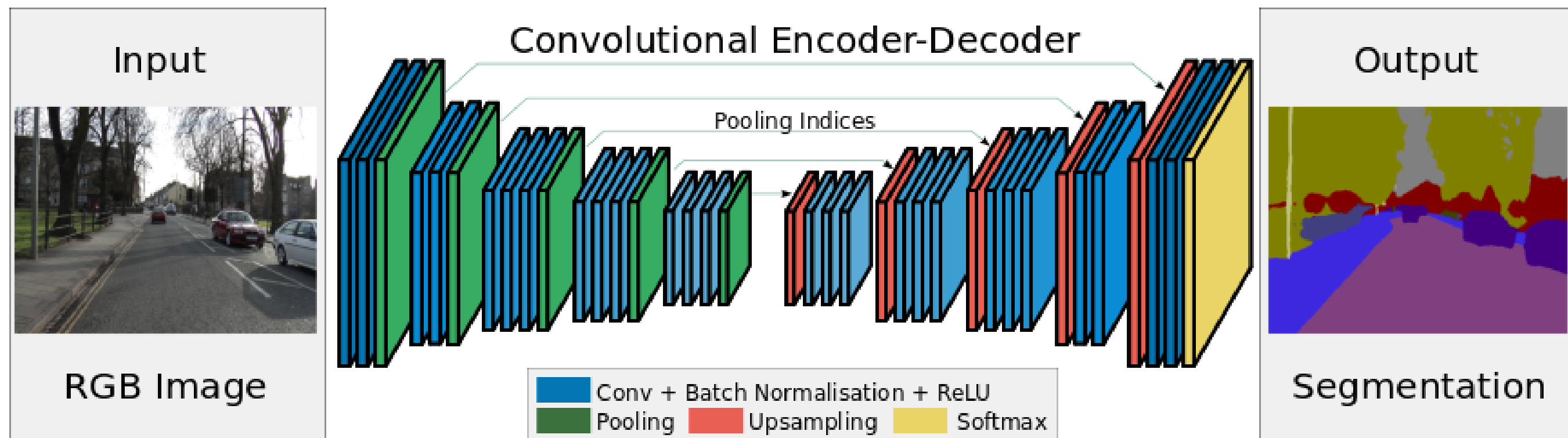
- Dense labeling/**Segmentation** - Encoder-Decoder



<http://mi.eng.cam.ac.uk/projects/segnet/>

Deep Learning & pixel classification

- Dense labeling/**Segmentation** - Encoder-Decoder



<http://mi.eng.cam.ac.uk/projects/segnet/>

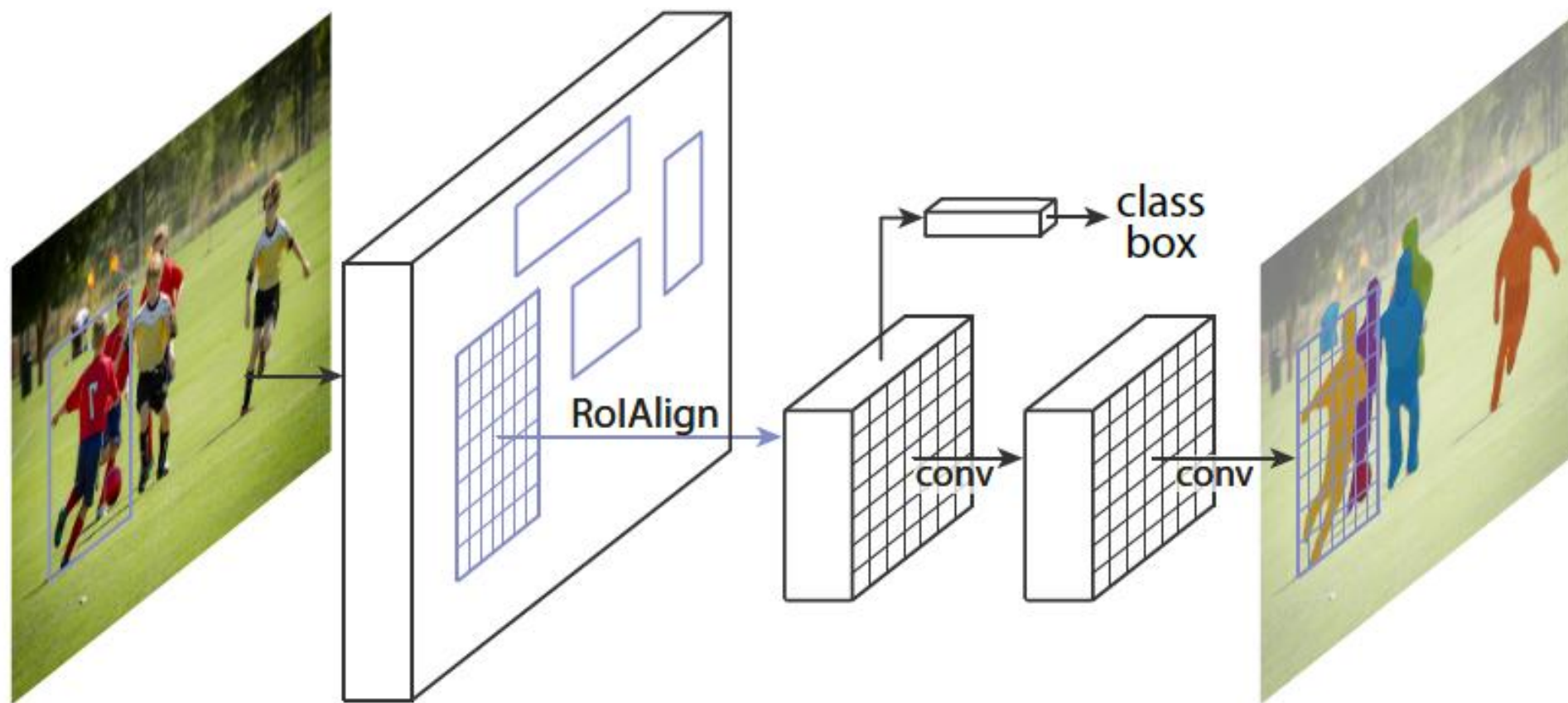
How many cars are there?

Deep Learning & pixel classification

- **Detection + Instance Segmentation**

Mask R-CNN - 2017

(R-CNN + Semantic Segmentation)



Mask R-CNN.

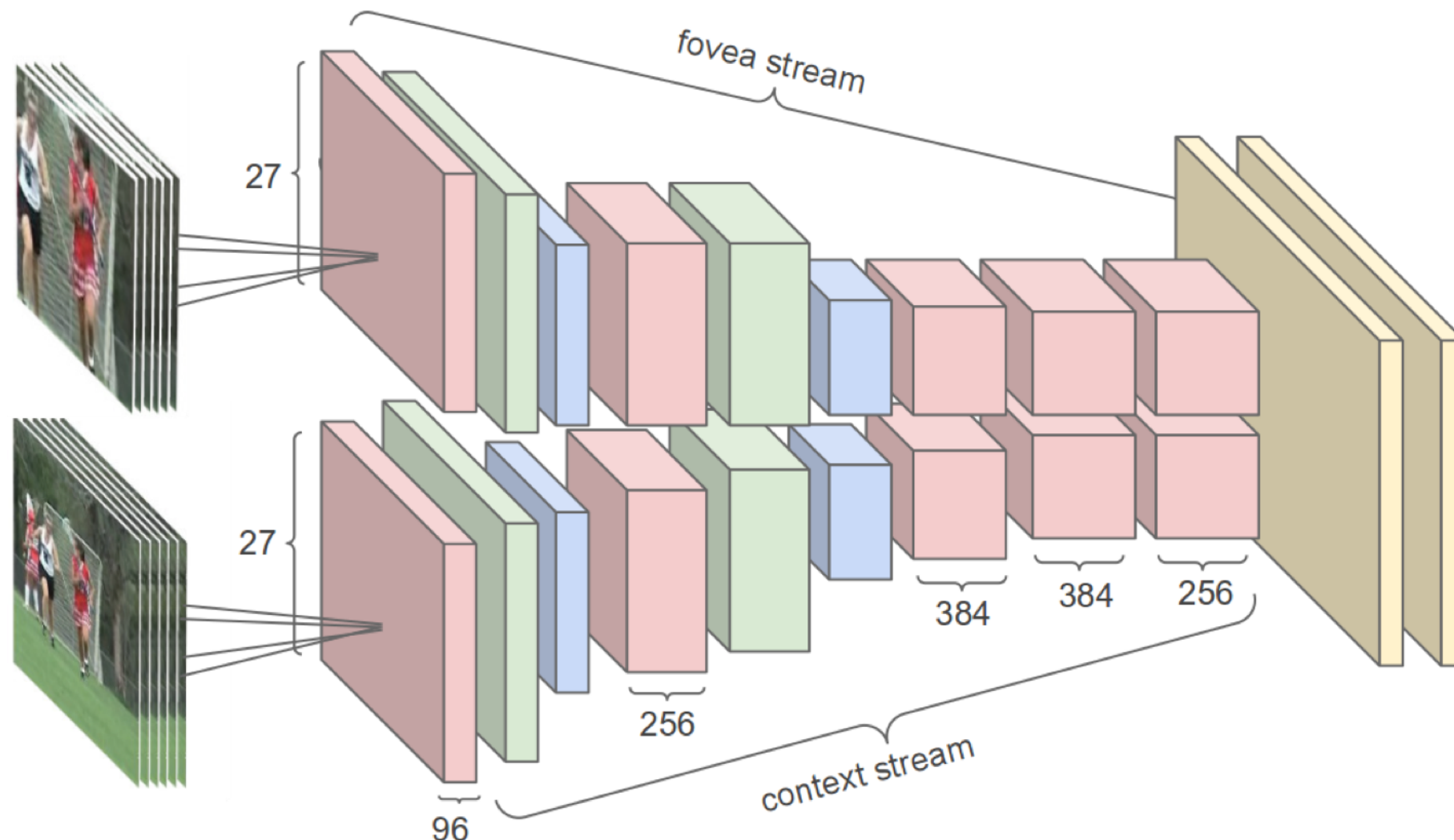
He, K., Gkioxari, G., Dollár, P., & Girshick, R. ICCV 2017.

More architectures ...

- More details? Interaction? Time?
- More “complex” input data

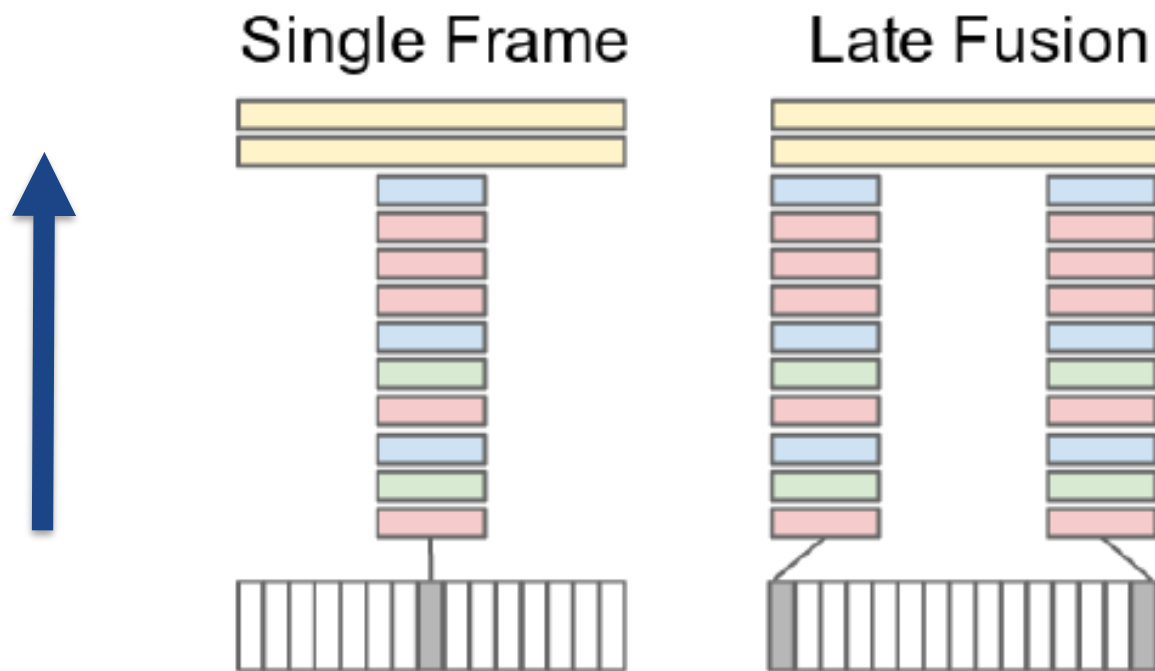
Deep Learning & Video

- Fuse multiple frame info
- high-resolution center crop + low-resolution full image (reduced input size, more efficient training)



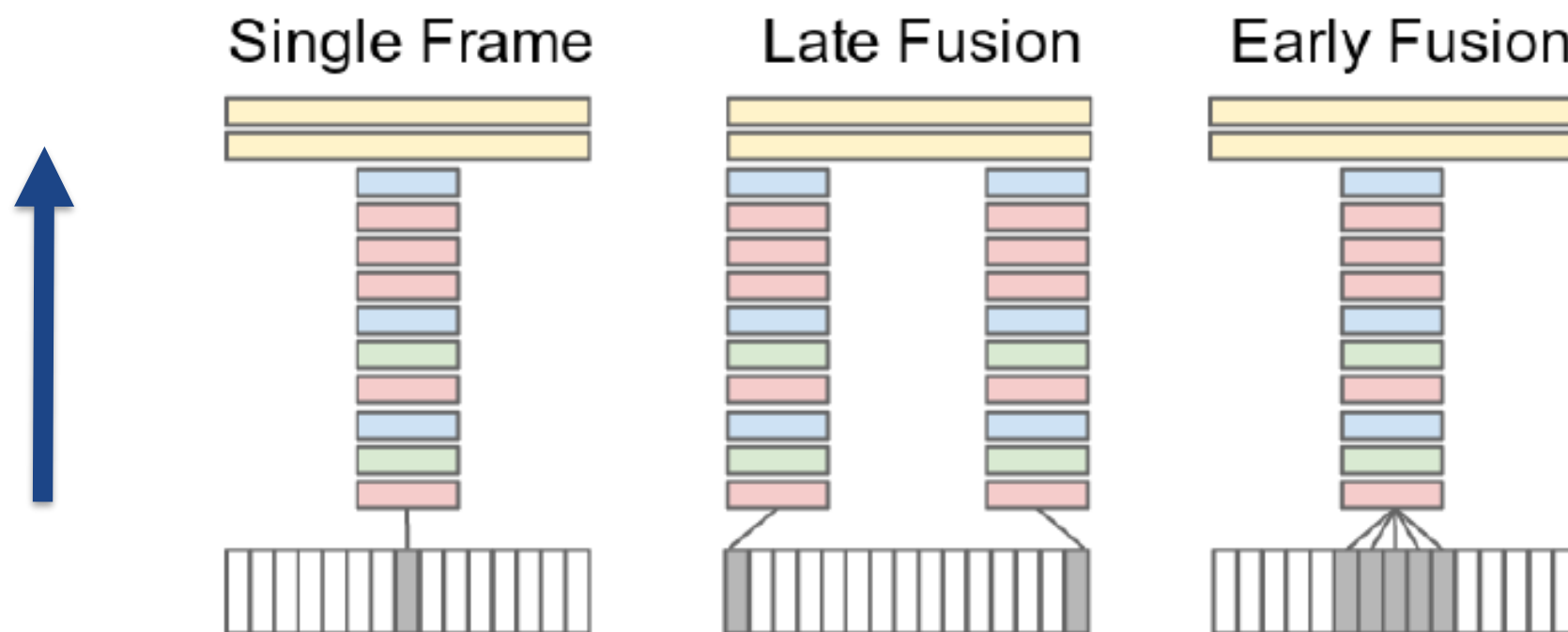
Deep Learning & Video

- Fuse multiple frame info. Many strategies



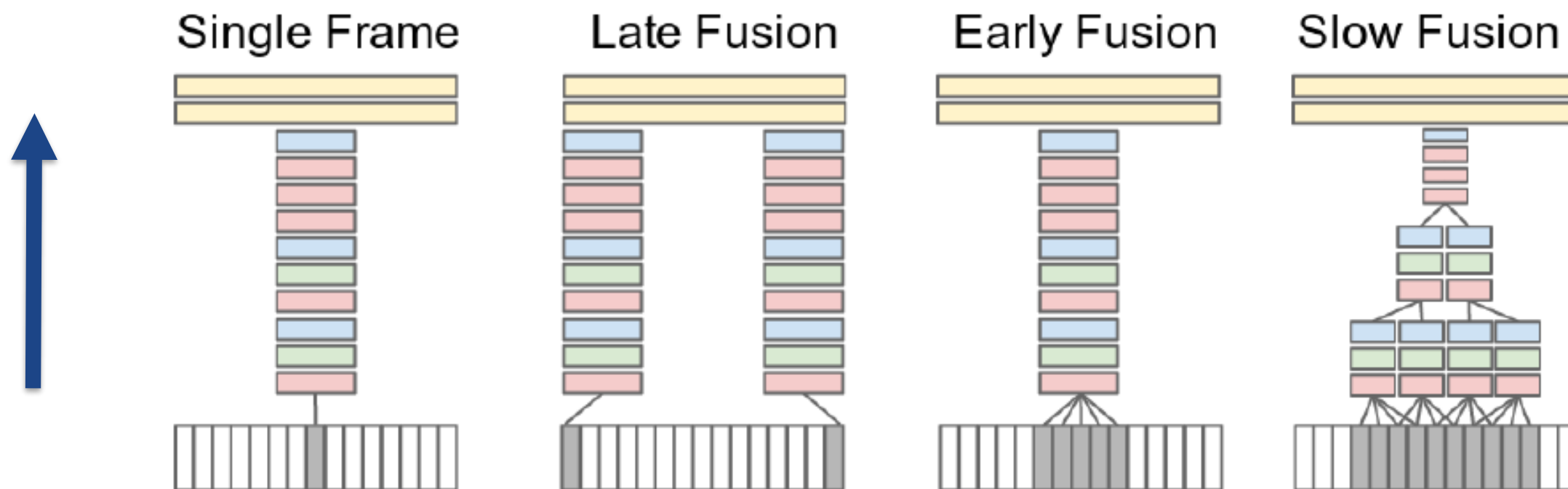
Deep Learning & Video

- Fuse multiple frame info. Many strategies



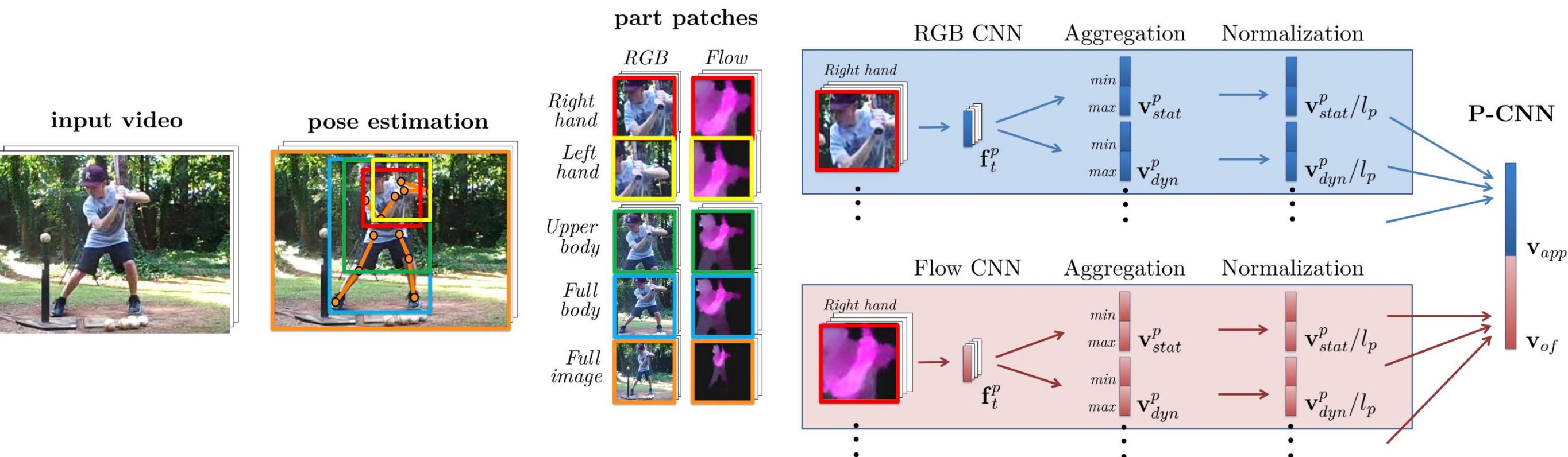
Deep Learning & Video

- Fuse multiple frame info. Many strategies



Deep Learning & Multi-Modal

- Multi-modal input from images
 - Deep learning features: Pose-based CNN
 - Combine **parts**, **pose** and **flow** with CNNs (aggregates motion and appearance information)



Deep Learning & Video

Do you see any problem with this way of treating sequential/video data?

Deep Learning && Video

Do you see any problem with this way of treating sequential/video data?

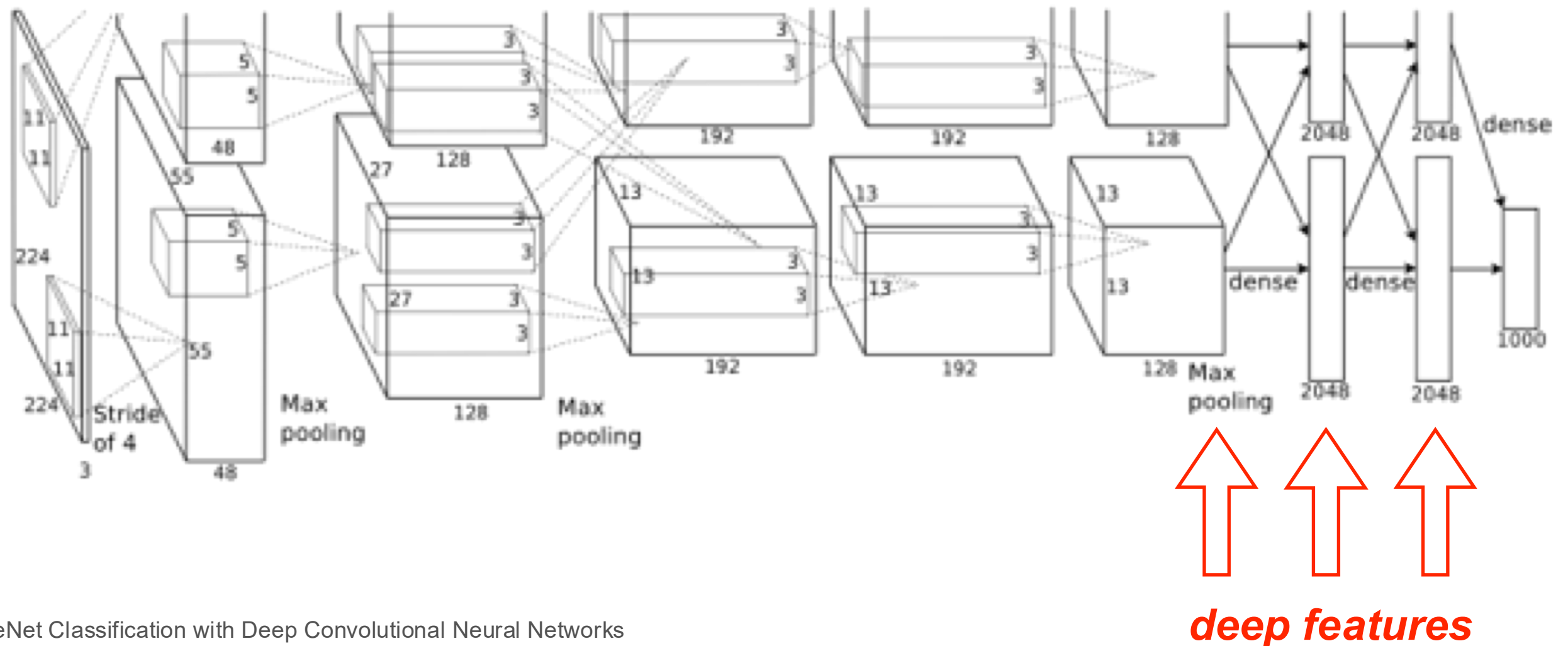
Later in the course: RNN, TCN, Transformers, ...

(More modern-adequate architectures to deal with sequential and multi-modal data)

Summarizing: CNNs & Transfer Learning

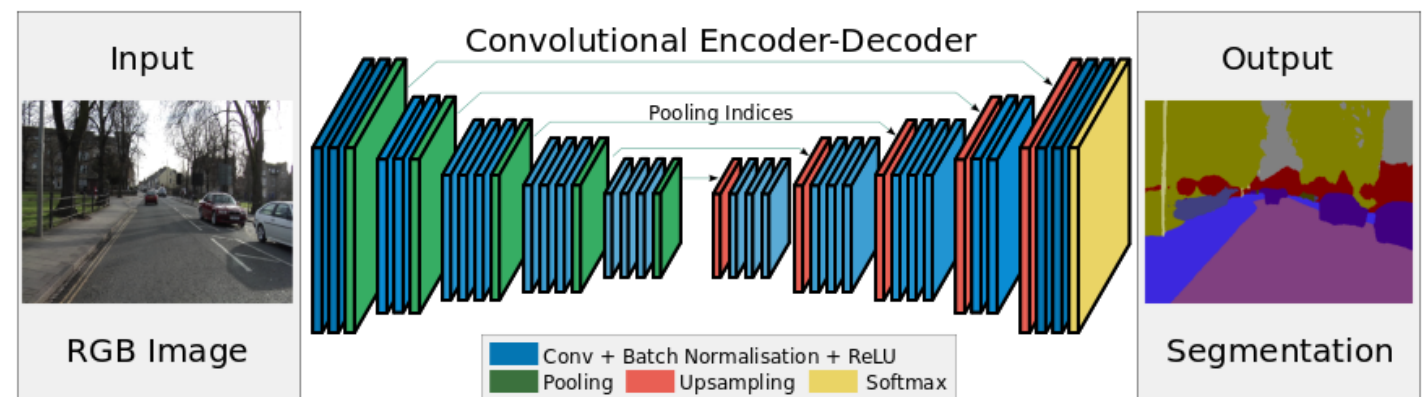
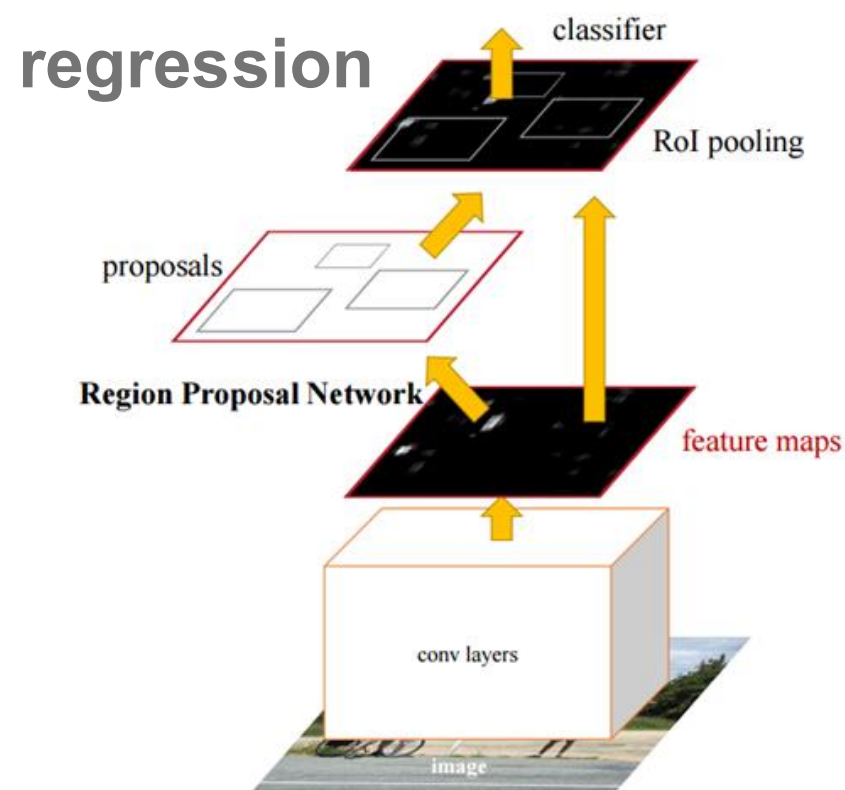
- CNNs are able to generalize well!
 - great **features**
 - **fine-tuning**

In the next Lab
you'll practice
some of this

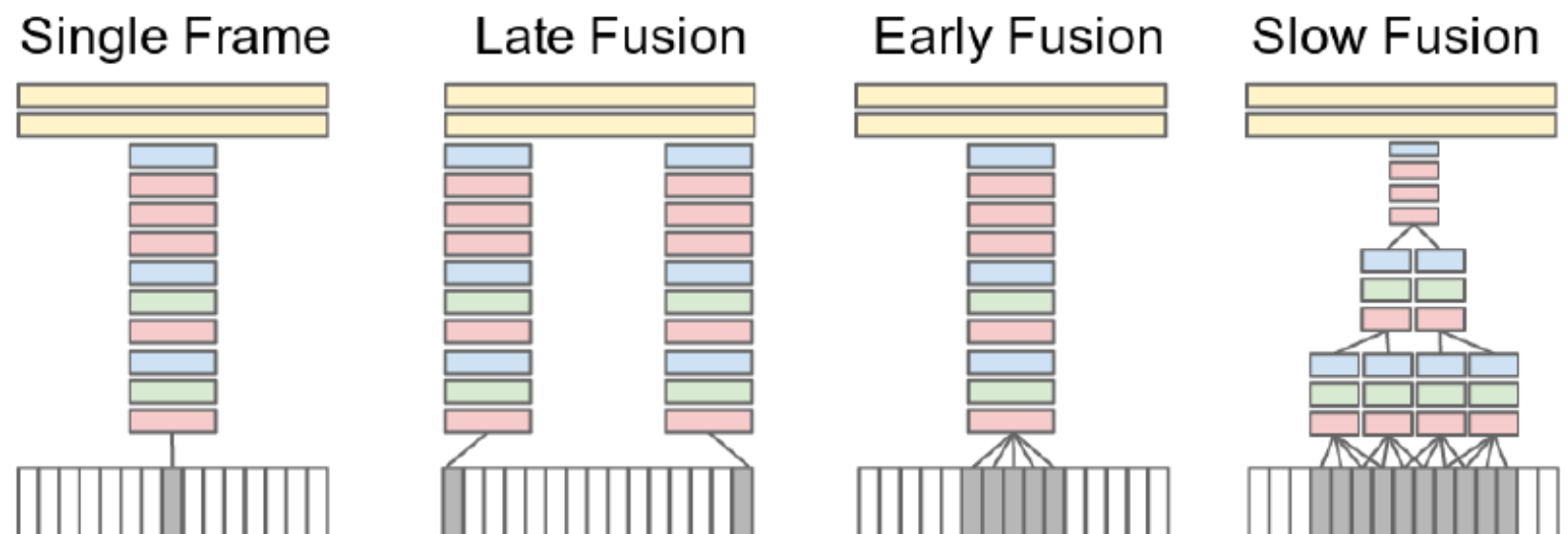


Summarizing - More architectures ...

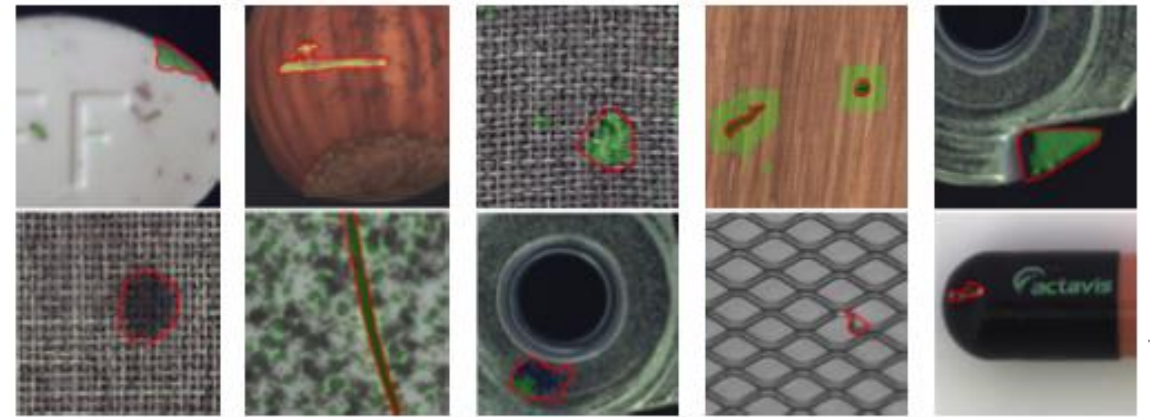
- CNN - Not only image-classification



fusion of multiple
“sources”

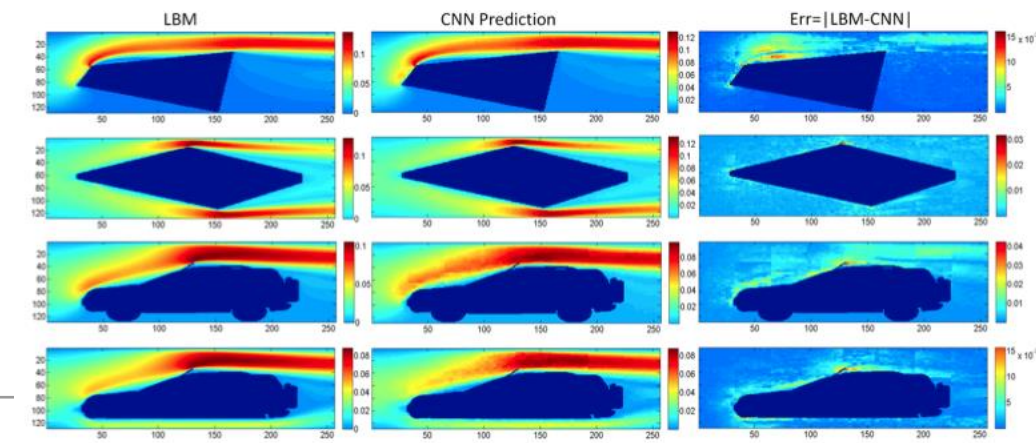


Other applications 1/2



- Industrial inspection (2D)
 - Defects, cracks and scratches: classification, detection or segmentation
 - Few defect examples, so transfer learning and anomaly detection are used
- Sensor signals (1D convolution)
 - Vibration or current signals → motor/bearing fault detection in real time
 - The same operation as our 1D convolution slides
 - Diagnosis using EEG (sleep, epilepsy), ECG
- Medical imaging (3D)
 - CT/MRI segmentation (U-Net)
- Climate data (Liu et al., 2016)
 - Detecting extreme weather events (cyclones, atmospheric rivers) in climate simulation data

Other applications 2/2



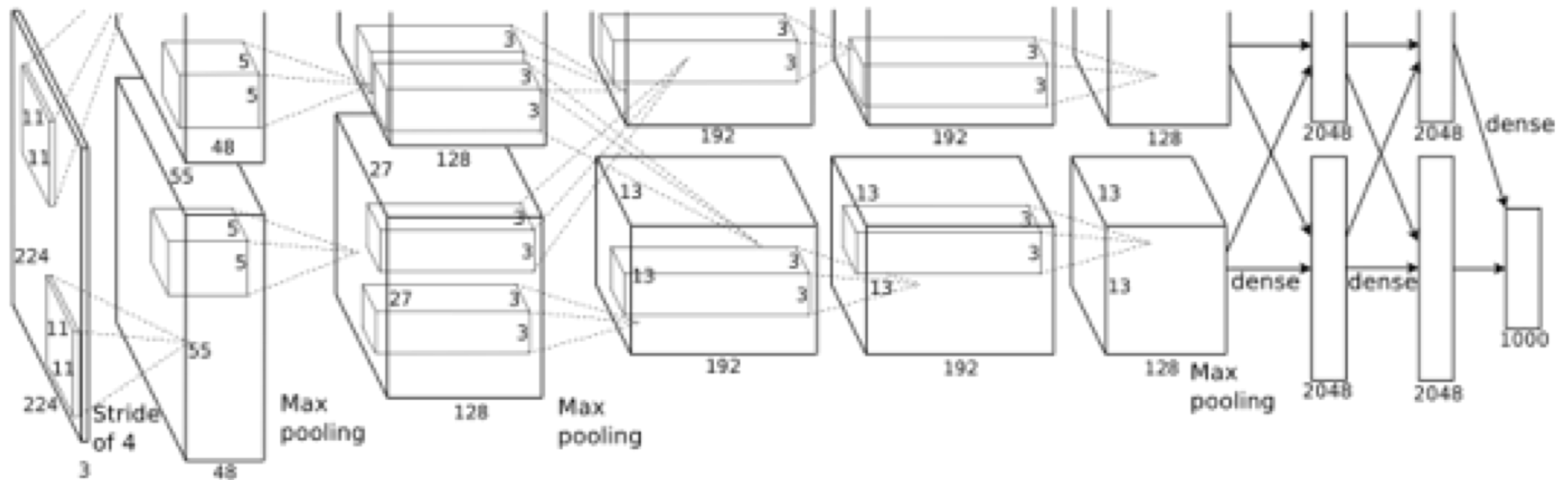
Solver (LBM) vs CNN prediction and error (Guo et al., KDD 2016)

- **Simulation surrogates (2D/3D fields)**
 - Input: geometry on a grid. Output: steady flow field (velocity).
 - CNN encoder–decoder, same idea as SegNet but with regression per pixel.
 - 2 orders of magnitude faster than a GPU CFD solver (4 orders vs CPU).
 - Use: fast design exploration and optimization loops.
- **Materials: microstructure → properties** (Yang et al., Comput. Mater. Sci. 2018)
 - Input: 3D microstructure image. Output: effective stiffness (regression).
 - Trained on finite-element simulations, then replaces them at inference time.
 - **Key idea**
 - Simulation data lives on grids, so convolutions apply directly.
 - Train once on simulations, then predict in milliseconds.

Examples to understand CNNs

- Params? feature map size?

- *How many params.* does the 2nd conv. layer have? and the 5th?



- Input of 240x240x3; Conv1 (48 kernels, 3x3) - Pooling (stride 2) - Conv2 (48 kernels, 3x3) —> **size of feature map after Conv2?**

Some demos to understand CNNs

- karpathy : [DEMO online, CNN](#)

- visualisation (places CNN)

<http://people.csail.mit.edu/torralba/research/drawCNN/drawNet.html>

Visualizing and Understanding Convolutional Neural Networks
Matthew Zeiler and Rob Fergus. **2013**

- <https://poloclub.github.io/cnn-explainer/>

Wang et al., "CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization", IEEE TVCG (VIS 2020)

Lab 3: CNNs applied in Computer Vision

- Understand a **DNN implemented from scratch**
- Implement a toy CNN with Keras
- Fine-tune a well-known CNN architecture with Keras

All this is applied to Image Classification

- Optional tasks:
 - explore semantic segmentation model (with PyTorch)

***Please check
the prior-work
task (prepare
your set up
and data)***

Lab 3: CNNs applied in Computer Vision

- Frameworks
 - **Tensorflow+Keras, Pytorch, ...**
- We will implement a model from scratch for classification
- But then, we will use already existing models.
 - *Retrain them with your dataset*

TO-DO ...

- Lab 3 - **NEXT WEEK (Oct 6)**
- PLEASE have your computer ready with Tensorflow2+Keras (ideally with GPU available) AND/OR we will use Google COLAB

ASSIGNMENT BEFORE YOUR LAB:

1. Pick 5 to 10 classes from one of these datasets (do not take all images from each class if you don't have space)
 - <https://www.kaggle.com/kmader/food41/version/5#>
 - <http://www.robots.ox.ac.uk/~vgg/data/pets/>
 - <http://www.robots.ox.ac.uk/~vgg/data/flowers/>
 - **any other dataset you have?**
2. Put them in folders like —————>
3. Upload to Google Drive if you plan to use COLAB

```
data/  
  dogs/  
    dog001.jpg  
    dog002.jpg  
    ...  
  cats/  
    cat001.jpg  
    cat002.jpg  
    ...
```

Bibliography - Resources for some of the materials today

- Stanford classes on deep learning for Computer Vision (<http://cs231n.stanford.edu>) and Deep Learning (<https://cs230.stanford.edu/>)
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016. <http://www.deeplearningbook.org>
- Deep Learning Summer School Montreal: <https://mila.quebec/en/cours/deep-learning-summer-school-2017/>
- Understanding deep learning, Simon J.D. Prince, MIT Press, <https://udlbook.github.io/udlbook/>
 - Many figures in this presentation come from the materials of this book